

Exemple code

mercredi 4 juin 2025 15:05

GROUP SUPPORT :

Regression Linéaire

Regression Logistique

KNeighbors Classifier

<https://colab.research.google.com/drive/1iL58TzmZuIKDf-Z9ECDE4PGcRpK6kZ5D?usp=sharing>

NLP & Regression Logistique :

<https://colab.research.google.com/drive/1jUCwX8dSOmOCao9tHGQAFdl1y3Wuxcrm?usp=sharing#scrollTo=pgpabP9sZk0n>

A. Sélection des données

SELECT colonne1, colonne2 FROM table WHERE condition;

Fonctions utiles :

- DISTINCT : retire les doublons
- AS : alias
- * : sélectionne toutes les colonnes
- CASE WHEN THEN ELSE END : conditions dans la sélection

SELECT DISTINCT firstname FROM customer;

SELECT lastname AS nom, firstname AS prenom FROM customer;

SELECT name, CASE WHEN active = 1 THEN 'Oui' ELSE 'Non' END AS actif FROM user;

B. Insertion, modification, suppression

INSERT INTO table (col1, col2) VALUES (val1, val2);

UPDATE table SET col1 = val1 WHERE condition;

DELETE FROM table WHERE condition;

Important :

- Toujours spécifier WHERE sauf si volontairement global.

INSERT INTO customer (name) VALUES ('Alice');

UPDATE customer SET email = 'alice@mail.com' WHERE id = 1;

DELETE FROM customer WHERE id = 30;

C. Filtres :

-- Opérateurs : =, <>, !=, >, <, >=, <=

-- Logiques : AND, OR, NOT

-- NULL : IS NULL, IS NOT NULL

-- IN / BETWEEN / LIKE / EXISTS

SELECT * FROM customer WHERE lastname = 'Smith' AND age > 30;

SELECT * FROM product WHERE price BETWEEN 10 AND 20;

SELECT * FROM user WHERE email LIKE '%@gmail.com';

SELECT * FROM orders WHERE EXISTS (SELECT 1 FROM products WHERE orders.product_id = products.id);

D. Tri, Limitation :

ORDER BY colonne [ASC|DESC]

LIMIT n OFFSET m

SELECT * FROM customer ORDER BY lastname ASC LIMIT 10 OFFSET 5;

3. AGRÉGATION & GROUP BY

Fonctions d'agrégat :

- COUNT() : compte les lignes
- SUM() : somme
- AVG() : moyenne
- MIN() / MAX() : valeurs extrêmes
- COUNT(DISTINCT col) : valeurs uniques

SELECT COUNT(*) FROM customer;

SELECT AVG(price) FROM product;

SELECT COUNT(DISTINCT customer_id) FROM orders;

GROUP BY :

SELECT colonne, AGREGAT(colonne2) FROM table GROUP BY colonne;

SELECT lastname, COUNT(*) FROM customer GROUP BY lastname;

HAVING vs WHERE

-- WHERE : filtre ligne avant le GROUP BY
-- HAVING : filtre groupe après GROUP BY
SELECT lastname, COUNT(*) AS nb FROM customer
GROUP BY lastname
HAVING nb > 1;

4. JOINTURES

Types de jointures :

Type	Description
INNER	Données communes entre 2 tables
LEFT	Toutes les lignes de gauche + jointes à droite
RIGHT	Inverse de LEFT
FULL OUTER	Union des LEFT et RIGHT
CROSS	Produit cartésien

SELECT a.name, b.order_date
FROM customer a
INNER JOIN orders b ON a.id = b.customer_id;

5. FONCTIONS UTILES

Chaînes de caractères

- CONCAT(a, b) : concaténation
- UPPER(x) / LOWER(x) : majuscules/minuscules
- SUBSTRING(x, start, length) : extrait
- LENGTH(x) : longueur
- TRIM() : supprime les espaces
- REPLACE(str, old, new) : remplacement

SELECT CONCAT(firstname, ' ', lastname) AS fullname FROM customer;

Dates

- NOW() : date/heure actuelle
- YEAR(), MONTH(), DAY() : extraction
- DATEDIFF(date1, date2) : différence
- DATE_FORMAT(date, '%Y-%m-%d') : format
- CURRENT_DATE, CURRENT_TIME : date/heure actuelles
- EXTRACT() : extrait une partie d'une date

SELECT MONTH(order_date), COUNT(*) FROM orders GROUP BY MONTH(order_date);

6. CTE - Common Table Expressions

Les CTE permettent de nommer des sous-requêtes pour améliorer lisibilité et réutilisation.

WITH total_sales AS (
SELECT customer_id, SUM(amount) AS total
FROM sale
GROUP BY customer_id)
SELECT * FROM total_sales WHERE total > 1000;

CTE récursives

WITH RECURSIVE nombre(n) AS (
SELECT 1
UNION ALL
SELECT n + 1 FROM nombre WHERE n < 10
)

SELECT * FROM nombre;

7. WINDOW FUNCTIONS

Permet de calculer des agrégats sans grouper.

Fonctions disponibles :

- ROW_NUMBER()
- RANK(), DENSE_RANK()
- NTILE(n) : divise en n groupes
- LAG(), LEAD() : accès aux valeurs précédentes/suivantes
- SUM() OVER(...)
- AVG() OVER(...)

Exemple :

```
SELECT customer_id, sale_id,  
       ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY sale_date) AS rang  
FROM sale;
```

-- Moyenne mobile sur 3 lignes glissantes

```
SELECT customer_id, amount,  
       AVG(amount) OVER(PARTITION BY customer_id ORDER BY sale_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS  
moyenne_mobile  
FROM sale;
```

CHEATSHEET SQL ESSENTIELLE ✨



Commandes fondamentales

- SELECT, INSERT, UPDATE, DELETE
- WHERE, ORDER BY, LIMIT, OFFSET
- GROUP BY, HAVING
- DISTINCT, AS, CASE WHEN, IN, BETWEEN, LIKE, EXISTS, IS NULL



Fonctions d'agrégat

- COUNT(*), COUNT(DISTINCT col)
- SUM(col), AVG(col)
- MIN(col), MAX(col)



Fonctions texte

- CONCAT(), UPPER(), LOWER()
- SUBSTRING(), LENGTH()
- TRIM(), REPLACE()



Fonctions date

- NOW(), CURRENT_DATE, CURRENT_TIME
- YEAR(), MONTH(), DAY(), EXTRACT()
- DATEDIFF(), DATE_FORMAT()



Jointures

- INNER JOIN, LEFT JOIN, RIGHT JOIN
- FULL OUTER JOIN, CROSS JOIN



Avancé

- WITH, WITH RECURSIVE (CTE)
- ROW_NUMBER() OVER(...), RANK(), DENSE_RANK()
- NTILE(n), LAG(), LEAD()
- SUM() OVER(...), AVG() OVER(...)
- EXPLAIN pour diagnostic de performance

Numpy / Pandas

lundi 26 mai 2025 14:00

https://colab.research.google.com/drive/1k2Wu_F1N3KT1b6N2chAE9hfiZMq6kuVv#scrollTo=up5pXsM-gbik

```
# We can set numbers for how many rows and columns will be displayed
pd.set_option('display.min_rows', 10) #default will be 10
pd.set_option('display.max_columns', 20)

# Read csv file from url
url="https://github.com/MaskiVal/Html/raw/main/online_store_customer_data.csv"
df = pd.read_csv(url)
df.head(3)

# display first 3 rows
df.head(3)
# display last 6 rows
df.tail(6)

# Display random 7 sample rows
df.sample(7)

# Display info DataFrame
df.info()

# display datatypes
df.dtypes

# compter le nombre d'occurrences par objets
df.dtypes.value_counts()

# Montre la forme du DataFrame
df.shape

# Montre les colonnes du DataFrame
df.columns

# display Age columns first 3 rows data
df["Age"].head(3)

# display first 4 rows of Age, Transaction_date and Gender columns
df[["Age", "Transaction_date", "Gender"]].head(4)

# for display 2nd to 6th rows
display(df[1:6])

# for display starting to 10th
display(df[9:])

# for display last two rows
display(df.iloc[-2,:])

# Montre les colonnes entières du DataFrame dont le genre est "Male"
display(df.loc[df["Gender"]=="Male",:])

# Supprimer une colonne que nous n'avons pas besoin 'Transaction_ID'
df.drop("Transaction_ID", axis = 1, inplace = True)

# create new df dataframe from df.copy() method.
df2 = df.copy()

# renommer les colonnes
df_col.rename(columns={"Transaction_date": "Date", "Gender": "Sex"}, inplace = True)
df_col.head(3)

# Add a new ajusted column which value will be Amount_spent * 100
df_col["new_col"] = df_col.Amount_spent * 100

# changing Female to Woman and Male to Man in Sex column.
#first argument in loc function is condition and second one is columns name.
df_col.replace("Female", "Woman", inplace = True)
df_col.replace("Male", "Man", inplace = True)

# change le type d'un objet en datetime64 format
df_col["Date"] = df_col["Date"].astype("datetime64[ns]")

# change de float64 à float32 of Referral columns
df_col["Referral"] = df_col["Referral"].astype("float32")

### 1) Quelle est la forme la plus représentée?
df_ovni['Shape Reported'].value_counts().head(1)
df_ovni['Shape Reported'].mode().head(1)[0]

### 2) Quelle est la ville où l'on a le plus de rapports de forme « FIREBALL » ?
df_ovni['City'][df_ovni['Shape Reported'] == 'FIREBALL'].value_counts().head(1)

### 3) Quel est le pourcentage de rapports où la couleur est présente ? remplacez
les valeurs de couleurs manquantes par « NONE »
round(100 - (df_ovni['Colors Reported'].isna().sum() / len(df_ovni) * 100), 2)
df_ovni['Colors Reported'].notna().mean() * 100
df_notna = df_ovni[df_ovni['Colors Reported'].notna()]
df_notna = df_ovni[~(df_ovni['Colors Reported'].isna())]
df_ovni.fillna('None')

### 4) Quelle est la décennie où l'on a recensé le plus de cas d'ovnis ?
df_ovni['Time'][0].split('/')[2][:3] + '0'
df_ovni['Decade'] = df_ovni['Time'].apply(lambda x: x.split('/')[2][:3] + '0')
ou
from datetime import datetime
df_ovni['datetime_dec'] = df_ovni['Time'].apply(lambda x: datetime.strptime(x, '%m/%d/%Y %H:%M').year//10*10)
df_ovni['Decade'].value_counts().head(1)

### 5) Affichez le top 10 des villes où l'on voit le plus d'OVNIS en excluant New York
top_10 = df_ovni['City'][df_ovni['City'] != 'New York']
top_10.value_counts().head(10)
top_10

### 6) Afficher un dataframe de ces 10 villes avec les décennies, où les valeurs
seront le nombre d'ovnis par combinaison
df_TOP10 = pd.pivot_table(df_ovni, index='City', columns='Decade', values='State',
aggfunc='count')
df_TOP10.fillna(0, inplace=True)
df_TOP10[df_TOP10.index.isin(top_10.index)]
```

```
from datetime import datetime
from datetime import timedelta
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
NUMPY :
array.ndim
array.shape
array.size
array.dtype
array.min() donne le min
array.max() donne le max
array.mean() donne la moyenne
np.median(array) donne la medienne
array.reshape(ligne,colonne) modifie la forme de la matrice
np.zeros(5) 1 dimension np.zeros((3,4)) 2 dimensions
np.ones(5) 1 dimension np.ones((5,5)) 2 dimensions
np.full((3,3),7) tableau rempli de 7
np.eye(3) matrice identité 3 x 3
np.diag([1,2,3]) matrice diagonale 1 2 3
np.empty()
np.random.random()
np.random.randn(moyenne, ecart-type, nbr de numéro)
np.absolute(array) donne les valeurs absolues de array (que des valeurs positives)
```

Pour convertir en array array = np.array(liste, tuple ...) retourne une array

array ou df .sum(axis = 0) somme des colonnes
array ou df .sum(axis =1) somme des lignes

np.resize(array(2,2)) modifie l'array ou a.resize() modifie l'array

flatten pour aplatir l'array (1 ligne)

array.T Transposition - transforme ligne en colonne

a.sort() modifie l'array

np.sort(array, axis = 0 ou 1) ne modifie pas l'array

p.insert(array, index, valeur) ou np.append(a,b)
np.concatenate()

np.hstack(array, colonne.shape(...),...) empile horizontalement voir la doc.
np.vstack(array, colonne.shape(...),...) empile verticalement voir la doc.

Slicing :
array = (2,3) recupère 1 élément unique
array = [5] recupère 1 la ligne entière
array = [:,2] recupère 1 colonne complète
array = [1:4,1:4] recupère sous matrice [ligne,colonne]

Afficher tous les chiffres d'un array >4 : array[array > 4]
PANDAS :

df pour DataFrame

Transformer un array en dataframe : df=pd.DataFrame(array ou list ou dict)
Importer un fichier pd.read_csv('adresse ou nom du fichier')

df.head()
df.tail()
df.info

df.rename(columns={'col1':'nom_1','col2':'nom_2, etc ...}, inplace=True) pour renommer les entêtes de colonnes

df['colonne'].describe permet décrire la colonne

df.value_counts() compte le nbre d'occurrences de chaque valeur

Pour convertir une 'str' date en date et connaître le type : pd.to_datetime(df['colone_date']).dtypes

Supprimer une ligne du df : df.drop(nbreindex, axis = 0, inplace = True)
Supprimer une colonne du df : df.drop(['colonne'], axis = 1, inplace = True)

df.sample(7) génère un échantillon de 7 lignes aléatoires dans le df
df.dtypes aussi df.select_dtypes('l'objet')
df.columns voir les différentes colonnes
df['col'] affiche la colonne
df.shape(1 ou 0) nombre de lignes ou de colonnes
len(df) nombre de lignes
df.describe()
df.isnull().sum() voir le nombre de valeur null

Df_ 2 = nom du df à copier.copy()

[df,['col1','col2', ...] selectionne les colonnes

df.iloc[10 : 20] selectionne les lignes de 10 à 20 exemple df.iloc[0 : 10]
df.iloc[:, ['col1','col2','col3', ...] selectionne les colonnes col1', 'col2', 'col3'

df.loc[:, 'col1' : 'coln'] selectionne les colonnes entre 'col1' : 'coln'
df.loc[df['colonne'] > 10, ['col1', 'col5']] selectionne les lignes > 10 et uniquement les 'col1', 'col5'

df.groupby(['col1','col2', ...]).mean() Groupe selon le filtre et calcule l'agrégation (sum,cumsum,std etc)

df.isna().sum fait la somme des valeurs null du df
df['colonne'].isna().sum fait la somme des valeurs null de la colonne
df[['col1','col2']].isna().sum fait la somme des valeurs null de les colonnes selectionnées
df.dropna() supprime les ligne avec valeurs null
df.fillna(value = ...) remplace les valeurs null par ...

Filtrer des données : df[df['colonne'] > 10]

Faire un calcul sur une colonne : `df['nouvelle_colonne'] = df['colonne1'] * df['colonne2']`
ex somme cumulée : `df['sommecumulée'] = df['CA'].cumsum()`

Changer le nom dans une colonne `df.replace('Female', 'Woman', inplace = True)` inplace sert à modifier le df

Appliquer une fonction : `df['calcul'] = df['colonne'].apply(nom_de_la_fonction)`

Faire un histogramme d'un df : `df.hist('colonnes')` ou `df.prix.hist()`

Filtrer une colonne > moyenne : `moy = df.nom_colonne.mean()`
`df[df['colonne'] > moy]`

Date dans PYTHON

`from datetime import datetime`

`Datetime.strptime()` : convertie une chaîne de caractère en format date

`Datetime.strftime()` : convertie un format date en chaîne de caractère

Exemple : `date = datetime.strptime('2018-01-04', '%Y - %m - %d')`
pour l'afficher au format français
`date_2 = datetime.strftime(date, '%d- %m - %Y')`

Date dans PANDAS

Le plus simple utiliser : `pd.to_datetime(['colonne_date'])`

Si la valeur n'est pas standard, il est difficile d'adapter le format, dans ce cas :
`df['DATE'].apply(lambda x : datetime.strptime(x, '%Y - %m - %d'))`

Opération sur les dates :
`from datetime import timedelta`
`print(datetime.now() + timedelta(days=4))`

Exemple : `df_weather['date'] = pd.to_datetime(df_weather['date'])`

Formater une date :
Énoncé donné : 2018-01-04 at 10 heure et 30 min attention les espaces sont compris dans le code
`df.replace(' at ', '').replace('heure et ', ':').replace(' min', '')`

Pour extraire exemple le mois d'une date : `df['Date'].dt.month`

`df.to_parquet('nom du fichier.parquet', index=False)` enregistrer le fichier parquet
`df.to_sql('table', id_connexion)` enregistrer le fichier sql

Nettoyage des données MERGE(jointures) ET FILL MISSING VALUE

lundi 26 mai 2025 14:09

```
# Fusionner 2 DataFrame
Result = pd.merge(left, right, how='left', on=['key1', 'key2'])
ou
df2018=pd.merge(df_main,df_opinion,how='left', left_on='DATE',right_on='date')
# La fonction isna() permet d'identifier les valeurs manquantes
dataframe.isna().sum()

# Supprimer 1 colonne si la majorité des valeurs sont manquantes
dataframe.drop(columns=['Bonus'], inplace=True)

# Supprimer une ligne si elle contient beaucoup de NaN
dataframe.dropna(subset=['Salaire', 'Département', 'Nom'], # Sélection des
colonnes à vérifier
                how="all", # Supprimer la ligne seulement si toutes les
colonnes spécifiées sont NaN
                inplace=True) # Modifier directement le DataFrame sans créer
une copie

# Remplacer les valeurs manquantes par une valeur fixe
df['Age'].fillna(df['Age'].median(), inplace=True) # ou mean ou mode
#autre syntaxe
df['Age'] = df['Age'].fillna(df['Age'].median())
ou
dataframe['Département'].fillna('Inconnu', inplace=True)
# Pour traiter les valeurs manquantes des variables catégorielles,
# vous pouvez les remplacer par la valeur la plus fréquente (mode), car elle
représente la catégorie la plus courante.
mode_value = data['column_name'].mode()[0]
data['column_name'] = data['column_name'].fillna(mode_value)

# Remplacer les valeur manquantes par une valeur variable
# Les méthodes "ffill" Forwardfill et "bfill" Backwardfill sont pratiques dans
ce cas.
# Appliquer ffill (forward fill) sur la colonne 'Température (°C)' // index 0
à ma fin
df['Température (°C)'] = df['Température (°C)'].ffill()
# Appliquer bfill (backward fill) sur la colonne 'Humidité (%)' // index de
fin à 0
df['Humidité (%)'] = df['Humidité (%)'].bfill()
```

MATPLOTLIB/SEABORN

lundi 26 mai 2025 14:10

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
# Scatter plot of sepal length vs sepal width -- scatter plot de sepal length vs sepal width
plt.figure(figsize=(8, 6))
for species, color in zip(df['species'].unique(), ['red', 'green', 'blue']):
    species_data = df[df['species'] == species]
    plt.scatter(species_data['sepal_length'], species_data['sepal_width'], color=color,
                label=species)
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Sepal Length vs Sepal Width')
plt.legend()
plt.grid(True)
plt.show()
```

```
# Histogram of petal length -- histogramme de la longueur du pétale
plt.figure(figsize=(8, 6))
for species, color in zip(df['species'].unique(), ['red', 'green', 'blue']):
    species_data = df[df['species'] == species]
    plt.hist(species_data['petal_length'], bins=20, color=color, alpha=0.5, label=species)
plt.xlabel('Petal Length (cm)')
plt.ylabel('Frequency')
plt.title('Distribution of Petal Length')
plt.legend()
plt.grid(True)
plt.show()
```

```
# Box plot of sepal length -- boîte à moustaches de la longueur du sépale
plt.figure(figsize=(8, 6))
plt.boxplot([df[df['species'] == species]['sepal_length'] for species in
df['species'].unique()],
            labels=df['species'].unique())
plt.xlabel('Species')
plt.ylabel('Sepal Length (cm)')
plt.title('Distribution of Sepal Length by Species')
plt.grid(True)
plt.show()
```

```
# Exemple
plt.figure(figsize=(14,5))
plt.bar(view.index, view['MAX_TEMPERATURE_C'],color='skyblue',edgecolor='black')
plt.plot(view.index,view['SUNHOUR'], color='#FF3333', marker='o', linestyle='-')
plt.title('Max températures')
plt.xlabel('Mois')
plt.ylabel('Températures')
plt.show()
```

```
# Create a figure with multiple subplots -- Créer une figure avec plusieurs sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
```

```
# Scatter plot of sepal length vs sepal width -- scatter plot de sepal length vs sepal width
for ax, species, color in zip(axs.flat, df['species'].unique(), ['red', 'green', 'blue']):
    species_data = df[df['species'] == species]
    ax.scatter(species_data['sepal_length'], species_data['sepal_width'], color=color,
                label=species)
    ax.set_xlabel('Sepal Length (cm)')
    ax.set_ylabel('Sepal Width (cm)')
    ax.set_title(f'{species} - Sepal Length vs Sepal Width')
    ax.legend()
    ax.grid(True)
```

```
# Adjust layout to prevent overlap
plt.tight_layout()
```

```
# Show the plot
plt.show()
```

```
# Convert the 'Date' column to datetime format -- Convertir la colonne 'Date' au format
datetime
stocks_df['Date'] = pd.to_datetime(stocks_df['Date'])
```

```
# Plot the stock prices, dividends, and earnings over time -- Tracer les prix des actions,
les dividendes et les bénéfices au fil du temps
plt.figure(figsize=(12, 8))
# S&P 500 index prices -- prix de l'indice S&P 500
```

```
plt.plot(stocks_df['Date'], stocks_df['SP500'], label='S&P 500', color='blue',
         linestyle='-')
# Dividends -- Dividendes
```

```
plt.plot(stocks_df['Date'], stocks_df['Dividend'], label='Dividend', color='green',
         linestyle='--')
```

```
# Earnings -- Bénéfices
plt.plot(stocks_df['Date'], stocks_df['Earnings'], label='Earnings', color='red',
         linestyle='-.')
plt.xlabel('Date')
plt.ylabel('Value')
plt.title('S&P 500 Index, Dividends, and Earnings Over Time')
plt.legend()
plt.grid(True)
plt.show()
```

```
# Set the style -- Définir le style qui est ici "whitegrid"
sns.set_style("whitegrid")
```

```
# Scatter plot of sepal length vs sepal width, colored by species -- scatter plot de sepal
length vs sepal width, coloré par espèce
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='sepal_length', y='sepal_width', hue='species', palette='Set1',
               legend='full')
plt.title('Sepal Length vs Sepal Width (Colored by Species)')
```

```
*** GROUPE SUPPORT ***
```

```
# Graphique à barre matplotlib / barplot
plt.figure()
plt.bar(x=df_students["Nom"], height = df_students["Note_Maths"])
plt.title("Notes de maths")
plt.ylabel("Notes")
plt.show()
```

```
# Graphique à barre matplotlib/ barplot avec plusieurs couleurs
plt.figure()
colors = {'1A' : 'red', '1B' : 'orange', '1C' : 'skyblue'}
for classe in df_students['Classe'].unique():
    data = df_students[df_students['Classe'] == classe]
    plt.bar(data['Nom'], data['Note_Maths'],
            color=colors[classe], label=classe)
plt.title("Notes de maths")
plt.ylabel("Notes")
plt.legend()
plt.show()
```

```
# Graphique à barre SEABORN/ barplot avec plusieurs couleurs
plt.figure()
sns.barplot(data = df_students.sort_values("Classe"), x = 'Nom',
            y = 'Note_Maths', hue = 'Classe', palette = 'magma')
plt.title("Notes de maths")
plt.ylabel("Notes")
plt.legend()
plt.show()
```

```
# Histogramme SEABORN/ HIST
sns.histplot(data = df_students, x = 'Note_Anglais', bins=20)#,
            hue = 'Classe', multiple = "dodge", bins=20)
plt.show()
```

```
# Boite a moustache SEABORN/ BOXPLOT
sns.boxplot(data=df_students, x = 'Classe', y = 'Note_Histoire',
            hue = 'Classe', palette = "magma")
plt.show()
```

```
# Boite a moustache avec le calcul d'un moyenne sur colonnes
SEABORN/ BOXPLOT
df_students['Moyenne'] = df_students.iloc[:, -3:].mean(axis = 1)
sns.boxplot(data = df_students, x = 'Classe', y = 'Moyenne', hue
            = 'Classe', palette = "magma")
```

```
# Nuage de point + regression linéaire SEABORN / SCATTERPLOT
fig, ax = plt.subplots()
sns.scatterplot(ax = ax, data=df_students, x = 'Note_Histoire', y
            = 'Note_Anglais', hue='Classe', palette = 'magma')
sns.regplot(ax = ax, data=df_students, x = 'Note_Histoire', y =
            'Note_Maths', scatter = False)
ax.set_xlabel("Note d'histoire")
ax.set_ylabel("Note d'anglais")
fig.show()
```

```
# camembert avec value_counts() Matplotlib / PIE
plt.figure()
x = df_students['Classe'].value_counts()
labels = x.index
explode = [0.1,0,0]
colors = sns.color_palette("magma", len(labels))
plt.pie(x, labels = labels, autopct="%1.1f%%", explode=explode,
        colors = colors)
plt.title("Répartition des élèves par classe")
plt.show()
```

```

plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.legend(title='Species')
plt.show()

# Box plot of sepal length, colored by species -- boîte à moustaches de la longueur du
# sépale, colorée par espèce
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, hue='species', y='sepal_length', palette='pastel')
plt.title('Distribution of Sepal Length by Species')
plt.xlabel('Species')
plt.ylabel('Sepal Length (cm)')
plt.show()

# Calculate correlation matrix for numerical variables -- Calculer la matrice de corrélation
# pour les variables numériques
numerical_corr_matrix = df[['sepal_length', 'sepal_width', 'petal_length',
'petal_width']].corr()

# Create a correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(numerical_corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap (Numerical Variables)')
plt.show()

# Create a mask to hide the upper triangle -- Créer un masque pour masquer le triangle
# supérieur
mask = np.triu(np.ones_like(numerical_corr_matrix, dtype=bool))

# Create a correlation triangle matrix -- Créer une matrice de corrélation triangulaire
plt.figure(figsize=(8, 6))
sns.heatmap(numerical_corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", mask=mask,
linewidths=.5)
plt.title('Correlation Triangle Matrix (Numerical Variables)')
plt.show()

# Create a pair plot with hue as the 'species' column -- Créer un graphique pair avec la
# couleur de la colonne 'species'
sns.pairplot(df, hue='species', palette='Set1')
plt.title('Pair Plot of Iris Dataset (Colored by Species)')
plt.show()

# Set the style -- Définir le style
sns.set_style("whitegrid")

# Create a figure with multiple subplots -- Créer une figure avec plusieurs sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Scatter plot of sepal length vs sepal width, colored by species -- scatter plot de sepal
# length vs sepal width, coloré par espèce
sns.scatterplot(data=df, x='sepal_length', y='sepal_width', hue='species', palette='Set1',
ax=axs[0, 0])
axs[0, 0].set_title('Sepal Length vs Sepal Width (Colored by Species)')
axs[0, 0].set_xlabel('Sepal Length (cm)')
axs[0, 0].set_ylabel('Sepal Width (cm)')
axs[0, 0].legend(title='Species')

# Histogram of petal length, colored by species -- histogramme de la longueur du pétale,
# coloré par espèce
sns.histplot(data=df, x='petal_length', hue='species', palette='Set2', bins=20, kde=True,
ax=axs[0, 1])
axs[0, 1].set_title('Distribution of Petal Length (Colored by Species)')
axs[0, 1].set_xlabel('Petal Length (cm)')
axs[0, 1].set_ylabel('Frequency')
axs[0, 1].legend(title='Species')

# Box plot of sepal length, colored by species -- boîte à moustaches de la longueur du
# sépale, colorée par espèce
sns.boxplot(data=df, x='species', y='sepal_length', palette='pastel', ax=axs[1, 0])
axs[1, 0].set_title('Distribution of Sepal Length by Species')
axs[1, 0].set_xlabel('Species')
axs[1, 0].set_ylabel('Sepal Length (cm)')

# Remove empty subplot -- Supprimer le sous-graphique vide
fig.delaxes(axs[1, 1])

# Adjust layout to prevent overlap -- Ajuster la disposition pour éviter le chevauchement
plt.tight_layout()

# Show the plot -- Afficher le graphique
plt.show()

# Line plot of S&P 500 index, dividends, and earnings over time with seaborn -- Tracer les
# prix des actions, les dividendes et les bénéfices au fil du temps avec seaborn
plt.figure(figsize=(12, 8))

# S&P 500 index prices -- prix de l'indice S&P 500
sns.lineplot(data=stocks_df, x='Date', y='SP500', label='S&P 500', color='blue')

# Dividends -- Dividendes
sns.lineplot(data=stocks_df, x='Date', y='Dividend', label='Dividend', color='green')

# Earnings -- Bénéfices
sns.lineplot(data=stocks_df, x='Date', y='Earnings', label='Earnings', color='red')
plt.xlabel('Date')
plt.ylabel('Value')
plt.title('S&P 500 Index, Dividends, and Earnings Over Time')
plt.legend()
plt.show()

# vous ajouterez une légende, des graduations d'axe lisibles et un titre (il n'y a pas de
# fonction seaborn pour cela, utilisez la syntaxe matplotlib)
plt.rcParams['figure.figsize']=(10,4)
sns.set(font_scale=1)
sns.scatterplot(data = df_weather, x = 'DATE', y = 'MAX_TEMPERATURE_C', hue =
'SUNHOUR', palette = sns.color_palette("flare", as_cmap=True))
plt.legend(title="OPINION", labels=['very bad', "bad", "not goog not bad", "good", "very
good"], loc=1)

```

```

plt.ylabel('Températures')
plt.title('Max temperature, par mois et par avis')

# réaliser un heatmap
df_weather = df_weather.select_dtypes('number')
sns.heatmap(df_weather.corr(), cmap='Spectral', center=0, square=True, xticklabels=1,
yticklabels=1)
plt.show()

#pairplot version 1
sns.pairplot(data = df_weather.sample(n = 10), vars
=['MAX_TEMPERATURE_C', 'MIN_TEMPERATURE_C', 'HUMIDITY_MAX_PERCENT',
'PRESSURE_MAX_MB', 'DAY', 'SUNHOUR',
'WINDSPEED_MAX_KMH', 'TEMPERATURE_MORNING_C', 'TEMPERATURE_NOON_C', 'VISIBILITY_AVG_KM'],
hue='OPINION', palette="bright")
plt.show()

#pairplot version 2
df_weather_part=df_weather.iloc[:,-10:] #slice des colonnes des 10 dernières colonnes
incluant la colonne OPINION
#display(df_weather_part)
df_weather_sns=df_weather_part.sample(n = 10) # créons un pairplot à partir d'un
echantillon de 10 lignes aléatoires dans notre dataset
sns.pairplot(df_weather_sns,hue='OPINION', palette="bright" ) # dessiner un Pairplot à
partir d'un échantillon, et colorier and par OPINION
plt.title("pairplot colorié par different OPINION")
plt.show()

# Regression linéaire
sns.regplot(data=df_weather, x="TEMPERATURE_NOON_C", y='MAX_TEMPERATURE_C') # the exercice
above find that Temperature_Moon_c have positive correlation env 0.98
plt.title("températures à midi en fonction des températures maximales ")
plt.show()

#Créer une figure avec plusieurs sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(20, 14))
# Définition du premier graphique

# Scatter plot of month vs Temperature_max -- scatter plot de max temp par mois
sns.scatterplot(data=df_weather, x='MONTH', y='MAX_TEMPERATURE_C', hue='SUNHOUR',
palette="coolwarm", ax=axs[0, 0])
axs[0,0].set_title('month en fonction de temperature max')
axs[0,0].set_xlabel('MONTH')
axs[0,0].set_ylabel('MAX_TEMPERATURE_C')
axs[0,0].legend(title='T°C')

-----
#Créer une figure avec plusieurs sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(20, 14))

# Définition du premier graphique
# scatter plot de max_temp par mois
sns.scatterplot(data=df_weather, x='MONTH', y='MAX_TEMPERATURE_C', hue='SUNHOUR',
palette="coolwarm", ax=axs[0, 0])
axs[0,0].set_title('month en fonction de temperature max')
axs[0,0].set_xlabel('MONTH')
axs[0,0].set_ylabel('MAX_TEMPERATURE_C')
axs[0,0].legend(title='T°C')

# Définition du second graphique
# Histogram of petal length, colored by species -- histogramme de la longueur du pétale,
coloré par espèce
sns.histplot(data=df_weather, x='MAX_TEMPERATURE_C', hue='SUNHOUR', palette="YlOrBr",
ax=axs[0, 1])
axs[0, 1].set_title('Distribution de humidité par opinion')
axs[0, 1].set_xlabel('MAX_TEMPERATURE_C')
axs[0, 1].set_ylabel('frequence')
axs[0, 1].legend(title='T°C')

# Définition du troisième graphique
# boîte à moustaches de la longueur du sépale, colorée par espèce
sns.boxplot(data=df_weather, x='MONTH', y='MAX_TEMPERATURE_C', hue='OPINION',
palette='pastel', ax=axs[1, 0])
axs[1, 0].set_title('Distribution of max_temperature by month')
axs[1, 0].set_xlabel('month')
axs[1, 0].set_ylabel('max_temperature')

# Définition du quatrième graphique
sns.violinplot(data = df_weather, x = "MONTH", y ="HUMIDITY_MAX_PERCENT",
hue='OPINION',inner = "quartile", ax=axs[1, 1])
axs[1, 1].set_title('Distribution of humidity_max by month')
axs[1, 1].set_xlabel('month')
axs[1, 1].set_ylabel('max_humidity')

# Ajuster la disposition pour éviter le chevauchement
plt.tight_layout()

# Show the plot -- Afficher le graphique
plt.show()

```

DATE TIME PYTHON/PANDAS

lundi 26 mai 2025 14:23

```
# Importing libraries
import numpy as np
import pandas as pd
from datetime import datetime
```

```
# Time avec Python
# Le premier argument est votre objet chaîne de caractères
# Le deuxième argument est le format, ici, c'est un format anglo-saxon.
var_date = datetime.strptime('2018-01-04', '%Y-%m-%d')

# Le premier argument est votre objet date
# Le deuxième argument est le format, ici, c'est un format français.
datetime.strptime(var_date, '%d/%m/%Y')
>>> '04/01/2018'

# Time avec Pandas
pd.to_datetime(df['DATE'])
```

Apply & Lambda Functions

lundi 26 mai 2025 14:25

```
# Création d'un fonction
def convert_CelsToFahr(degrees):
    return (degrees*1.8 + 32)
convert_CelsToFahr(10)

# Utilisation de la fonction avec apply
df_weather['MAX_TEMPERATURE_C'].apply(convert_CelsToFahr)

# J'obtiens alors une nouvelle série avec le résultat. Je peux enregistrer le résultat dans
une nouvelle colonne
df_weather['MAX_TEMPERATURE_F'] = df_weather['MAX_TEMPERATURE_C'].apply(convert_CelsToFahr)

# Création d'un nouvelle fonction
def warmer_time(morning, noon, evening):
    # Les paramètres d'entrée sont les températures du matin, de midi et du soir.
    if morning > noon and morning > evening:
        return 'morning'
    if noon > morning and noon > evening:
        return 'noon'
    if evening > noon and evening > morning:
        return 'evening'
    else:
        return 'undecided'

# Utilisation de la fonction
def function_name(one_var):
    # code dans lequel chaque colonne a une position dans la variable
    return one value
df[new_col] = df[[col1, col2, ...]].apply(function_name, axis = 1)

# Exemple création fonction + Application avec apply
def warmer_time(temperature):
    morning = temperature[0]
    noon = temperature[1]
    evening = temperature[2]
    if morning > noon and morning > evening:
        return 'morning'
    if noon > morning and noon > evening:
        return 'noon'
    if evening > noon and evening > morning:
        return 'evening'
    else:
        return 'undecided'

weather['warmer_time'] = weather[['TEMPERATURE_MORNING_C', 'TEMPERATURE_NOON_C',
'TEMPERATURE_EVENING_C']].apply(warmer_time, axis = 1)

# Utilisation Lambda Fonction
df_weather['MAX_TEMPERATURE_F'] = df_weather['MAX_TEMPERATURE_C'].apply(lambda degrees: degrees*
1.8 + 32)
# exemple lambda fonction
df_weather['rain_message'] = df_weather['PRECIP_TOTAL_DAY_MM'].apply(lambda mm: f'Il va y avoir
{mm} mm de pluie aujourd'hui')
```

Exemple :

```
df['majeur'] = df['age'].apply(lambda x: 'Oui' if x >= 18 else 'Non')

df['total'] = df.apply(lambda row: row['prix'] * row['quantité'], axis=1)

df['categorie'] = df['prix'].apply(lambda x: 'cher' if x > 150 else 'bon marché')

df['nom'] = df['nom'].apply(lambda x: x.lower().strip())
# .strip() optionnel supprime espace avant et après

df['mention'] = df['note'].apply(lambda x: 'Échec' if x < 10 else 'Passable' if x < 12 else 'Assez bien' if x < 14
else 'Bien')

df['longueur_nom'] = df['nom'].apply(lambda x: len(x))
# Compte le nombre de lettres dans chaque nom.

df['nom_majuscule'] = df['nom'].apply(lambda x: x.upper())
df['info'] = df.apply(lambda row: f'({row['nom']}) a {row['age']} ans", axis=1)
# Concatène plusieurs colonnes pour créer une phrase

revenu_moyen = df['revenu'].mean()
df['revenu_sup_moy'] = df['revenu'].apply(lambda x: x > revenu_moyen)
# Déterminer si revenu > moyenne

df['initiale'] = df['nom'].apply(lambda x: x[0])
# Extraction de l'initiale du nom

df['revenu_net'] = df['revenu'].apply(lambda x: x * 0.8)
# Calculer le revenu annuel net (hypothèse : impôt 20 %)

df['code'] = df['nom'].apply(lambda x: dictionnaire.get(x, 'Inconnu'))
# Mapper un champ via dictionnaire

df['nom_sans_voyelles'] = df['nom'].apply(lambda x: ''.join([c for c in x if c.lower() not in 'aeiou']))
# Supprimer les voyelles dans les noms

df['mineur_pauvre'] = df.apply(lambda row: row['age'] < 18 and row['revenu'] < 20000, axis=
1).replace(True, 'Pauvre').replace(False, 'Riche')
# Détection de mineurs avec revenus faibles

df['cat_revenu'] = df['revenu'].apply(lambda x: 'Faible' if x < 20000 else 'Moyen' if x < 40000 else 'Élevé')
# Catégoriser les revenus
```


Pivot Table

lundi 26 mai 2025 18:26

```
# Construction Pivot_table()
df_weather.pivot_table(columns= 'OPINION', # columns
                        index = 'MONTH',    # rows
                        values = 'MAX_TEMPERATURE_C', # values
                        aggfunc = 'mean', ) # aggregate function

# https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.pivot\_table.html
# Exemple
df_weather.pivot_table(values = 'MAX_TEMPERATURE_C', index = 'MONTH', aggfunc
= 'sum')
# Exemple
view=df_weather.pivot_table(index = 'MONTH', values =
['MAX_TEMPERATURE_C', 'SUNHOURL'], aggfunc = 'mean')
```

Plotly

lundi 26 mai 2025 18:27

import plotly.express as px

exemple de code

```
fig = px.bar(df_weather,
             x = 'DAY',
             y= 'MAX_TEMPERATURE_C',
             animation_frame= 'MONTH',
             range_x = [0,31],
             range_y= [0,45],
             color='MAX_TEMPERATURE_C',
             title='Temperature par jour par mois',
             labels =({'DAY' : 'jour du mois',
                      'MAX_TEMPERATURE_C' : 'Temperature maximum'}))

fig.update_layout(title={
    'x' : 0.5,
    'xanchor' : 'center'})
fig.show()
```

ou encore

```
fig = px.scatter(data_frame = df_weather, x= "DATE", y="MAX_TEMPERATURE_C", color =
"OPINION",color_discrete_sequence = ['red','blue',' #F733F5','orange','#08cc0a'])
fig.update_layout(title_text = 'Températures annuelles suivant les Avis de la population',
legend_title="Avis")
```

Pair Plot

```
fig= px.scatter_matrix(df_weather, dimensions=['UV_INDEX', 'SUNHOUR', 'MONTH','DAY'])
fig.show()
```

Bubble Chart

```
fig=px.scatter(df_weather, x="DATE",y=["MAX_TEMPERATURE_C",'PRECIP_TOTAL_DAY_MM'],
size='PRECIP_TOTAL_DAY_MM',title="Températures Max et Précipitations")
fig.update_layout(showlegend=False,yaxis_title='Température')
fig.show()
```

ou encore graph animé

```
fig =
px.scatter(df_weather,x='MAX_TEMPERATURE_C',y='WINDSPEED_MAX_KMH',color='MAX_TEMPERATURE_C',a
nimation_frame="MONTH",color_continuous_scale="Jet", title = 'température VS vent')
fig.update_xaxes(range=[0,25])
fig.update_yaxes(range=[0,40])
fig.update_layout(xaxis_title='Températures', yaxis_title='Vitesse du Vent')
fig.update_layout(coloraxis_colorbar=dict(title="Échelle des températures" ))
fig.show()
```

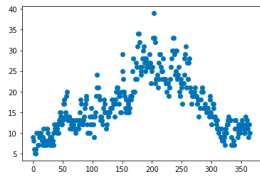
et encore graphique animé avec animation_frame

```
fig =
px.bar(df_weather,x='DAY',y=['MAX_TEMPERATURE_C','PRECIP_TOTAL_DAY_MM','WINDSPEED_MAX_KMH'] ,
animation_frame='MONTH',color_discrete_sequence = ['#F733F5','orange','#08cc0a'])
fig.update_xaxes(range=[0,31])
fig.update_layout(title={'text': "Températures et Précipitations suivant opinions",'x':
0.5, 'xanchor': 'center'})
fig.update_layout(xaxis_title='Jours',yaxis=None)
fig.update_layout(showlegend=True, legend_title='Légendes')
fig.show()
```

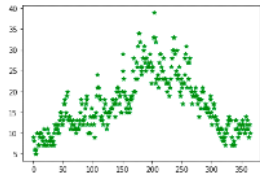
Type de Graphique

Lundi 26 mai 2025 18:32

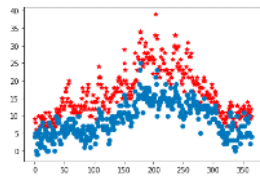
```
plt.scatter(x = df["column1"], y = df["column2"])
plt.show()
```



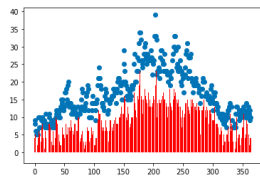
```
plt.scatter(x = df["column1"], y = df["column2"], color = "green", marker = "**")
plt.show()
```



```
plt.scatter(x = df["column1"], y = df["column2"], color = "red", marker = "**")
plt.scatter(x = df["column1"], y = df["column3"])
plt.show()
```



```
plt.scatter(x = df["column1"], y = df["column2"])
plt.bar(x = df["column1"], height = df["column3"], color = "red")
plt.show()
```



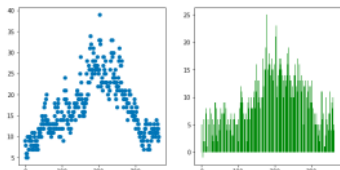
Définition de la figure
Le paramètre `figsize` permet de définir la largeur et la hauteur de la figure
`plt.figure(figsize = (10,5))`

Définition du premier graphique

```
plt.subplot(121)
plt.scatter(x = df["column1"], y = df["column2"])
```

Définition du second graphique

```
plt.subplot(122)
plt.bar(x = df["column1"], height = df["column3"], color = "green")
plt.show()
```



Définition de la figure
Le paramètre `figsize` permet de définir la largeur et la hauteur de la figure
`plt.figure(figsize = (10,5))`

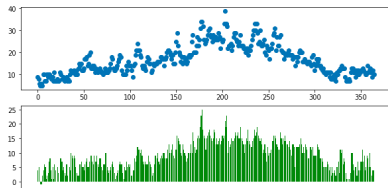
Définition du premier graphique

```
plt.subplot(211)
plt.scatter(x = df["column1"], y = df["column2"])
```

Définition du second graphique

```
plt.subplot(212)
plt.bar(x = df["column1"], height = df["column3"], color = "green")
```

`plt.show()`



Palette couleur Matplotlib

```
from matplotlib import colormaps
list(colormaps)
```

Générer un graph

```
plt.figure(figsize=(10,6))

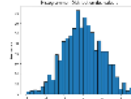
scatter = plt.scatter(x, y, s=sizes, c=y, cmap='viridis', alpha=0.6, edgecolors='black')

plt.title('Distribution des Valeurs', fontsize=16, fontweight='bold')
plt.xlabel('Valeurs', fontsize=10)
plt.ylabel('Fréquence', fontsize=10)
plt.grid(axis='y', alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

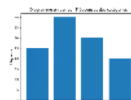
```
# HISTOGRAMME
data = np.random.normal(0, 1, 1000)

plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogramme - Distribution des valeurs')
plt.xlabel('Valeurs')
plt.ylabel('Fréquence')
plt.show()
```

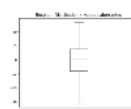


```
# DIAGRAMME EN BARRE
categories = ['A', 'B', 'C', 'D']
values = [25, 40, 30, 20]

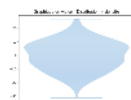
plt.bar(categories, values)
plt.title('Diagramme en barres - Fréquences des catégories')
plt.xlabel('Catégories')
plt.ylabel('Fréquence')
plt.show()
```



```
# BOX PLOT - BOITE A MOUSTACHE
data = np.random.normal(0, 10, 200)
plt.boxplot(data)
plt.title('Boxplot - Distribution + valeurs aberrantes')
plt.show()
```



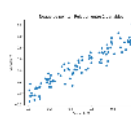
```
# VIOLIN PLOT
plt.violinplot(data)
plt.title('Graphique en violon - Distribution + densité')
plt.show()
```



```
# CANEBERT
labels = ['A', 'B', 'C', 'D']
sizes = [15, 30, 45, 10]
plt.pie(sizes, labels=labels, autopct='%1.2f%%')
plt.title('Canebert - Proportions (usage limité)')
plt.show()
```

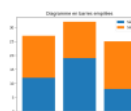


```
# SCATTER PLOT - NUAGE DE POINTS
x = np.random.rand(100)
y = x + np.random.normal(0, 0.1, 100)
plt.scatter(x, y)
plt.title('Nuage de points - Relation entre 2 variables')
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.show()
```



```
# DIAGRAMME EN BARRES EMPILÉES
categories = ['Groupe 1', 'Groupe 2', 'Groupe 3']
values1 = [12, 15, 8]
values2 = [15, 13, 17]

# Groupées
plt.bar(categories, values1, label='Série 1')
plt.bar(categories, values2, bottom=values1, label='Série 2')
plt.legend()
plt.title('Diagramme en barres empilées')
plt.show()
```



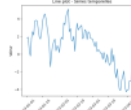
```
# HEATMAP - MATRICE DE CORRELATION
data = pd.DataFrame(np.random.rand(5, 5))
sns.heatmap(data, annot=True)
plt.title('Heatmap - Matrice de corrélations')
plt.show()
```



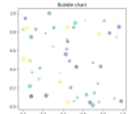
```
# HEATMAP AVEC MASK
mask = np.triu(np.ones_like(data, dtype=bool))
data = pd.DataFrame(np.random.rand(5, 5))
sns.heatmap(data, annot=True, mask=mask)
plt.title('Heatmap - Matrice de corrélations')
plt.show()
```



```
# LINE PLOT
dates = pd.date_range('20230101', periods=100)
values = np.cumsum(np.random.randn(100))
plt.plot(dates, values)
plt.title('Line plot - Séries temporelles')
plt.xlabel('Date')
plt.ylabel('Valeur')
plt.xticks(rotation=45)
plt.show()
```



```
# BUBBLE CHART
x = np.random.rand(50)
y = np.random.rand(50)
sizes = np.random.randint(10, 100, 50)
colors = np.random.rand(50)
plt.scatter(x, y, s=sizes, c=colors, alpha=0.5)
plt.colorbar()
plt.title('Bubble chart')
plt.show()
```



SCATTER PLOT NUAGE DE POINT AVEC HUE ET SIZE EN FONCTION DES VALEURS

```
# Création de données synthétiques
np.random.seed(42)
n = 100
x = np.random.rand(n)
y = x + np.random.normal(0, 0.1, n)
categories = np.random.choice(['Groupe A', 'Groupe B', 'Groupe C'], size=n)
```

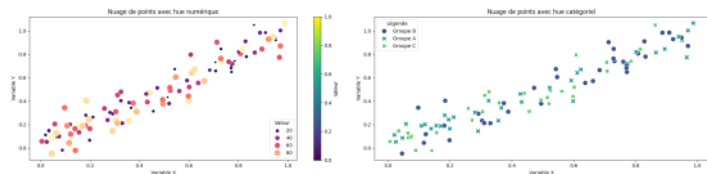
```

values = np.random.randint(1, 100, n)
# Création d'un DataFrame
df = pd.DataFrame({
    'Variable X': x,
    'Variable Y': y,
    'Catégorie': categories,
    'Valeur': values
})

# Nuage de points avec hue numérique
plt.figure(figsize=(20, 5))
plt.subplot(1, 2, 1)
scatter = sns.scatterplot(
    data=df,
    x='Variable X',
    y='Variable Y',
    hue='Valeur',      # Couleur par valeur numérique
    size='Valeur',     # Taille par valeur numérique
    sizes=(20, 200),  # Plage de tailles
    palette='magma'    # Palette différente
)
plt.title('Nuage de points avec hue numérique')
plt.xlabel('Variable X')
plt.colorbar(scatter.collections[0], label='Valeur')
plt.legend(title='Valeur', loc=4)

# Nuage de points avec hue catégoriel
plt.subplot(1, 2, 2)
sns.scatterplot(
    data=df,
    x='Variable X',
    y='Variable Y',
    hue='Catégorie',  # Couleur par catégorie
    style='Catégorie', # Forme différente par catégorie
    s=100,            # Taille des points
    palette='viridis' # Palette de couleurs
)
plt.title('Nuage de points avec hue catégoriel')
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.legend(title='Légende')
plt.tight_layout()
plt.show()

```



REGRESSION LINEAIRE

1 - faire la régression
2 - Mesurer la précision MAE (Erreur moyenne absolue)
3 - Mesurer également la précision avec RMSE plus précise, elle accentue la détection des forts écarts

Interprétation des résultats & Évaluation du modèle

Métriques courantes en régression :

- MAE (Mean Absolute Error) : Erreur absolue moyenne
- MSE (Mean Squared Error) : Erreur quadratique moyenne
- RMSE (Root Mean Squared Error) : Racine carrée de l'erreur quadratique moyenne
- R² (coefficient de détermination) : Proportion de la variance expliquée par le modèle

Métriques courantes en classification :

- Accuracy (précision) : Proportion de prédictions correctes
- Precision : Proportion des prédictions positives qui sont réellement positives
- Recall (rappel) : Proportion des cas positifs réels qui sont correctement prédits
- F1-score : Moyenne harmonique de la précision et du rappel
- Matrice de confusion : Tableau qui montre les prédictions correctes et incorrectes pour chaque classe

Optimisation des paramètres

Techniques d'optimisation courantes :

- Grid Search : Teste toutes les combinaisons possibles de paramètres spécifiés
- Random Search : Teste aléatoirement des combinaisons de paramètres dans un espace défini
- Cross-validation : Divise les données en plusieurs sous-ensembles pour valider la robustesse du modèle

```
# Exemple
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import pandas as pd

# Chargement des données
data = pd.read_csv('house_prices.csv')

# Définition des features (X) et de la cible (y)
X = data[['surface', 'bedrooms', 'bathrooms', 'age']] # Variables explicatives
y = data['price'] # Variable à prédire

# Division en jeux d'entraînement et de test (75% entraînement, 25% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Instanciation et entraînement du modèle
model = LinearRegression()
model.fit(X_train, y_train)

# Prédictions
y_pred = model.predict(X_test)

# Régression linéaire
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Diviser la donnée pour la tester : train_test_split()
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.8, random_state = 42) # ou test_size = 0.2
# Entraîner le modèle façon "fit"
model = LinearRegression()
model.fit(X_train, y_train)
y_predict = model.predict(X_test)
# contrôle et validation du modèle avec : Coef R², Erreur moyenne ABSOLUE (MAE), ou la RMSE qui accentue la détection des forts écarts
```

Régression linéaire Simple

```
#A RETENIR:
#Evaluation de la performance du modèle:
#Par les métriques adaptées au modèles
#Autre possibilité d'évaluation de la performance du modèle: Je vérifie que le score du train et le score du test sont assez proches.
#Si score_train > score_test → OVERFITTING :
#Le modèle a trop bien appris les données d'entraînement (par cœur) et n'arrive pas à généraliser sur des données nouvelles (test).
#Si score_train ≈ score_test mais tous les deux faibles → UNDERFITTING :
#Le modèle n'a pas capté suffisamment la structure des données, ni sur l'entraînement, ni sur le test.
#→ Le modèle est trop simple ou les features ne sont pas assez informatives.
#Idéalement (GOODFITTING) :
#R² est proche de 1.
#Et les scores sur train et test sont similaires (pas d'écart significatif).
#→ Cela signifie que le modèle généralise bien et explique correctement la variable cible.
```

https://colab.research.google.com/drive/1EYt50ic9oX_ZunRh9issGRfuxXmXuiAn#scrollTo=4h_kVYZJRUCz

```
# Import bibliothèque de manipulation de dataframe
import pandas as pd
import numpy as np
```

```
# Import des bibliothèques de viz
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Import split data
from sklearn.model_selection import train_test_split
```

```
# Import modèles de ML Supervisé Régression
from sklearn.linear_model import LinearRegression
```

```
# Import modèles de ML Supervisé Classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
```

```
# Import modèle de ML NON Supervisé
from sklearn.neighbors import NearestNeighbors
```

```
# Import des métriques
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

```
# Import outil standardisation de la donnée
from sklearn.preprocessing import StandardScaler
```

```
# Import pipeline
```

CLASSIFICATION

Type d'algo de Classification :

- Logistic Regression : Détection de la probabilité qu'un client clique sur une annonce en ligne (problème de clic ou non clic).
- KNN : Recommandation de produits basée sur des utilisateurs similaires dans un système de e-commerce.
- Random Forest : Prédiction de l'approbation d'un prêt bancaire en fonction des données du client (revenu, historique de crédit, etc.).
- SVM : Classification d'images pour différencier des lettres manuscrites dans un système OCR.
- Gradient Boosting : Prédiction des prix de logements (transformé en - classification, par exemple : "moins de 200k", "200k à 500k", "plus de 500k").
- Naive Bayes : Détection de spams dans des emails en analysant les mots fréquemment associés au spam.

```
# Acquisition du jeu de donnée
X = df.drop(columns = 'COLONNE CIBLE') # dataframe sans la colonne cible
y = df['COLONNE CIBLE'] # colonne cible
```

```
# 1 - Les données sont divisées en ensembles d'entraînement et de test.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
# 2 - Le modèle est entraîné sur les données d'entraînement.
knn = KNeighborsClassifier(n_neighbors = 3)
knn.fit(X_train, y_train)
```

```
# 3 - Le modèle prédit les étiquettes sur les données de test.
y_pred = knn.predict(X_test)
```

```
# 4 - Les performances sont évaluées.
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Précision du modèle KNN avec k=3 : {accuracy:.2f}")
```

Régression logistique

```
# Import bibliothèque de manipulation de dataframe
import pandas as pd
```

```
# Import des bibliothèques de viz
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Import split data
from sklearn.model_selection import train_test_split
```

```
# Import modèles de ML Supervisé Régression
from sklearn.linear_model import LinearRegression
```

```
# Import modèles de ML Supervisé Classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
```

```
# Import modèle de ML NON Supervisé
from sklearn.neighbors import NearestNeighbors
```

```
# Import des métriques
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

```
# Import outil standardisation de la donnée
from sklearn.preprocessing import StandardScaler
```

```
# Import pipeline
from sklearn.pipeline import Pipeline
```

```
# Gestion des warnings
import warnings
```

```
# --- Chargement du jeu de données Wine Quality (Red) ---
# Utiliser pd.read_csv car fetch_ucirepo peut être moins direct pour ce dataset spécifique
try:
```

```
    # L'URL pointe directement vers le fichier CSV pour le vin rouge
    csv_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
```

```
    # Les noms de colonnes sont dans la première ligne, mais séparés par ';', on ajuste read_csv
    df = pd.read_csv(csv_url, sep=';')
    print("Jeu de données Wine Quality (Red) chargé avec succès.")
except Exception as e:
    print(f"Erreur lors du chargement du jeu de données depuis l'URL: {e}")
    print("Vérifiez votre connexion internet ou l'URL.")
print(df.head())
```

```
# Créer une variable cible binaire: 'good_quality' (1 si quality > 5, 0 sinon)
threshold = 5
```

```
# Crée une série de True/False si quality > threshold, puis convertit en 0/1
df['good_quality'] = df['quality'].apply(lambda x : 1 if x > threshold else 0)
df.sample(5)
```

```
# Features pour la classification (X_class): Toutes les colonnes SAUF 'quality' (la base de y_class), 'good_quality' (la cible elle-même) et 'pH' (utilisé en régression)
# On exclut les cibles ('quality', 'good_quality') et la cible de l'autre tâche ('pH')
X = df.drop(columns=['quality', 'good_quality', 'pH'])
y = df['good_quality']
```

```
print(f"\nFeatures pour la classification (prédiction de good_quality): {list(X.columns)}")
```

```
# value_counts() compte les occurrences de chaque classe
# normalize=True affiche les proportions (pourcentages) au lieu des comptes bruts
print(f"Cible pour la classification: {y.name}")
print(f"\nDistribution de la variable cible 'good_quality':")
print(df['good_quality'].value_counts())
```

```
# 1. Distribution de la variable cible binaire ('good_quality')
plt.figure(figsize=(6, 4))
```

```
# sns.countplot est idéal pour visualiser le nombre d'occurrences de chaque catégorie
sns.countplot(x=y)
plt.title('Distribution de la Classe Binaire (good_quality)')
plt.xlabel('Quality')
plt.ylabel('Nombre d'échantillons')
```

```
# Remplacer les labels 0 et 1 par des textes plus explicites
plt.xticks([0, 1], ['Mauvais/Moyen', 'Bon'])
plt.show()
```

```
# Afficher les variable les plus corrélées (>0.2) avec good_quality
quality_corr= df.corr()['good_quality'].drop(['quality', 'good_quality'])
quality_corr = quality_corr[abs(quality_corr) > 0.2].sort_values(ascending=False)
quality_corr
```

```
# Créer une liste features_to_plot de ces features
features_to_plot=quality_corr.index
```

```
print(features_to_plot)
```

```

confusion_matrix, ConfusionMatrixDisplay

# Import outil standardisation de la donnée
from sklearn.preprocessing import StandardScaler

# Import pipeline
from sklearn.pipeline import Pipeline

# Gestion des warnings
import warnings

# --- Chargement du jeu de données Wine Quality (Red) ---
# Utiliser pd.read_csv car fetch_ucirepo peut être moins direct pour ce dataset spécifique
try:
    # L'URL pointe directement vers le fichier CSV pour le vin rouge
    csv_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
    # Les noms de colonnes sont dans la première ligne, mais séparés par ';', on ajuste read_csv
    df = pd.read_csv(csv_url, sep=';')
    print("Jeu de données Wine Quality (Red) chargé avec succès.")
except Exception as e:
    print(f"Erreur lors du chargement du jeu de données depuis l'URL: {e}")
    print("Vérifiez votre connexion internet ou l'URL.")

# Afficher un premier aperçu des données
print("\nAperçu du jeu de données Wine Quality (Red):")
df.head()

# Afficher des informations générales sur le DataFrame (dtypes, shape, etc.)
print("\nInformations sur le jeu de données:")
df.info()

# Afficher des statistiques descriptives pour les colonnes numériques
print("\nStatistiques descriptives:")
df.describe().T

print("\nDistribution des notes de qualité (variable cible initiale):")
print(df['quality'].value_counts().sort_index())

# Séparer les features (X) et la cible de régression (y_reg)
# --- Préparation pour la Régression (Prédire pH) ---
# Features (X_reg): Toutes les colonnes SAUF 'pH' et 'quality' (qui sera utilisée pour la classification)
X_reg = df.drop(['pH', 'quality'], axis=1)

# Sélection de la target (y) pour la régression
y_reg = df['pH']

# Afficher les noms des colonnes pour vérifier
print(f"\nFeatures pour la régression (prédiction de pH): {list(X_reg.columns)}")
print(f"Cible pour la régression: {y_reg.name}")

# EDA L'Analyse Exploratoire des Données
# Distribution de la variable cible (pH)
plt.figure(figsize=(10, 6))

# sns.histplot crée un histogramme, kde=True ajoute une courbe de densité estimée
sns.histplot(data=df, x='pH', kde=False)
plt.title('Distribution du pH du Vin Rouge')
plt.xlabel('pH')
plt.ylabel('Fréquence')
plt.show()

# Corrélation des autres features avec le pH
plt.figure(figsize=(10, 8))

# df.corr() calcule la matrice de corrélation de Pearson entre toutes les colonnes numériques
# ['pH'] sélectionne la colonne correspondant aux corrélations avec le pH
# .drop('pH') enlève la corrélation de pH avec lui-même (qui est toujours 1)
correlation_matrix_ph = df.corr()[['pH']].drop('pH') # Exclure l'auto-corrélation
# .sort_values() trie les corrélations pour une lecture facile
# .plot(kind='bar') crée un diagramme en barres
correlation_matrix_ph.sort_values(ascending=False).plot(kind='bar')
plt.title('Corrélation des Features avec le pH')
plt.ylabel('Coefficient de Corrélation')
plt.show()

# Afficher aussi les valeurs numériques des corrélations
print("\nCorrélation des features avec le pH:")

# Affiche les valeurs triées pour référence
print(df.corr()[['pH']].sort_values(ascending=False))

##### Affichage sous forme de HeatMap - moi je préfère
cm = df.corr()
mask = np.triu(np.ones_like(cm))
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt=".2f", cmap='vlag', center=0, mask=mask)
plt.title('Corrélation en pH et variables explicatives')
plt.show()

# 1. Définir la feature(X) et la target (y)
# Choisir une feature, par ex. 'fixed acidity' (souvent corrélée négativement au pH)
feature_simple = 'fixed acidity'

# Il faut garder un format DataFrame (avec double crochets [[]]) même pour une seule colonne
X_simple = X_reg[[feature_simple]] # Sélectionner depuis X_reg

# 2. Diviser les données
# test_size=0.2 signifie que 20% des données iront dans l'ensemble de test, 80% pour l'entraînement
# random_state=42 assure que la division est la même à chaque exécution (reproductibilité)
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_simple, y_reg, test_size=0.2, random_state=42)
print(f"Taille de l'ensemble d'entraînement simple: {X_train_s.shape[0]} échantillons")
print(f"Taille de l'ensemble de test simple: {X_test_s.shape[0]} échantillons")

# Créer, entraîner et évaluer le modèle simple
# Je crée mon modèle
model_simple = LinearRegression()

# J'entraîne mon modèle
# .fit() apprend les coefficients (b0 et b1) à partir des données d'entraînement (X_train_s, y_train_s)
model_simple.fit(X_train_s, y_train_s)

# Prédictions
# .predict() utilise le modèle entraîné pour prédire le pH pour les données de test (X_test_s)
y_pred_s_test = model_simple.predict(X_test_s)

# Je fais également une prédiction sur les données d'entraînement afin de comparer les scores sur le train et sur le test :
y_pred_s_train = model_simple.predict(X_train_s)

# J'évalue les performances de mon modèle
# R-squared (R²) : Coefficient de détermination. Proportion de la variance de la cible expliquée par le modèle.
# Va de -infini à 1. Proche de 1 signifie que le modèle explique bien la variance. Proche de 0

quality_corr

# Créer une liste features_to_plot de ces features
features_to_plot=quality_corr.index

print(features_to_plot)

# Afficher un Subplot 2 x 2 avec des boxplots good quality / features_to_plot
plt.figure(figsize=(15,10))
for i, feature in enumerate(features_to_plot):
    plt.subplot(2,2, i+1)
    sns.boxplot(x='good_quality', y= feature, data=df)
    plt.title(f'{feature} / good_quality')
    plt.xticks([0, 1], ['Mauvais/Moyen', 'Bon'])
    plt.tight_layout()
plt.show()

# Créer un pipeline pour la classification (Standardisation + LogReg)
# Le scaler sera entraîné sur les features de X_class
pipeline = Pipeline(steps=[('scaler', StandardScaler()),
                           ('classifieur', LogisticRegression())])

# Diviser les données (X_class et y_class)
# stratify=y_class est crucial ici pour gérer le déséquilibre potentiel des classes
X = df.drop(columns=['quality', 'good_quality', 'pH'])
y = df['good_quality']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Entraîner le pipeline de classification
pipeline.fit(X_train, y_train)

# Faire des prédictions
y_pred_test = pipeline.predict(X_test)
y_pred_train = pipeline.predict(X_train)

# Évaluation avec Accuracy (veut dire "précision")
accuracy_train = accuracy_score(y_train, y_pred_train)
accuracy_test = accuracy_score(y_test, y_pred_test)
print(f"Accuracy sur le test: {accuracy_test:.4f}")
print(f"Accuracy sur le train: {accuracy_train:.4f}")

# Matrice de confusion fonction "confusion_matrix"
# Format: [[TN, FP], [FN, TP]]
# TN: Vrais Négatifs (Classe 0 prédite 0)
# FP: Faux Positifs (Classe 0 prédite 1) - Erreur Type I
# FN: Faux Négatifs (Classe 1 prédite 0) - Erreur Type II
# TP: Vrais Positifs (Classe 1 prédite 1)

cm = confusion_matrix(y_test, y_pred_test)
# Utiliser ConfusionMatrixDisplay pour un affichage graphique plus clair
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Mauvais/Moyen', 'Bon'])
disp.plot(cmap='Blues')
plt.title("Matrice de Confusion")
plt.show()

# Afficher le rapport de Classification
# Donne precision, recall, f1-score pour chaque classe ('Mauvais/Moyen', 'Bon')
# et des moyennes (macro avg, weighted avg)

print(classification_report(y_test, y_pred_test, target_names=['Mauvais/Moyen', 'Bon']))

Classification kNeighborsClassifier

# Import bibliothèque de manipulation de dataframe
import pandas as pd

# Import des bibliothèques de viz
import matplotlib.pyplot as plt
import seaborn as sns

# Import split data
from sklearn.model_selection import train_test_split

# Import modèles de ML Supervisé Régression
from sklearn.linear_model import LinearRegression

# Import modèles de ML Supervisé Classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

# Import modèle de ML NON Supervisé
from sklearn.neighbors import NearestNeighbors

# Import des métriques
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

# Import outil standardisation de la donnée
from sklearn.preprocessing import StandardScaler

# Import pipeline
from sklearn.pipeline import Pipeline

# Gestion des warnings
import warnings

# --- Chargement du jeu de données Wine Quality (Red) ---
# Utiliser pd.read_csv car fetch_ucirepo peut être moins direct pour ce dataset spécifique
try:
    # L'URL pointe directement vers le fichier CSV pour le vin rouge
    csv_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
    # Les noms de colonnes sont dans la première ligne, mais séparés par ';', on ajuste read_csv
    df = pd.read_csv(csv_url, sep=';')
    print("Jeu de données Wine Quality (Red) chargé avec succès.")
except Exception as e:
    print(f"Erreur lors du chargement du jeu de données depuis l'URL: {e}")
    print("Vérifiez votre connexion internet ou l'URL.")
print(df.head())

# Créer une variable cible binaire: 'good_quality' (1 si quality > 5, 0 sinon)
threshold = 5

# Crée une série de True/False si quality > threshold, puis convertit en 0/1
df['good_quality'] = df['quality'].apply(lambda x : 1 if x > threshold else 0)
df.sample(5)

# Features pour la classification (X_class): Toutes les colonnes SAUF 'quality' (la base de y_class), 'good_quality' (la cible elle-même) et 'pH' (utilisé en régression)
# On exclut les cibles ('quality', 'good_quality') et la cible de l'autre tâche ('pH')
X = df.drop(columns=['quality', 'good_quality', 'pH'])
y = df['good_quality']

print(f"\nFeatures pour la classification (prédiction de good_quality): {list(X.columns)}")

# value_counts() compte les occurrences de chaque classe

```

```

y_pred_s_train = model_simple.predict(X_train_s)

# J'évalue les performances de mon modèle
# R-squared (R²) : Coefficient de détermination. Proportion de la variance de la cible expliquée
# par le modèle.
# Va de -infini à 1. Proche de 1 signifie que le modèle explique bien la variance. Proche de 0
# signifie qu'il n'explique pas mieux que la moyenne.
r2_simple_train = r2_score(y_train_s, y_pred_s_train)
r2_simple_test = r2_score(y_test_s, y_pred_s_test)
print(f"Modèle Simple ({feature_simple}) - R² score sur le train: {r2_simple_train:.4f}")
print(f"Modèle Simple ({feature_simple}) - R² score sur le test: {r2_simple_test:.4f}")

# Mean Squared Error (MSE) : Erreur quadratique moyenne. Plus c'est bas, mieux c'est. Donne une
# idée de l'erreur moyenne au carré.
mse_simple = mean_squared_error(y_test_s, y_pred_s_test)
print(f"Modèle Simple ({feature_simple}) - MSE: {mse_simple:.4f}")

# Visualisation
plt.figure(figsize=(10, 6))

# plt.scatter affiche les points de données réels (ensemble de test)
plt.scatter(X_test_s, y_test_s, alpha=0.5, label='Données Réelles')

# plt.plot trace la ligne de régression prédite par le modèle pour les X de test
plt.plot(X_test_s, y_pred_s_test, color='red', linewidth=2, label='Ligne de Régression')
plt.title(f"Régression Linéaire Simple: {feature_simple} vs pH")
plt.xlabel(feature_simple)
plt.ylabel('pH')
plt.legend()
plt.show()

```

Régression linéaire Multiple

```

# Créer un pipeline: Standardisation + Régression Linéaire
# C'est une bonne pratique, même si toutes les features sont numériques
# Un pipeline est une séquence d'étapes. Ici :
# 'scaler' : Standardise les données (StandardScaler)
# 'regressor' : Applique la régression linéaire (LinearRegression)
pipeline = Pipeline(steps=[('scaler', StandardScaler()), ('regressor', LinearRegression())])

# Diviser les données (X_reg et y_reg)
# Note : On divise les données *avant* la standardisation. Le pipeline s'occupera
# d'appliquer le scaler appris sur le train aux données de train ET de test.
X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

# Entraîner le pipeline
# .fit() sur le pipeline applique .fit_transform() pour le scaler sur X_train_m,
# puis .fit() pour le regressor sur les données transformées et y_train_m.
pipeline.fit(X_train, y_train)

# Faire des prédictions
# .predict() sur le pipeline applique .transform() pour le scaler sur X_test_m,
# puis .predict() pour le regressor sur les données de test transformées.
y_pred_test = pipeline.predict(X_test)
y_pred_train = pipeline.predict(X_train)

# Évaluation
r2_test = r2_score(y_test, y_pred_test)
r2_train = r2_score(y_train, y_pred_train)
print(f"R² sur le test: {r2_test:.4f}")
print(f"R² sur le train: {r2_train:.4f}")

# On s'attend généralement à un R² meilleur qu'en régression simple si les features ajoutées
# sont utiles.
print("-----")
mse_test = mean_squared_error(y_test, y_pred_test)
mse_train = mean_squared_error(y_train, y_pred_train)
print(f"MSE_test: {mse_test:.4f}")
print(f"MSE_train: {mse_train:.4f}")

# Les coefficients (b1, b2, ...) indiquent comment la cible (pH) change pour une augmentation
# d'une unité de la feature correspondante, *après standardisation*, et en maintenant
# les autres features constantes. Un coefficient positif indique une relation positive,
# un négatif une relation négative. La magnitude donne une idée de l'importance relative.
coeffs = pd.DataFrame(
    pipeline_reg.named_steps['regressor'].coef_, # Accède aux coefficients du modèle DANS le
    pipeline
    index=X_reg.columns, # Noms des features utilisées pour la régression
    columns=['Coefficient'] # Nomme la colonne des coefficients)

print("\nCoefficients du modèle de régression multiple (pour prédire le pH, après
standardisation):")

# Trier par valeur absolue pour voir les plus influentes, ou par valeur pour voir le sens
print(coeffs.sort_values(by='Coefficient', ascending=False))

```

```

X = df.drop(columns=['quality', 'good_quality', 'pH'])
y = df['good_quality']

print(f"\nFeatures pour la classification (prédiction de good_quality): {list(X.columns)}")

# value_counts() compte les occurrences de chaque classe
# normalize=True affiche les proportions (pourcentages) au lieu des comptes bruts
print(f"Cible pour la classification: {y.name}")
print(f"Distribution de la variable cible 'good_quality':")
print(df['good_quality'].value_counts())

# 1. Distribution de la variable cible binaire ('good_quality')
plt.figure(figsize=(6, 4))

# sns.countplot est idéal pour visualiser le nombre d'occurrences de chaque catégorie
sns.countplot(x=y)
plt.title('Distribution de la Classe Binaire (good_quality)')
plt.xlabel('Quality')
plt.ylabel('Nombre d'échantillons')

# Remplacer les labels 0 et 1 par des textes plus explicites
plt.xticks([0, 1], ['Mauvais/Moyen', 'Bon'])
plt.show()

# Afficher les variable les plus corrélées (>0.2) avec good_quality
quality_corr = df.corr()['good_quality'].drop(['quality', 'good_quality'])
quality_corr = quality_corr[abs(quality_corr) > 0.2].sort_values(ascending=False)
quality_corr

# Créer une liste features_to_plot de ces features
features_to_plot = quality_corr.index
print(features_to_plot)

# Afficher un Subplot 2 x 2 avec des boxplots good quality / features_to_plot
plt.figure(figsize=(15,10))
for i, feature in enumerate(features_to_plot):
    plt.subplot(2,2, i+1)
    sns.boxplot(x='good_quality', y=feature, data=df)
    plt.title(f'{feature} / good_quality')
    plt.xticks([0, 1], ['Mauvais/Moyen', 'Bon'])
    plt.tight_layout()
plt.show()

# Créer un pipeline pour la classification (Standardisation + LogReg)
# Le scaler sera entraîné sur les features de X_class
pipeline = Pipeline(steps=[('scaler', StandardScaler()),
                           ('classifier', LogisticRegression())])

# Diviser les données (X_class et y_class)
# stratify=y_class est crucial ici pour gérer le déséquilibre potentiel des classes
X = df.drop(columns=['quality', 'good_quality', 'pH'])
y = df['good_quality']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
                                                    stratify=y)

# n_neighbors=5 est un choix courant pour k, mais pourrait être optimisé
k = 5
pipeline = Pipeline(steps=[('scaler', StandardScaler()),
                           ('classifier', KNeighborsClassifier(n_neighbors=k))])

# Pour KNN, .fit() stocke simplement les données d'entraînement
pipeline.fit(X_train, y_train)

# .predict() calcule les distances aux voisins et détermine la classe majoritaire
y_pred_train = pipeline.predict(X_train)
y_pred_test = pipeline.predict(X_test)

# Évaluation de la Performance du Modèle
accuracy_test = accuracy_score(y_test, y_pred_test)
accuracy_train = accuracy_score(y_train, y_pred_train)
print(f"Accuracy_test (k=5): {accuracy_test:.4f}")
print(f"Accuracy_train (k=5): {accuracy_train:.4f}")

# (Affichage Matrice/Rapport KNN comme avant)
cm = confusion_matrix(y_test, y_pred_test)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Mauvais/Moyen', 'Bon'])
disp.plot(cmap='Blues')
plt.title("Matrice de Confusion")
plt.show()

print("\n*5)

# Rapport de Classification (KNN)
print(classification_report(y_test, y_pred_test, target_names=['Mauvais/Moyen', 'Bon']))

```

ML NON SUPERVISE Exemples

vendredi 23 mai 2025 17:51

MODELE NearestNeighbors

```
# Import bibliothèque de manipulation de dataframe
import pandas as pd

# Import modèle de ML NON Supervisé
from sklearn.neighbors import NearestNeighbors

# Import outil standardisation de la donnée - mise à l'échelle
from sklearn.preprocessing import StandardScaler

# --- Chargement du jeu de données Wine Quality (Red) ---
# Utiliser pd.read_csv car fetch_ucirepo peut être moins direct pour ce dataset spécifique
try:
    # L'URL pointe directement vers le fichier CSV pour le vin rouge
    csv_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
    # Les noms de colonnes sont dans la première ligne, mais séparés par ';', on ajuste read_csv
    df = pd.read_csv(csv_url, sep=';')
    print("Jeu de données Wine Quality (Red) chargé avec succès.")
except Exception as e:
    print(f"Erreur lors du chargement du jeu de données depuis l'URL: {e}")
    print("Vérifiez votre connexion internet ou l'URL.")

# Exploration du df
df.head()

# Utiliser un ensemble cohérent de features pour trouver des voisins similaires
# Prenons les mêmes features que pour la classification (X_class)
# On crée un nouveau scaler et on l'entraîne sur l'ensemble de X_class
# Standardiser toutes ces features
X_class = df.drop(['quality', 'pH'], axis=1)          # drop permet de supprimer les colonnes que nous ne souhaitons pas utiliser
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_class)

# Nombre de voisins à trouver (y compris le point lui-même si k>0 et qu'il est dans les données)
# metric='euclidean' est la distance standard (ligne droite entre les points)
# algorithm='auto' choisit le meilleur algorithme en fonction des données
k=5          # les 5 voisins les plus proches

nn_model = NearestNeighbors(n_neighbors=k, metric='euclidean')
# .fit() indexe les données X_knn_scaled
nn_model.fit(X_scaled) # Entraîner sur les données standardisées X_class

# Sélectionner 3 points de X_class (qui sont non standardisés)
samples = X_class.sample(3)
Samples

# Trouver les voisins pour quelques points de l'ensemble de test (sample)
# Standardiser ces points d'abord avec le scaler_knn
X_test_scaled = scaler.transform(samples)

# Sélectionner les 3 premiers points de X_test_c (qui sont non standardisés)
samples_scaled = X_test_scaled
samples_scaled

# .kneighbors() prend les points standardisés et retourne distances et indices
distances, indices = nn_model.kneighbors(samples_scaled)

print("Indice les plus proches")
print(indices)
print('*'*40)
print("Distances les plus proches suivant indices ci-dessus")
print(distances)

# avec les distances suivant les index
for i in range(len(samples)):
    original_index = samples.index[i]
    print(f"Vins les plus proches de l'index {original_index} : ")
    print(df.loc[indices[i], ['pH', 'quality', 'alcohol']].to_markdown())

def find_nn(index, k):
    sample = X_class.loc[index:index]
    sample_scaled = scaler.transform(sample)
    d, i = nn_model.kneighbors(sample_scaled, k)
    print(i[0])
find_nn(2, 5)
```


ML (logistique et KNN)

vendredi 23 mai 2025 17:51

CHARGEMENT DES DONNEES

```
# Chargement des données
df =
pd.read_csv('https://raw.githubusercontent.com/Vaelastrasz/Heart_disease_dataset/refs/heads/main/he
art_failure_clinical_records_dataset.csv')

# Afficher les 5 premières lignes du dataset
df.head()
```

EXPLORATION DE LA DONNEES

```
# Affichage des infos du dataset
df.info()

# Description statistique du dataset
df.describe().T

df.shape

df.isna().sum()
```

VISUALISATION

Analyse univariées

L'analyse univariée est une méthode statistique utilisée pour comprendre la relation entre une variable indépendante unique et une variable dépendante. En analysant la relation entre ces deux variables, les chercheurs peuvent faire des prédictions et déduire des tendances en fonction des données collectées. La simplicité de la régression univariée en fait un outil essentiel pour les statisticiens et les data scientists.

```
# Utilisation d'une grille de plot pour visualiser la distribution des variables explicatives et
target
df.select_dtypes('number').hist(figsize=(15,15))
plt.show()
```

Analyse univariées

L'analyse bivariée est une méthode statistique qui consiste à examiner la relation entre deux variables distinctes. Cette technique analytique est essentielle dans divers domaines, notamment les statistiques, l'analyse de données, car elle aide les chercheurs et les analystes à comprendre comment une variable peut influencer ou être corrélée à une autre. En utilisant l'analyse bivariée, on peut découvrir des modèles, des tendances et des associations qui peuvent ne pas être immédiatement apparents lors de l'examen de chaque variable de manière isolée.

```
# Faire la visualisation graphique des variables explicatives face a la variable cible
binary_cols = ['anaemia', 'diabetes', 'high_blood_pressure', 'sex', 'smoking']
plt.figure(figsize=(15, 15))
for i, col in enumerate(df.columns[:-1]):
    plt.subplot(4, 3, i+1)
    # si col appartient à binary_col:
    if col in binary_cols:
        # sns.countplot
        sns.countplot(data = df, x = col, hue = 'DEATH_EVENT')
    # sinon
    # sns.boxplot
    else:
        sns.boxplot(data = df, x = 'DEATH_EVENT', y = col, showfliers = False)
plt.tight_layout()
plt.show()
```

```
#Matrice de corrélation (triangle heatmap)
plt.figure(figsize=(10, 10))
```

```
# Calculer corrélation cm = correlation matrix
cm = df.corr()
```

```
# Créer le masque
mask = np.triu(np.ones_like(cm))
```

```
# Afficher heatmap
sns.heatmap(cm, cmap = 'magma', annot = True, fmt = '.2F', mask = mask)
plt.show()
```

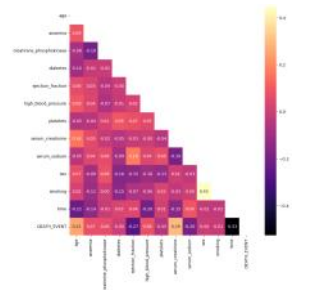
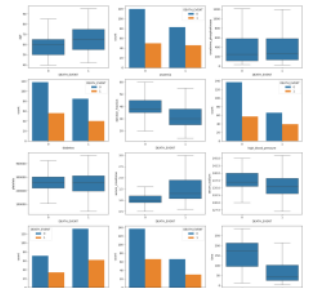
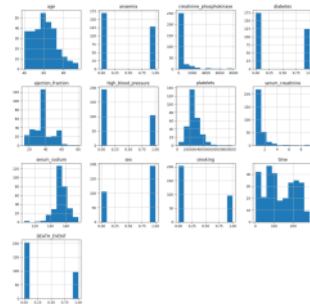
MACHINE LEARNING - CLASSIFICATION REGRESSION LOGISTIQUE

```
# Selection de X et Y
X = df.drop('DEATH_EVENT', axis = 1)
y = df['DEATH_EVENT']
```

```
# Creation d'une variable qui conserve les variables non binaires et une pour les variables binaires
#( celle pour les binary features a du être créer pour la visualtion des variables explicatives
face a la cible normalement)
binary_cols
non_binary = []
for col in df.columns:
    if col not in binary_cols:
        non_binary.append(col)
non_binary
```

```
# Création d'un pipeline pour transformer les variables non binaires a l'aide d'un standarscaler et
ignorer les variables binaires
# hint : utiliser passthrough pour les variables binaires dans le ColumnTransformer,
# 'passthrough' peut être utilisé comme un scaler sur une liste de variables pour les ignorer
('unchanged', 'passthrough', liste des variables a ignorer)
preprocessor = ColumnTransformer(
    transformers=[
        ('num',StandardScaler(), non_binary),
        ('unchanged', 'passthrough', binary_cols)
    ]
)
pipeline = Pipeline([
    ('preprocessor', preprocessor),
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report
```



```

    ('model', LogisticRegression(class_weight={0:1, 1:5}))
])

# Affichage de l'objet pipeline
Pipeline

# Étape de séparation des données en Entraînement et en Test
# Séparation du jeu de données en train test split (random_state = 42 test_size = 0.2)

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state = 42, test_size = 0.2)

# Entraînement du modèle via pipeline
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

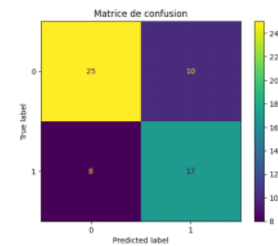
# Classification report pour Evaluer les performances du modèle (Regression Logistique)
print(classification_report(y_true = y_test, y_pred = y_pred))

# Créer la matrice de confusion
mc = confusion_matrix(y_pred = y_pred, y_true = y_test)

# Heatmap avec seaborn de la matrice de confusion
disp = ConfusionMatrixDisplay(mc)
disp.plot()
plt.title("Matrice de confusion")
plt.show()

```

	precision	recall	f1-score	support
0	0.76	0.71	0.74	35
1	0.63	0.68	0.65	25
accuracy			0.70	60
macro avg	0.69	0.70	0.69	60
weighted avg	0.70	0.70	0.70	60



MACHINE LEARNING - CLASSIFICATION KNN (KNearest Neighbors) // voisin le plus proche

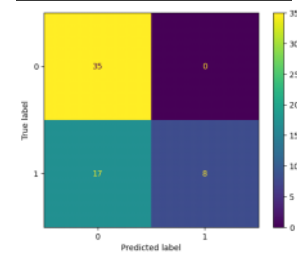
```

#Vous pouvez faire la meme chose avec d'autres modèle ci besoin eg.KNN
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier()
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', model)
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_true = y_test, y_pred = y_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot()

```

	precision	recall	f1-score	support
0	0.67	1.00	0.80	35
1	1.00	0.32	0.48	25
accuracy			0.72	60
macro avg	0.84	0.66	0.64	60
weighted avg	0.81	0.72	0.67	60



CROSS VALIDATION

vendredi 23 mai 2025 17:51

CROSS VALIDATION POUR L'ENTRAINEMENT DES DONNEES

Le cross validation est plus précis que le train test split

La technique du cross validation, utilise 3 ensemble différents (train, validation et test), sans pour autant réduire le train set.

```
from sklearn.model_selection import cross_validate
```

```
Exemple : cross = cross_validate(LinearRegression(),X, y, score = "r2", cv=5)  
cv = cross_validate(LinearRgression(), X,y, scoring = " neg-root_mean_squared_error")  
cv["test_srore"].mean()
```

CLUSTERING

lundi 26 mai 2025 20:40

LiveCoding

<https://colab.research.google.com/drive/1l85re6YX6i9zHqAOkkMKwNx86d2vrv05>

```
from sklearn.cluster import Kmeans
from sklearn.metrics import silhouette_score
```

1 - Création du modèle K-means

```
modelKM = KMeans(n_clusters=2, random_state=3)
modelKM.fit(X)
```

2 - Affichage centre des Clusters

```
modelKM.cluster_centers_
array([[5.00566038, 3.36981132, 1.56037736, 0.29056604],
       [6.30103093, 2.88659794, 4.95876289, 1.69587629]])
```

3 - Affichage du cluster d'appartenance

Pour chaque ligne de X, tu peux obtenir le cluster d'appartenance, c'est-à-dire celui qui a le centre de cluster le plus proche.

```
modelKM.labels_
```

4 - Affichage du cluster d'appartenance

L'inertie représente la notion d'éparpillement des points les uns par rapport aux autres.

Un bon clustering a :

une inertie inter-classe grande : les clusters sont bien séparés les uns des autres

une inertie intra-classe petite : les points à l'intérieur d'un cluster sont denses

On peut obtenir l'inertie avec la commande suivante. C'est une valeur sans échelle, liée à la distance globale. Elle est difficile à interpréter, mais nous nous en servirons dans la **méthode Elbow**.

```
modelKM.inertia_
```

Pour connaître le nombre de cluster optimal, on utilise la méthode **Elbow** et ou **Silhouette**.

Méthode ELBOW

Méthode Silhouette

Le score Silhouette est plus précis. Chaque point obtient un score entre -1 et 1.

- 1 : le point est très mal placé, il devrait sans doute être dans un autre cluster
- 0 : il y a hésitation dans le cluster d'appartenance (2 clusters ou plus sont à la même distance)
- 1 : le point est au même endroit que le centre d'un cluster

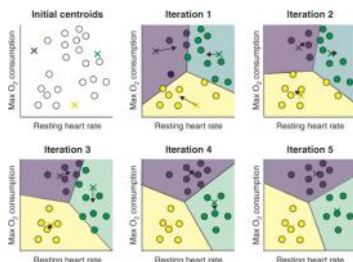
Ensuite on calcule la moyenne de l'ensemble des scores. S'il est proche de 1, tous les points sont très proches de leur clusters.

```
# Ici, nous faisons une boucle pour afficher le score Silhouette
# pour un nombre de clusters entre 2 et 8
from sklearn.metrics import silhouette_score
for k in range(2,9):
    modelKM = KMeans(n_clusters=k, random_state=3)
    modelKM.fit(X)
    print(silhouette_score(X, modelKM.labels_))
```

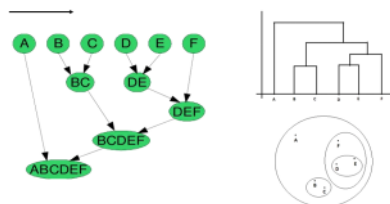
Apprentissage non supervisé

Mesure de similarité : Euclidienne L2, Manhattan L1, Cosinus etc ...

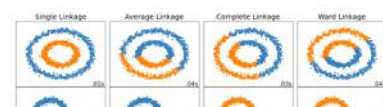
- Algorithme K-Means, partage les données en K clusters en minimisant la distance intra-clusters.



- Le Hierarchical Clustering crée une hiérarchie de clusters sous forme d'arbre (dendrogramme).

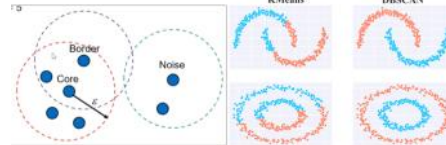


- Hierarchical Clustering : le "linkage" :
 - o Single (distance minimale entre deux clusters)
 - o Complete (distance maximale entre deux clusters)
 - o Average (moyenne)
 - o Ward (variance intra cluster)



DBSCAN / HDBSCAN

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifie des clusters denses basés sur un seuil de distance et un nombre minimum de points.



- HDBSCAN (Hierarchical DBSCAN) corrige les problèmes de densité

Les Distances en ML

Distance euclidienne (L2)
Distance Manhattan (L1)
Distance Minkowsky (Lp)
Distance de Chebyshev (L ∞)
Distance Haversine
Distance de Hamming
Cosine similarity (similitude cosinus)

KMeans Elbow / Silhouette

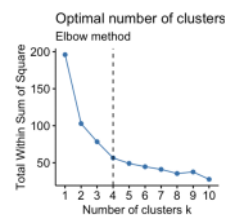
L'algorithme KMeans repose sur le nombre de clusters, k, afin de séparer au mieux les données en groupes homogènes. Se pose alors la question de savoir comment déterminer k afin d'obtenir le meilleur résultat.

Pour cela, on utilise trois méthodes :

1. Elbow
2. Silhouette
3. Calinski-Harabasz

KMeans : Elbow

La méthode **Elbow** permet de déterminer le nombre de clusters en calculant la **distance intra-cluster**. Elle est basée sur l'inertie du modèle (inertia_ égale à la somme des carrés des distances des points au sein de chaque cluster).

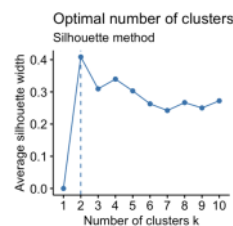


KMeans : Silhouette

La méthode **Silhouette** permet de déterminer le nombre de clusters en calculant la **distance entre les clusters**. Elle renvoie un score entre -1 et 1 :

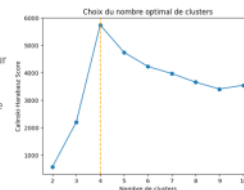
- 1 : le point est bien classé
- 0 : il y a un doute
- -1 : le point est sans doute mal classé

Le score est la moyenne obtenue pour tous les points.



KMeans : Calinski-Harabasz

La méthode **Calinski-Harabasz** calcule la distance intraculuster et intercluster pour déterminer le nombre de clusters optimal. Elle renvoie un score entre 0 et + ∞ . Plus le score est élevé, mieux c'est.



Optimisation des Hyperparamètres

GridSearch

- **Principe** : teste **toutes les combinaisons** possibles d'un grille de paramètres.
- **Exhaustif** mais **coûteux** en temps de calcul.
- Idéal pour un **petit espace** hyperparamétrique

```
from sklearn.model_selection import GridSearchCV

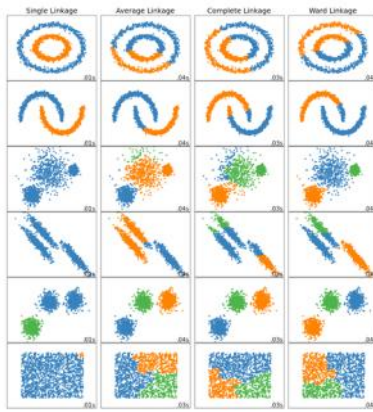
param_grid = {'n_neighbors': [3, 5, 7]}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

RandomizeSearchCV

- **Principe** : explore **aléatoirement** un sous-ensemble d'une distribution de paramètres.
- **Moins coûteux**, utile pour les grilles larges ou continues
- Il faut choisir le **nombre d'itérations**.

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {'n_neighbors': range(1, 11, 2), 'metric': ['minkowski', 'euclidean']}
random_search = RandomizedSearchCV(KNeighborsClassifier(), param_distributions=param_dist, n_iter=20, cv=5)
random_search.fit(X_train, y_train)
print(random_search.best_params_)
```



```
param_dist = {'n_neighbors': range(1,11,2), metric = ['minkowski', 'euclidean']}
random_search = RandomizedSearchCV(KM(), param_distributions=param_dist, n_iter=20, cv=5)
random_search.fit(X_train, y_train)
print(random_search.best_params_)
```

Code Antoine Daras

```
X = df.select_dtypes('number')
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
modelKM = KMeans(n_clusters=3)
modelKM.fit(X_scaled)
```

```
df['cluster'] = modelKM.labels_
df
```

```
# Les données ayant été standardisées, on utilise
inverse_transform()
# pour remettre les centres des clusters à l'échelle
d'origine.
centers = modelKM.cluster_centers_
centers_original = scaler.inverse_transform(centers)
centers_original
```

```
fig, ax = plt.subplots(1, 2, figsize=(10,6))
ax[0].set_title("3 clusters")
sns.scatterplot(data = df, x='sepal_length', y
='petal_width', hue = 'cluster', ax= ax[0])
ax[0].scatter(
    centers_original[:, 0], centers_original[:, 3],
    color='green', marker='x', s=200, label='Centres')
```

```
ax[1].set_title("3 original species")
sns.scatterplot(data = df, x='sepal_length', y
='petal_width', hue= 'species', ax= ax[1])
fig.show()
```

```
silhouette_score_list = []
for k in range (2,11):
    modelKM = KMeans(n_clusters=k, random_state=42)
    modelKM.fit_predict(X_scaled)
    silhouette_score_list.append(silhouette_score(X_scaled, modelKM.labels_))
    #print(silhouette_score(X_scaled, modelKM.labels_))
```

```
plt.plot(range(2,11),silhouette_score_list)
plt.show()
```

```
from yellowbrick.cluster import SilhouetteVisualizer
for k in range(2, 11):
    fig, ax = plt.subplots()
    model = KMeans(n_clusters=k, random_state=42)
    visualizer = SilhouetteVisualizer(model, ax=ax)
    visualizer.fit(X_scaled)
    visualizer.show()
```

Git

CI = Intégration Continue
CD = Déploiement Continu

- 3 zones :
- 1 - Zone de travail
 - 2 - Zone de préparation
 - 3 - Zone Local Repository en lien avec GitHub

- 1. Pour créer un dépôt Git **git init**
Vérifier l'état du dépôt **git status**
- 2. Préparation git **add .** ou **git add nom_fichier** (envoi le ou les fichier dans le staging ou index)
- 3. Transfer dans le Repository **git commit -m "message explicite de la modification"**
Vérifier l'état du dépôt **git status**
- 4. Pousser le ou les fichiers dans le Repository

git push : envoi le code sur github
-u : configure main comme branche par défaut pour les futurs **git push**
origin : indique le repo distant avec qui on est connecté
main : indique ou stocker le code
Une fois la commande **git push -u origin main** exécuté, git saura ou envoyer le code et on pourra simplement faire un **git push**.

Vulgarisation "Envoie mon code sur GitHub (origin) et place-le dans main."

git remote add <nom-du-dépôt> <url-du-dépôt>

Pour voir l'historique des modifs(commit) : **git log** ou **git log --oneline** ou **git log -p** (plus précis)

Exemple :
git status (ne prend pas en compte le fichier modifier)
git add notes.txt (glisse dans la zone de préparation)
git commit -m "message ..."
git status pour vérifier à nous l'état

.gitignore

Créer un fichier nommé **.gitignore**

git pull (récupérer les éléments distants sur github)

git Pull origin main
git pull : récupérer les modification sur le repo
origin : emplacement distant du repo (appelé remote en git, ici nommé origin)
main : branche principale du projet

Les Branches (ne jamais travailler sur main)

Sur Github (interface graphique) :

- 1. Créer un branche
- 2. Générer un nouveau fichier
- 3. Pull Request (de la branche créée avec branche supérieure) - bien documenter les changements
- 4. Fusionner / Merge

En LOCAL ligne de commande sur le Terminal :

git branch nom_de_la_branche
(attention la place au-dessus automatiquement)
git switch ma-branch : changer de branche
git switch -c nouvelle-branch : créer et bascule vers la nouvelle branche

Voir aussi **git checkout**

Git clone dans le Terminal (recupérer un repo distant)

git clone url_du_repo (utiliser les commande ls ou dir)
git branch -a : permet de vérifier les branches disponibles
git branch -git branch -r : permet de voir toutes les branches distantes

Gestion des Conflits

git diff : voir différence entre 2 fichiers à faire avant **git add** et **git commit**
git diff main origin/main : voir différence repo local et repo distant

git fetch : vérifie si github a des mise à jour sans toucher à mon code
*Il regarde si mon collègue a fait des changements sur GitHub.
Il ne les applique pas encore sur mon espace de travail*

git pull : récupérer et applique les changements de github
Quand tu veux mettre à jour ton repo local avec les nouvelles modifications de ton collègue

git merge : fusionne des branches en ligne de commande
Exemple :

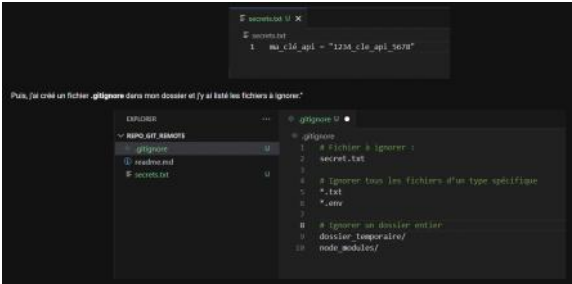
git switch main	# Je me place sur la branche main
git merge features	# Je fusionne features dans main

VS CODE / TERMINAL

Opérateur sur les fichiers
pwd affiche le répertoire dans lequel on se retrouve
ls : **ls -R *.py** affiche tous les fichiers avec l'extension py à partir du dossier actuel
ls -a affiche tous les fichiers, y compris les fichiers cachés
cp : (copy) copie un fichier
Exemple : **cp fichier.txt copie_fichier.txt**
mv : (move) déplace ou renomme un fichier
Exemple : **mv fichier.txt fichier_renomme.txt**
rm : (remove) supprime un fichier
Exemple : **rm fichier.txt** efface le fichier **fichier.txt**
alors que **rm -r** dir efface le dossier **dir** et les fichiers qu'il contient.
rmdir : supprime un dossier vide
find : chercher un fichier sur le disque
Exemple : **find . *.py** cherche tous les fichiers avec l'extension py à partir du dossier actuel (symbolisé par .)
touch : crée, s'il n'existe pas déjà, un fichier
mkdir : crée un dossier (répertoire)
Exemple : **touch fichier.txt**
grep : recherche des fichiers contenant une chaîne de caractères spécifiques
Exemple : **grep -l streamlit *.py** affiche les fichiers d'extension .py qui contiennent la chaîne « streamlit »

Entrées / Sorties
cat : affiche le contenu d'un fichier Exemple : **cat file.txt**
head et **tail** : affiche les premières/dernières lignes d'un fichier Exemple : **head file.txt**
echo affiche du texte dans le terminal Exemple : **echo hello world!**
> : redirige la sortie, par exemple vers un fichier Exemple : **echo hello world! > fichier.txt**
crée un fichier (fichier.txt) (ou le remplace s'il existe déjà) et y stocke la chaîne de caractère « hello world! »
ou bien **ls > liste_fichiers.txt** sauvegarde la liste des fichiers fournie par la commande **ls** dans le fichier **liste_fichiers.txt**
>> : similaire à >, mais ajoute le texte à la fin du fichier

Importer une bibliothèque (package)
pip install nom_package
pip install nom_package==1.41.1 (installe une version spécifique)
pip install -U nom_package (met à jour une bibliothèque déjà installée)
pip show numpy (montre la version installée)
pip list (montre toutes les bibliothèques installées)
Créer un environnement virtuel
Python -m venv nom_de_l'environnement
Activer un environnement virtuel
nom_de_l'environnementScripts/activate
Installer une bibliothèque
pip install numpy pandas matplotlib
Pour partager les bibliothèques avec d'autres utilisateurs
pip list --not-required > requirements.txt
Installation environnement virtuel d'un autre utilisateur
Créer un environnement virtuel **python -m venv env**
L'activer **envScripts/activate**
Installer les bibliothèques nécessaires **pip install -r requirements.txt**
Supprimer un environnement virtuel **rm -rf nom_de_l'environnement**



Récapitulatif des commandes git diff, git fetch, git pull

Commande	Action	Quand utiliser ?
git diff	Voir TES modifications locales	Avant git add
git diff --cached	Voir les différences entre l'index et le dernier commit	Avant git commit
git fetch	Récupérer les mises à jour de GitHub sans les appliquer	Avant un git pull pour voir ce qui a changé
git diff main origin/main	Comparer ta branche locale et la version distante	Après un git fetch pour voir les différences
git pull	Récupérer ET appliquer les changements	Pour synchroniser ton code avec GitHub

`git branch -d nom-de-la-branche` : supprime la branche qui a été fusionnée,
il est recommandé de la supprimer pour garder un repo propre.

`git merge` fusionne les branches en local, après fusion faire un `git push` pour actualiser github

Github

First_token : ghp_54ckbv48k6mzICYjjPbaH7loBK75pZ1dX7TT
(valable jusqu'au 18 aout 2025)

1. Créer et préparer le Repository = repo : `git init`
 2. Connecter le repo local à Github `git remote add origin https://github.com/ton-nom/ton-repo.git`
`git remote` : concerne la gestion des liens vers les repos distants sur github
`add` : ajout d'un nouveau lien
`origin` : nom que l'on donne au repo distant, par convention le repo principal est "origin"
`https://github.com/ton-nom/ton-repo.git` : adresse du repo sur github
 3. Pour vérifier que la connexion est bien établie `git remote -v`
 4. Si adresse url pas bonne la modifier avec `git remote set-url origin adresse_url_correcte_ici`
-

Manipuler des données avec Streamlit

Streamlit permet de facilement lire des données et de les afficher dans vos applications web. Vous trouverez ces fonctions [ici](#).

Voyons les principales fonctions :

- **st.dataframe()** : Elle prend un objet DataFrame en argument et l'affiche dans un format tabulaire clair et lisible.
- **st.table()** : Elle prend une structure de données compatible (liste de listes, NumPy array, DataFrame) et l'affiche dans une table HTML.
- **st.json()** : Elle prend un objet JSON en argument et l'affiche dans un format clair et indenté, facilitant sa lecture et sa compréhension.
- **st.metric()** : elle affiche un indicateur visuel clair et concis pour une métrique importante, comme la progression d'un processus, la performance d'un système ou la valeur d'un KPI.

Les fonctions suivantes sont un peu plus évoluées. Lisez la documentation pour mieux savoir vous en servir :

- **st.data_editor()** : Elle prend un objet DataFrame ou une liste de listes comme argument et affiche un éditeur de données interactif. Les utilisateurs peuvent visualiser, modifier et enregistrer les données directement dans l'application.
- **st.column_config** : Elle prend le nom d'une colonne et définit des options

À partir de l'adresse <<https://odyssey.wildcodeschool.com/expeditions/12594/quests/3089/pages/18012>>

Afficher un graphique avec Streamlit

Il est également possible de créer des graphiques avec des fonctions propres à Streamlit. Les voici :

- **st.area_chart()** : Cette fonction permet de créer un **graphique à aires pour visualiser des données continues qui changent au fil du temps**. Elle est particulièrement utile pour représenter des tendances, des cumuls ou des distributions de données.
- **st.bar_chart()** : Cette fonction permet de créer un **graphique à barres pour comparer des valeurs catégorielles ou numériques**. Elle est utile pour visualiser des distributions de données, des comparaisons entre groupes ou des progressions temporelles.
- **st.line_chart()** : Cette fonction permet de créer un **graphique linéaire pour visualiser des relations entre des variables continues**. Elle est utile pour montrer des tendances, des corrélations ou des progressions temporelles.
- **st.map()** : Cette fonction permet de créer une **carte pour visualiser des données géographiques**. Elle est utile pour afficher la distribution de données sur une carte, les tendances régionales ou les comparaisons géographiques. Elle se base sur la valeur de la latitude et de la longitude.
- **st.scatter_chart()** : Cette fonction permet de créer un **diagramme à nuages de points** pour visualiser la relation entre deux variables continues. Elle est utile pour identifier des tendances, des regroupements, des valeurs aberrantes et des corrélations.

À partir de l'adresse <<https://odyssey.wildcodeschool.com/expeditions/12594/quests/3089/pages/18108>>

Afficher un graphique avec Matplotlib Afficher un graphique avec Seaborn

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3
4 # Données d'exemple
5 x = [1, 2, 3, 4, 5]
6 y = [3, 5, 7, 2, 1]
7
8 # Création de la figure du graphique
9 fig, ax = plt.subplots()
10 ax.plot(x, y)
11 plt.xlabel("Axe X")
12 plt.ylabel("Axe Y")
13 plt.title("Graphique linéaire")
14
15 # Afficher du graphique avec Streamlit
16 st.pyplot(fig)
17
```

```
1 import streamlit as st
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Charger le dataset Iris
6 data = sns.load_dataset('iris')
7
8 # Créer un graphique barplot
9 sns.barplot(x='species', y='sepal_length', data=data)
10
11 # Afficher le graphique avec Streamlit
12 st.pyplot(plt.gcf())
```

Afficher un graphique avec Plotly

```
1 import streamlit as st
2 import seaborn as sns
3 import plotly.express as px
4
5 iris = sns.load_dataset("iris")
6 fig = px.scatter(
7     iris,
8     x="sepal_width",
9     y="sepal_length",
10    color="species",
11    size="petal_length",
12    hover_data=["petal_width"])
13
14 st.plotly_chart(fig)
```

Amélioration fonctionnalités du site

Générer un menu interactif

```
pip install streamlit-option-menu
import streamlit as st
from streamlit_option_menu import option_menu
```



```
1. Installer le module Streamlit Option Menu avec pip

1 pip install streamlit-option-menu

2. Importer le module, créer le menu et récupérer la valeur sélectionnée

1 import streamlit as st
2 # Importation du module
3 from streamlit_option_menu import option_menu
4
5 # Création du menu qui va afficher les choix qui se trouvent dans la variable options
6 selection = option_menu(
7     menu_title=None,
8     options = ["Accueil", "Photos"]
9 )

D'autres paramètres permettent de styliser et personnaliser vos menus de navigation, tels que icons, default\_index, orientation, etc.

3. En fonction de l'option sélectionnée afficher le contenu correspondant dans votre application

1 # On indique au programme quoi faire en fonction du choix
2 if selection == "Accueil":
3     st.write("Bienvenue sur la page d'accueil !")
4 elif selection == "Photos":
5     st.write("Bienvenue sur mon album photo")
6 # ... et ainsi de suite pour les autres pages
```

Organiser l'affichage en colonne

```
1 import streamlit as st
2
3 # Création de 3 colonnes
4 col1, col2, col3 = st.columns(3)
5
6 # Contenu de la première colonne :
7 with col1:
8     st.header("A cat")
9     st.image("https://static.streamlit.io/examples/cat.jpg")
10
11 # Contenu de la deuxième colonne :
12 with col2:
13     st.header("A dog")
14     st.image("https://static.streamlit.io/examples/dog.jpg")
15
16 # Contenu de la troisième colonne :
17 with col3:
18     st.header("An owl")
19     st.image("https://static.streamlit.io/examples/owl.jpg")
20
```

Sidebar

```
1 import streamlit as st
2
3 # On affiche un menu déroulant (selectbox) DANS la barre latérale (sidebar)
4 # L'utilisateur peut choisir son moyen de contact préféré parmi trois options
5 add_selectbox = st.sidebar.selectbox(
6     "How would you like to be contacted?", # Question affichée
7     ("Email", "Home phone", "Mobile phone") # Options proposées
8 )
9
10 # Autre façon d'utiliser la sidebar avec un "with", pour grouper plusieurs éléments
11 with st.sidebar:
12     # On affiche des boutons radio dans la sidebar pour choisir un mode de livraison
13     add_radio = st.radio(
14         "Choose a shipping method", # Titre de la question
15         ("Standard (5-15 days)", "Express (2-5 days)") # Choix proposés
16     )
17
```

Sécuriser les Appli avec Authentification simple

```
pip install streamlit-authenticator
import streamlit as st
from streamlit_option_menu import option_menu
from streamlit_authenticator import Authenticator
```

```

1 import streamlit as st
2 from streamlit_authenticator import Authenticate
3
4 # Nos données utilisateurs doivent respecter ce format
5 lesDonneesDesComptes = {
6     'usernames': {
7         'utilisateur': {
8             'name': 'utilisateur',
9             'password': 'utilisateurMDP',
10            'email': 'utilisateur@gmail.com',
11            'failed_login_attempts': 0, # Sera géré automatiquement
12            'logged_in': False, # Sera géré automatiquement
13            'role': 'utilisateur'
14        },
15        'root': {
16            'name': 'root',
17            'password': 'rootMDP',
18            'email': 'admin@gmail.com',
19            'failed_login_attempts': 0, # Sera géré automatiquement
20            'logged_in': False, # Sera géré automatiquement
21            'role': 'administrateur'
22        }
23    }
24 }
25
26 authenticator = Authenticate(
27     lesDonneesDesComptes, # Les données des comptes
28     "cookie name", # Le nom du cookie, un str quelconque
29     "cookie key", # La clé du cookie, un str quelconque
30     30, # Le nombre de jours avant que le cookie expire
31 )

```

Utiliser `authenticator.login()` pour afficher le formulaire de connexion

```

1 def accueil():
2     st.title("Bienvenu sur le contenu réservé aux utilisateurs connectés")
3
4
5 if st.session_state["authentication_status"]:
6     accueil()
7     # Le bouton de déconnexion
8     authenticator.logout("Déconnexion")
9
10 elif st.session_state["authentication_status"] is False:
11     st.error("L'username ou le password est/sont incorrect")
12 elif st.session_state["authentication_status"] is None:
13     st.warning('Les champs username et mot de passe doivent être remplie')

```

<code>st.session_state()</code>	Cela permet de conserver les informations même lorsque l'utilisateur interagit avec l'application, comme après un rafraîchissement de la page ou un changement d'état.
---------------------------------	--

Publication sur Stream Cloud

1 - Allez sur <https://streamlit.io/>

2 - Déposer l'application Streamlit (au minimum le code et le requirements.txt) dans un repository Github

Créez un repository Github et déposez y :

- Le fichier python de votre application,
- Le fichier requirements.txt qui contient l'architecture de votre environnement virtuel
- Tout autre fichier dont votre application a besoin pour fonctionner

Pour générer `requirements.txt`, taper dans le Terminal `pip freeze > requirements.txt`

3 - Publier l'appli sur Stream Cloud

- Connectez-vous à votre compte Streamlit Cloud.
- Cliquez sur le bouton "Create app" et sélectionné l'option GitHub
- Remplissez les instructions à l'écran et cliquez sur "Déploy" À ce stade votre application n'est visible que de vous.
- Pour la rendre accessible à toute personne disposant du lien vous devez modifier un paramètre dans les settings. Cliquez sur les 3 petits points à droite de votre applicationnet ensuite settings. Ensuite cliquez dans "Sharing" et choisissez l'option `This app is public and searchable`.

- Dans l'outil en ligne effectue une recherche en utilisant `\s`

`\w` : Pour rechercher tous les caractères alphanumériques (lettres et chiffres) dans le texte

- Dans l'outil en ligne effectue une recherche en utilisant `\w`

- Recherche d'une liste de motifs.
Par exemple pour rechercher toutes les lettres 'o' 'm' et 'a' utilise le motif `[oma]` dans l'outil en ligne
- Recherche d'une plage de motifs.
Par exemple pour rechercher toutes les lettres allant de 'a' 'm' utilise le motif `[a-m]` dans l'outil en ligne
- Et pour exclure un ensemble de caractère de la recherche,
on rajoute `^` dans le motif. Dans l'outil en ligne utilise le motif `[^a-d]` pour sélectionner tous les caractères sauf les lettres de 'a' 'b', 'c' et 'd'.

API GéoCodage Folium

mardi 27 mai 2025 09:15



12W Cours
FastAPI

```
import folium
```

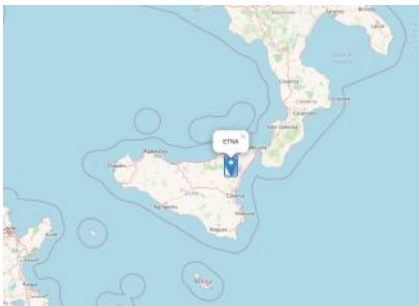
```
# Trouver un point avec les coordonnées
```

```
m = folium.Map(location=[37.6, 14.2], zoom_start=7)  
m
```



```
# ajouter un repère sur la carte
```

```
m = folium.Map(location=[37.6, 14.2], zoom_start=7)  
folium.Marker(  
    location=[37.76, 14.98],  
    popup='ETNA'  
).add_to(m)  
m
```



```
import folium
```

Api adresse.gouv : <https://geo.api.gouv.fr/adresse>

Exo 1 :

<https://colab.research.google.com/drive/1mOFv1ga1Nouf2XE3Fzekpt1D6yKkwjS#scrollTo=e7dNMdT8WU8h>

Exo 2 :

<https://colab.research.google.com/drive/1mOFv1ga1Nouf2XE3Fzekpt1D6yKkwjS>

Live Code API / Geocoding :

<https://colab.research.google.com/drive/1QCNfQpRmHpNsp4EAb0GzqQVwkaqbqffz>

Cours sur les API



9W Cours
API REST

Requête GET :

Exemple de requête

Ce code montre comment faire une requête **GET** pour obtenir les prévisions météo de Paris.
L'attribut **status_code** permet de s'assurer que la réponse obtenue est correcte (200).

```
import requests  
  
response =  
requests.get('https://api.openweathermap.org/data/2.5/weather?q=Paris')  
if response.status_code == 200:  
    data = response.json()  
    print(data)  
else:  
    print('Erreur:', response.status_code)
```

Réponse json : *comparable à un dictionnaire*

```
{  
  "ville": "Paris",  
  "température": 18.5,  
  "météo": "Ensoleillé",  
  "humidité": 65  
}
```

Status code / code d'erreur :

Les status codes

Les **status codes** retournés par l'API se regroupent en **quatre groupes** :

Plage	Type	Signification
200	✓ Succès	La requête a réussi.
300	🔄 Redirection	L' URL a changé ou est ailleurs temporairement.
400	✗ Erreur client	L' utilisateur a fait une erreur.
500	⚠ Erreur serveur	Le serveur a un problème.

Voici les **status codes** courants :

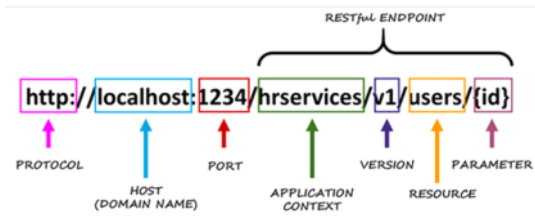
Code	Type	Signification
400	Bad Request	La requête est mal formulée (par ex. Erreur de paramètre)
401	Unauthorized	Clef API ou token manquant ou incorrect
403	Forbidden	Accès interdit , même avec une clef
404	Not Found	Erreur d' URL
429	Too Many Requests	Limite de requêtes autorisées dépassée
500	Internal Server Error	Le serveur a un bug ou une panne
503	Service Unavailable	L'API est temporairement hors ligne ou trop sollicitée

Si besoin d'un Token pour avoir accès :

```
url = 'https://api.example.com/data'
headers = {
    'Authorization': 'Bearer YOUR_ACCESS_TOKEN'
}

response = requests.get(url, headers=headers)
```

Éléments d'une url - **end point** pour accéder à la **ressource** :



Cours Json (java script object notation)



9W Cours
Le format ...

```
import json
```

Il existe **2 méthodes principales** dans le module json, pour lire et écrire des fichiers JSON : **load** (lecture) et **dump** (écriture). On ajoute un "s" à la fin lorsque l'on traite des chaînes de caractères (string).

Fonction	Description
<code>json.load(file)</code>	Lit un fichier .json → objet Python
<code>json.dump(file)</code>	Écrit un objet Python → fichier JSON
<code>json.loads(string)</code>	Convertit une chaîne JSON → objet Python
<code>json.dumps(json)</code>	Convertit un objet Python → chaîne JSON

```
data = """{"entier": 0,
          "dico" : {"clef1" : 1, "clef2": 2},
          "chaîne" : "hello !"}"""
data = json.loads(data)
data["dico"]["clef1"] # Retourne 1
```

Pour interroger une **API REST** et obtenir en retour un JSON, on utilise la bibliothèque **requests** avec la méthode **get** (lecture).

```
import requests

r = requests.get("https://api.exemple.com/data")
```

L'objet retourné par requests dispose de l'attribut **text** et la méthode **json()**.

Expression	Description
<code>r.text</code>	Le contenu brut (chaîne JSON)
<code>r.json()</code>	Convertit en objet Python (dict/liste)

Note :
`r.json()` est équivalent à `json.loads(r.text)`

On peut convertir un objet JSON en DataFrame. S'il est **"plat"** (les valeurs ne sont pas des dictionnaires), on utilise la méthode **read_json()** de pandas.

```
import pandas as pd

df = pd.read_json("fichier_ou_url.json")
```

[Request API data using Python in 8 minutes!](#)



```
import requests

base_url = "https://pokeapi.co/api/v2/"

def get_pokemon_info(name):
    url = f"{base_url}/pokemon/{name}"
    response = requests.get(url)

    if response.status_code == 200:
        pokemon_data = response.json()
        return pokemon_data
    else:
        print(f"Echec récupération des données {response.status_code}")

pokemon_name = "pikachu"

pokemon_info = get_pokemon_info(pokemon_name)

if pokemon_info:
    print(f"Name: {pokemon_info['name'].capitalize()}")
    print(f"Id: {pokemon_info['id']}")
    print(f"Height: {pokemon_info['height']}")
    print(f"Weight: {pokemon_info['weight']}")
```

Si l'objet JSON est "imbriqué" (les valeurs peuvent être des dictionnaires/objets JSON), on utilise la méthode `json_normalize()` de pandas.

```
import pandas as pd

df = json_normalize(data, record_path=..., meta=...)
```

- **record_path** : chemin vers une liste imbriquée
⇒ Chaque **clé** de la liste devient **une colonne** du DataFrame
⇒ Chaque **élément** de la liste devient une **ligne** du DataFrame
- **meta** : **clés** à extraire en plus, en **contexte** pour chaque ligne

```
data = {
  "école": "Lycée A",
  "élèves": [
    {"nom": "Alice", "âge": 15},
    {"nom": "Bob", "âge": 16}
  ]
}

json_normalize(data,
               record_path="élèves",
               meta="école")
```

nom	âge	école
Alice	15	Lycée A
Bob	16	Lycée A

Attention !

On ne peut pas parcourir plusieurs listes en parallèle. Dans ce cas, il faut créer plusieurs DataFrames ou réorganiser son JSON...

lundi 2 juin 2025 08:26

Live Code NLP & WordCloud + Machine Learning :
<https://colab.research.google.com/drive/1Yq3YrZrpmPDI0et3eHaNImBVY3AFYNjs>

Avant d'utiliser les données d'un texte :

```
import nltk
nltk.download()
Ou
import nltk
nltk.download('popular')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem.snowball import SnowballStemmer
```

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

NLP Page 37

plus fréquents

Par exemple, nous pouvons prendre que les 2 mots les plus présents dans le dataset :

```
vectorizer = CountVectorizer(max_features = 2)
vectorizer.fit(df['sentence'])
vectorizer.get_feature_names_out()
```

Pour afficher la liste des noms : `get_feature_names_out()`

```
vectorizer.get_feature_names_out()
>>> ['apples', 'do', 'good', 'is', 'it', 'like', 'not', 'you']
```

Pour transformer en corpus de texte

```
sparse_matrix = vectorizer.transform(df['sentence'])
```

Pour le reconverter (c'est déconseillé si la matrice est grande) en matrice "dense", tu peux

utiliser la fonction suivante : `sparse_matrix.toarray()`

BOW et Machine Learning

L'étape de train-test split doit être réalisé avant l'entraînement du CountVectorizer pour s'assurer qu'il n'y ait pas d'overfitting.

Les étapes sont donc les suivantes :

- Train test split
- Fit et transform du Vectorizer sur le train set
- Fit et score du classifieur sur le train set
- Transform (sans fit !) du Vectorizer sur le test set
- Predict et score du classifieur sur le test set

TF-IDF (plus précis que le bow)

Voir [quête](https://colab.research.google.com/drive/1eLUcFWQYBZjMA8rk8kYelbet2HZScCWf2usp=sharing) <https://colab.research.google.com/drive/1eLUcFWQYBZjMA8rk8kYelbet2HZScCWf2usp=sharing>

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer()
tfidf.fit(df['sentence'])
```

Les paramètres sont globalement les

mêmes : **lowercase, stop_words, ngram_range, max_features**

<https://colab.research.google.com/drive/18VvtG5cmC1qK2esRncDkrtXwRRcvC2usp=sharing>

```
import pandas as pd
df = pd.read_excel("content/drive/MyDrive/Consumer_Complaints.xlsx")
```

```
df['Issue'] # la colonne à analyser
```

```
# 2. Nettoyage du texte (preprocessing)
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
import re
stop_words = set(stopwords.words('english'))
def clean_text(text):
    text = text.lower()
    text = re.sub(r"([^\w\s])", "", text) # retire punctuation
    words = text.split()
    return ' '.join([w for w in words if w not in stop_words])
clean_texts = [clean_text(t) for t in df['Issue']]
```

```
# 3. Tokenisation
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt_tab')
tokenized = [word_tokenize(t) for t in clean_texts]
```

```
##### 3.1 nuage de mots
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import spacy
nlp = spacy.load("en_core_web_sm")
nlp.max_length = 2_000_000 # pour les grand dataframe de base limité à 1 000 000 mots
wordcloud = WordCloud(width= 480, height=400, background_color="black")
sentence = " ".join(clean_texts)
sent_tokens = nlp(sentence)
dico = nltk.FreqDist([token.lemma_ for token in sent_tokens])
wordcloud.generate_from_frequencies(dico)
plt.figure()
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.margins(x=0, y=0)
plt.show()
```

```
# 4. Stemming ou Lemmatisation = choix stemming // Lemmatising plus long et un petit peu plus précis
from nltk.stem.snowball import SnowballStemmer
stemmer = SnowballStemmer('english')
stemmed_texts = [stemmer.stem(word) for word in tokens] for tokens in tokenized]
```

```
# 5 Vectorisation TfidfVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(min_df=3)
X = vectorizer.fit_transform(' '.join(tokens) for tokens in stemmed_texts)]
```

```
# 6. Classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, df['Issue'], test_size=0.2, random_state=42)
model = KNeighborsClassifier(n_neighbors=3, metric='cosine')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
# 7. Évaluation
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score,
ConfusionMatrixDisplay
import seaborn as sns
import matplotlib.pyplot as plt
y_pred = model.predict(X_test)
print("Accuracy/précision:", accuracy_score(y_test, y_pred))
```

```
for token in sent_tokens:
    print(token, token.lemma_)
```

```
>>> You you
      are be
      better well
      when when
      I I
      am be
      well well
```

Exemple de fonction pour nettoyer le code

Live Code NLP + Machine Learning : <https://colab.research.google.com/drive/1Yq3YyRzpmPD0et3eHoNimBVY3AFYNg>

```
texte = "Voici un texte avec une <strong>balise</strong> HTML."
```

```
def clean_html(text):
    return re.sub("<[^>+>", "", text)
clean_html(texte)
```

```
>>> Voici un texte avec une balise HTML.
```

```
text = "I really don't like this movie, it's really fun ! @;"
def clean_stopwords(text):
    return ' '.join([word for word in re.sub("[^\w\s]", "", text).split() if word not in stop_words])
```

```
clean_stopwords(text.lower())
```

```
>>> really don t like really fun
```

```
def stem(text):
    return ' '.join([stemmer.stem(word) for word in text.split()])
stem('really don t like playing football')
```

```
>>> realli don t like play footbal
```

Fonction qui regroupe les fct ci-dessus :

```
def clean_text(text):
    text = text.lower()
    text = stem(text)
    return text
```

```
text = "I really don't like playing basketball, it's not really fun ! @;"
```

```
clean_text(text)
```

```
>>> realli don t like play basketbal not realli fun
```

```
print("F1-score:", f1_score(y_test, y_pred, average='weighted'))
print("Matrice de confusion:\n", confusion_matrix(y_test, y_pred))
print("Rapport de classification:\n", classification_report(y_test, y_pred))
```

```
# 8 prediction sur nouveau commentaire
nouveau_texte = "Took or threatened to take legal action"
nettoye = clean_text(nouveau_texte)
vecteur = vectorizer.transform([nettoye])
prediction = model.predict(vecteur)
print("Classe prédite :", prediction[0])
```

Pydantic Normalisation des données

mercredi 4 juin 2025 10:59

Voir exemple code Class Client (BaseModel)

EmailStr de Pydantic pour valider les adresses email.
Field pour ajouter des contraintes supplémentaires aux champs.
Optional de typing pour indiquer qu'un champ peut être None.

Définition :
id: int : Un identifiant unique pour chaque client.
nom: str = Field(..., min_length=2, max_length=100) :
... signifie que le champ est obligatoire.
min_length=2, max_length=100 impose des contraintes sur la longueur du nom.
email: EmailStr : Utilise le type spécial EmailStr pour valider automatiquement le format de l'email.
date_inscription: date : Pydantic convertira automatiquement les chaînes de date en objets date.
age: Optional[int] = Field(None, ge=18, le=120) :
Optional[int] indique que l'âge peut être un entier ou None.
Field(None, ge=18, le=120) spécifie que si un âge est fourni, il doit être entre 18 et 120 ans.
ville: str : La ville du client, sans contraintes particulières.
pays: str = Field(..., max_length=50) : Le pays du client, limité à 50 caractères.

Utilisation :
Nous créons un dictionnaire client_data avec des données de test.
Client(client_data)** crée une instance de Client en déballant le dictionnaire.

Pydantic valide automatiquement toutes les données et effectue les conversions nécessaires.

Voir exemple code Class Produit (BaseModel)

Définition :
id: int : Un identifiant unique pour chaque produit.
nom: str = Field(..., min_length=3, max_length=100) :
Impose une longueur minimale de 3 et maximale de 100 caractères pour le nom du produit.
prix: float = Field(..., gt=0) :
gt=0 signifie "greater than 0", assurant que le prix est toujours positif.
description: str = Field("", max_length=1000) :
La description est optionnelle (chaîne vide par défaut) et limitée à 1000 caractères.
categories: List[str] = Field(..., min_items=1, max_items=5) :
Utilise List[str] pour définir une liste de chaînes.
Impose au moins une catégorie et au maximum 5.

Utilisation :
Nous créons un dictionnaire produit_data avec des données de test.
Produit(produit_data)** crée une instance de Produit en validant les données.

Voir exemple code Class Produit Avancé(BaseModel)

Importation de **field_validator :**
field_validator est un décorateur de Pydantic v2 utilisé pour créer des fonctions de validation personnalisées. On l'utilise toujours avec le décorateur **classmethod**.

Définition :
Les champs sont définis de manière similaire aux exemples précédents.
Validateurs personnalisés :
@field_validator('id') :
o Utilise une expression régulière pour s'assurer que l'ID commence par 'PROD' suivi de 6 chiffres.
o Lève une ValueError si le format est incorrect.
@field_validator('prix') :
o Vérifie que le prix est positif.
o Arrondit le prix à deux décimales.
@field_validator('code_barre') :
o Vérifie que le code-barres contient exactement 13 chiffres.

Fonctionnement des validateurs :
Chaque validateur prend la valeur du champ (**v**) comme argument.
cls est la classe du modèle (*rarement utilisé mais requis par Pydantic*).
Les validateurs peuvent modifier la valeur avant de la retourner.
Pydantic v2 utilise des annotations de type pour une meilleure clarté et performance

Utilisation :
Nous utilisons **ProduitAvance.model_validate(produit_data)** pour créer une instance validée du modèle.
Si une validation échoue, une **ValidationError** est levée avec le message d'erreur spécifié.

Voir exemple code Class Commande(BaseModel)

Structure du modèle :
id, produit, quantite, prix_unitaire : Champs de base d'une commande.
reduction : Champ optionnel (peut être None) pour une éventuelle réduction en pourcentage.
prix_total : Le prix total de la commande, qui sera validé.

Validateur calculer_prix_total
Ce validateur utilise **@model_validator(mode='after')**, une nouveauté de Pydantic v2. Il s'exécute après que tous les champs ont été validés individuellement.
Le mode 'after' permet d'accéder directement aux attributs de l'instance via **self**.
Logique du validateur
Calcule le prix total en utilisant directement les attributs de l'instance.
Arrondit le résultat à deux décimales pour éviter les problèmes de précision des nombres flottants.
Compare le prix calculé avec la valeur fournie, en utilisant une petite tolérance (0.01) pour gérer les erreurs d'arrondi.

Gestion des erreurs
Si le prix total fourni ne correspond pas au calcul, une **ValueError** est levée avec un message explicatif.

Utilisation
Nous utilisons **Commande.model_validate(commande_data)** pour créer et valider

```
from pydantic import BaseModel, EmailStr, Field, field_validator,
model_validator
from typing import Optional, List, dict
from datetime import date
import re
import pandas as pd
```

Exemple :
class Vente(BaseModel):
id: int
date: date
montant: float
produit: str
Utilisation
vente = Vente(id=1, date="2023-08-21", montant="100.50", produit="Laptop")
print(vente)
Sortie: id=1 date=datetime.date(2023, 8, 21) montant=100.5 produit='Laptop'

class Client(BaseModel):
id: int
nom: str = Field(..., min_length=2, max_length=100)
email: EmailStr
date_inscription: date
age: Optional[int] = Field(None, ge=18, le=120)
ville: str
pays: str = Field(..., max_length=50)
Exemple d'utilisation
client_data = {
 "id": 1,
 "nom": "Alice Dupont",
 "email": "alice.dupont@email.com",
 "date_inscription": "2023-01-15",
 "age": 30,
 "ville": "Paris",
 "pays": "France"
}
client = Client(**client_data)
print(client)

class Produit(BaseModel):
id: int
nom: str = Field(..., min_length=3, max_length=100)
prix: float = Field(..., gt=0)
description: str = Field("", max_length=1000)
categories: List[str] = Field(..., min_items=1, max_items=5)
Exemple d'utilisation
produit_data = {
 "id": 1,
 "nom": "Smartphone XYZ",
 "prix": 599.99,
 "description": "Un smartphone haut de gamme avec de nombreuses fonctionnalités",
 "categories": ["Électronique", "Téléphonie"]
}
produit = Produit(**produit_data)
print(produit)

class ProduitAvance(BaseModel):
id: str
nom: str
prix: float
code_barre: str
@field_validator('id')
@classmethod
def id_valide(cls, v: str) -> str:
 if not re.match(r"^[PROD]\d{6}\$", v):
 raise ValueError("L'ID doit être au format PROD suivi de 6 chiffres")
 return v
@field_validator('prix')
@classmethod
def prix_valide(cls, v: float) -> float:
 if v <= 0:
 raise ValueError("Le prix doit être positif")
 return round(v, 2) # Arrondir à deux décimales
@field_validator('code_barre')
@classmethod
def code_barre_valide(cls, v: str) -> str:
 if not re.match(r"^[0-9]{13}\$", v):
 raise ValueError("Le code-barres doit contenir exactement 13 chiffres")
 return v
Exemple d'utilisation
produit_data = {
 "id": "PROD000123",
 "nom": "Téléviseur LED 55 pouces",
 "prix": 699.995,
 "code_barre": "1234567890123"
}
produit = ProduitAvance.model_validate(produit_data)
print(produit)

class Commande(BaseModel):
id: str
produit: str
quantite: int
prix_unitaire: float
reduction: Optional[float] = None
prix_total: float
@model_validator(mode='after')
def calculer_prix_total(self) -> 'Commande':
 prix_calculé = self.quantite * self.prix_unitaire * (1 - self.reduction/100 if self.reduction else 1)
 prix_calculé = round(prix_calculé, 2)
 if abs(self.prix_total - prix_calculé) > 0.01: # Tolérance pour les erreurs d'arrondi
 raise ValueError(f"Le prix total {self.prix_total} ne correspond pas au calcul
(prix_calculé={prix_calculé})
 self.prix_total = prix_calculé
 return self
Exemple d'utilisation
commande_data = {
 "id": "CMD001",
 "produit": "Smartphone XYZ",
 "quantite": 2,
 "prix_unitaire": 599.99,
 "reduction": 10

Gestion des erreurs

Si le prix total fourni ne correspond pas au calcul, une **ValueError** est levée avec un message explicatif.

Utilisation

Nous utilisons **Commande.model_validate(commande_data)** pour créer et valider l'instance.

Le validateur de modèle s'exécute après la validation individuelle des champs.

GESTION DES DONNEES MANQUANTES

Structure du modèle DonneesSondage :

id et **question** sont des champs obligatoires.

reponse est un champ optionnel qui peut être None.

score a une valeur par défaut de 0.0 et doit être entre 0 et 10.

commentaire a une chaîne vide comme valeur par défaut.

Utilisation de Optional :

Optional[str] pour **reponse** indique que ce champ peut être une chaîne ou None.

Cela permet de représenter des questions sans réponse.

Utilisation de Field :

- Field(default=0.0, ge=0.0, le=10.0)** pour **score** :
 - default=0.0** définit la valeur par défaut.
 - ge=0.0** signifie "greater than or equal to 0.0".
 - le=10.0** signifie "less than or equal to 10.0".
- Cela garantit que le score est toujours dans la plage 0-10, même s'il n'est pas fourni.

Valeur par défaut simple :

commentaire: str = "" utilise une simple valeur par défaut sans contraintes supplémentaires.

Exemples d'utilisation :

sondage_complet montre un cas où toutes les données sont fournies.

sondage_partiel montre comment Pydantic gère les champs manquants en utilisant les valeurs par défaut.

DONNEES IMBRIQUES ET LISTES

Modèles imbriqués :

Adresse est un modèle simple pour représenter une adresse.

HistoriqueAchat représente un achat individuel avec une liste de produits.

Structure du modèle ClientComplet :

id et **nom** sont des champs simples.

adresses est une liste d'objets **Adresse**.

HistoriqueAchat est une liste d'objets **HistoriqueAchat**.

preferences est un dictionnaire avec des clés string et des valeurs float.

Utilisation de List et Dict :

List[Adresse] indique une liste d'objets **Adresse**.

List[HistoriqueAchat] pour une liste d'objets **HistoriqueAchat**.

Dict[str, float] pour un dictionnaire avec des clés string et des valeurs float.

Validation imbriquée :

Pydantic valide récursivement toutes les structures imbriquées.

Chaque **Adresse** et **HistoriqueAchat** dans les listes est validé individuellement.

Exemple d'utilisation :

Le dictionnaire **client_data** contient des données complexes et imbriquées.

Lors de la création de **ClientComplet**, Pydantic :

- Valide la structure globale.
- Crée des instances **Adresse** pour chaque adresse.
- Crée des instances **HistoriqueAchat** pour chaque achat.
- Vérifie que les préférences sont bien un dictionnaire de string vers float.

Facilité d'accès : Une fois validé, vous pouvez accéder facilement à des données

imbriquées (par exemple, **client_complet.adresses[0].ville**)

```
# Exemple d'utilisation
commande_data = {
    "id": "CMD001",
    "produit": "Smartphone XYZ",
    "quantite": 2,
    "prix_unitaire": 599.99,
    "reduction": 10,
    "prix_total": 1079.98
}

commande = Commande.model_validate(commande_data)
print(commande)
```

GESTION DES DONNEES MANQUANTES

class DonneesSondage(BaseModel):

```
id: int
question: str
reponse: Optional[str] = None
score: float = Field(default=0.0, ge=0.0, le=10.0)
commentaire: str = ""

# Exemples d'utilisation
sondage_complet = DonneesSondage(id=1, question="Êtes-vous satisfait ?",
reponse="Oui", score=8.5)
sondage_partiel = DonneesSondage(id=2, question="Recommanderiez-vous notre
service ?")
print(sondage_complet)
print(sondage_partiel)
```

MODELES IMBRIQUES ET LISTES

class Adresse(BaseModel):

```
rue: str
ville: str
code_postal: str

class HistoriqueAchat(BaseModel):
date: str
montant: float
produits: List[str]

class ClientComplet(BaseModel):
id: int
nom: str
adresses: List[Adresse]
historique_achats: List[HistoriqueAchat]
preferences: Dict[str, float]

# Exemple d'utilisation
client_data = {
    "id": 3,
    "nom": "Charlie Brown",
    "adresses": [
        {"rue": "123 Rue Principale", "ville": "Paris",
"code_postal": "75001"},
        {"rue": "456 Avenue Secondaire", "ville": "Lyon",
"code_postal": "69001"}
    ],
    "historique_achats": [
        {"date": "2023-03-15", "montant": 150.75, "produits":
["Livre", "DVD"]},
        {"date": "2023-04-01", "montant": 89.99, "produits": ["T-
shirt", "Chaussettes"]}
    ],
    "preferences": {"Électronique": 0.8, "Livres": 0.6,
"Vêtements": 0.4}
}
client_complet = ClientComplet(**client_data)
print(client_complet)
```

Intégration de Pydantic avec Pandas

```
import pandas as pd
from pydantic import BaseModel, Field, ValidationError, field_validator
from typing import List
from datetime import date

class Vente(BaseModel):
    date: date
    produit_id: int
    quantite: int = Field(gt=0)
    prix_unitaire: float = Field(gt=0)

    @field_validator('prix_unitaire')
    @classmethod
    def arrondir_prix(cls, v: float) -> float:
        return round(v, 2)

def valider_donnees_ventes(donnees_brutes: List[dict]) -> List[Vente]:
    ventes_validees = []
    erreurs = []

    for idx, donnee in enumerate(donnees_brutes):
        try:
            vente = Vente.model_validate(donnee)
            ventes_validees.append(vente)
        except ValidationError as e:
            erreurs.append(f"Erreur à la ligne {idx+1}: {e}")

    if erreurs:
        print("Erreurs de validation détectées:")
        for erreur in erreurs:
            print(erreur)

    return ventes_validees

# Exemple de données brutes
donnees_brutes = [
    {"date": "2023-08-01", "produit_id": 1, "quantite": 5, "prix_unitaire":
10.99},
    {"date": "2023-08-02", "produit_id": 2, "quantite": -1, "prix_unitaire":
15.50}, # Erreur : quantité négative
    {"date": "2023-08-03", "produit_id": 3, "quantite": 3, "prix_unitaire":
0}, # Erreur : prix nul
    {"date": "2023-08-04", "produit_id": 4, "quantite": 2, "prix_unitaire":
20.005},
```

```
]
# Validation des données
ventes_validees = valider_donnees_ventes(donnees_brutes)
# Création du DataFrame avec les données validées
df_ventes = pd.DataFrame([v.model_dump() for v in ventes_validees])
print(df_ventes)
```

- Modèle Vente :**
 - Définit la structure attendue pour chaque vente.
 - Utilise **Field** pour définir des contraintes sur **quantité** et **prix_unitaire**.
 - Utilise un **field_validator** pour arrondir le prix à deux décimales.
- Fonction valider_donnees_ventes :**
 - Prend une liste de dictionnaires (données brutes) en entrée.
 - Utilise **Vente.model_validate()** pour créer et valider chaque instance **Vente**.
 - Collecte les erreurs de validation avec les numéros de ligne correspondants.
 - Retourne une liste d'objets **Vente** validés.
- Gestion des erreurs :**
 - Les erreurs sont collectées et affichées, permettant une correction ciblée des données source.
 - Le processus continue même en cas d'erreurs, permettant de valider l'ensemble des données.
- Intégration avec Pandas :**
 - Après validation, les données sont converties en DataFrame Pandas.
 - Utilisation de **[v.model_dump() for v in ventes_validees]** pour convertir les objets Pydantic en dictionnaires.
- Avantages de cette approche :**
 - **Validation avant analyse :** Assure que seules les données valides entrent dans le DataFrame.
 - **Nettoyage automatique :** Par exemple, l'arrondi des prix.
 - **Détection précise des erreurs :** Identifie exactement quelles lignes et quels champs posent problème.
 - **Typage fort :** Garantit que les types de données dans le DataFrame sont corrects (par exemple, date est bien une date).
- Utilisation dans un workflow d'analyse :**
 - Cette approche peut être utilisée comme une étape de prétraitement dans un pipeline d'analyse de données.
 - Elle permet de détecter et de corriger les problèmes de données avant qu'ils n'impactent l'analyse.
 - Facilite la documentation et la reproductibilité de vos analyses en définissant clairement la structure attendue des données.
- Extensibilité :**
 - Le modèle Vente peut être facilement étendu pour inclure des validations plus complexes ou des champs supplémentaires.
 - Par exemple, vous pourriez ajouter un validateur pour s'assurer que la date de vente n'est pas dans le futur.

Exemple d'extension du modèle :

```
from datetime import date, datetime
class VenteEtendue(Vente):
    categorie_produit: str

    @field_validator('date')
    @classmethod
    def date_non_future(cls, v: date) -> date:
        if v > date.today():
            raise ValueError("La date de vente ne peut pas être dans le futur")
        return v

    @field_validator('categorie_produit')
    @classmethod
    def categorie_valide(cls, v: str) -> str:
        categories_valides = ['Électronique', 'Vêtements', 'Alimentation',
                              'Maison']
        if v not in categories_valides:
            raise ValueError(f"Catégorie invalide. Doit être l'une de : {categories_valides}")
        return v
```

Cas pratique : Analyse des données de ventes

Voir collab https://colab.research.google.com/drive/1lluswFzXJDAtekk2cyvVm2E_onXm3lcn?usp=sharing

```
from pydantic import BaseModel, field_validator, Field, model_validator
from typing import List
from datetime import date
import pandas as pd
import matplotlib.pyplot as plt

class Produit(BaseModel):
    id: str = Field(..., pattern=r'^P\d{4}$')
    nom: str
    categorie: str
    prix: float = Field(..., gt=0)

class Client(BaseModel):
    id: str = Field(..., pattern=r'^C\d{6}$')
    nom: str
    email: str
    date_inscription: date

class Vente(BaseModel):
    id: str = Field(..., pattern=r'^V\d{8}$')
    date: date
    produit: Produit
    client: Client
    quantite: int = Field(..., gt=0)
    montant_total: float
    @model_validator(mode='after')
    def verifier_montant_total(self) -> 'Vente':
        montant_calculé = round(self.produit.prix * self.quantite, 2)
        if abs(self.montant_total - montant_calculé) > 0.01:
            raise ValueError(f"Le montant total {self.montant_total} ne correspond pas au calcul {montant_calculé}")
        return self

    @field_validator('date')
    @classmethod
    def date_non_future(cls, v: date) -> date:
        if v > date.today():
            raise ValueError("La date de vente ne peut pas être dans le futur")
        return v

def valider_donnees_ventes(donnees_brutes: List[dict]) -> List[Vente]:
    ventes_validees = []
    erreurs = []

    for idx, donnee in enumerate(donnees_brutes):
        try:
            vente = Vente.model_validate(donnee)
            ventes_validees.append(vente)
        except ValueError as e:
            erreurs.append(f"Erreur à la ligne {idx+1}: {str(e)}")

    if erreurs:
        print("Erreurs de validation détectées:")
        for erreur in erreurs:
            print(erreur)

    return ventes_validees

# Données de test
donnees_brutes = [
    {
        "id": "V00000001",
        "date": "2023-08-01",
        "produit": {"id": "P0001", "nom": "Smartphone", "categorie": "Électronique", "prix": 599.99},
        "client": {"id": "C000001", "nom": "Alice Dupont", "email": "alice@example.com", "date_inscription": "2023-01-15"},
        "quantite": 1,
        "montant_total": 599.99
    },
    {
        "id": "V00000002",
        "date": "2023-08-02",
        "produit": {"id": "P0002", "nom": "Livre", "categorie": "Librairie", "prix": 19.99},
        "client": {"id": "C000002", "nom": "Bob Martin", "email": "bob@example.com", "date_inscription": "2023-02-20"},
        "quantite": 3,
        "montant_total": 59.97
    },
]
```

```

        {
            "id": "V00000003",
            "date": "2023-08-03",
            "produit": {"id": "P0003", "nom": "Cafetière", "categorie":
"Électroménager", "prix": 89.99},
            "client": {"id": "C000003", "nom": "Claire Dubois", "email":
"claire@example.com", "date_inscription": "2023-03-10"},
            "quantite": 1,
            "montant_total": 89.99
        },
        {
            "id": "V00000004",
            "date": "2023-08-04",
            "produit": {"id": "P0004", "nom": "T-shirt", "categorie": "Vêtements",
"prix": 29.99},
            "client": {"id": "C000004", "nom": "David Lefebvre", "email":
"david@example.com", "date_inscription": "2023-04-05"},
            "quantite": 2,
            "montant_total": 59.98
        },
        {
            "id": "V00000005",
            "date": "2023-08-05",
            "produit": {"id": "P0001", "nom": "Smartphone", "categorie":
"Électronique", "prix": 599.99},
            "client": {"id": "C000002", "nom": "Bob Martin", "email":
"bob@example.com", "date_inscription": "2023-02-20"},
            "quantite": 1,
            "montant_total": 599.99
        },
        {
            "id": "V00000006",
            "date": "2023-08-06",
            "produit": {"id": "P0005", "nom": "Tablette", "categorie":
"Électronique", "prix": 349.99},
            "client": {"id": "C000005", "nom": "Emma Tremblay", "email":
"emma@example.com", "date_inscription": "2023-05-12"},
            "quantite": 1,
            "montant_total": 349.99
        },
        {
            "id": "V00000007",
            "date": "2023-08-07",
            "produit": {"id": "P0006", "nom": "Casque audio", "categorie":
"Électronique", "prix": 129.99},
            "client": {"id": "C000001", "nom": "Alice Dupont", "email":
"alice@example.com", "date_inscription": "2023-01-15"},
            "quantite": 1,
            "montant_total": 129.99
        }
    ]
}

# Validation des données
ventes_validees = valider_donnees_ventes(donnees_brutes)
# Création du DataFrame
df_ventes = pd.DataFrame([
    {
        "id_vente": v.id,
        "date": v.date,
        "produit_id": v.produit.id,
        "produit_nom": v.produit.nom,
        "categorie": v.produit.categorie,
        "client_id": v.client.id,
        "client_nom": v.client.nom,
        "quantite": v.quantite,
        "montant_total": v.montant_total
    } for v in ventes_validees
])

# Analyse des données
print(df_ventes.head())
# Ventes totales par catégorie
ventes_par_categorie = df_ventes.groupby('categorie')
['montant_total'].sum().sort_values(ascending=False)
print("\nVentes totales par catégorie:")
print(ventes_par_categorie)
# Visualisation des ventes par catégorie
plt.figure(figsize=(10, 6))
ventes_par_categorie.plot(kind='bar')
plt.title("Ventes totales par catégorie")
plt.xlabel('Catégorie')
plt.ylabel('Montant total des ventes')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Analyse des clients les plus actifs
clients_actifs = df_ventes.groupby('client_id').agg({
    'client_nom': 'first',
    'montant_total': 'sum',
    'id_vente': 'count'
}).sort_values('montant_total', ascending=False)
clients_actifs.columns = ['Nom', 'Montant total des achats', 'Nombre d\'achats']
print("\nTop 5 des clients les plus actifs:")
print(clients_actifs.head())

# Vous pouvez ajouter d'autres analyses ici...
```

Explications détaillées :

- Modèles Pydantic :**
 - Produit, Client, et Vente sont définis avec des validations spécifiques.
 - Utilisation de Field avec le paramètre pattern pour des validations regex sur les IDs.
 - Utilisation de model_validator et field_validator pour les validations personnalisées.
- Validateurs personnalisés :**
 - verifier_montant_total est maintenant un model_validator avec mode='after', ce qui permet d'accéder à tous les champs de l'instance.
 - date_non_future reste un field_validator, mais avec la syntaxe mise à jour de Pydantic v2.
- Fonction de validation :**
 - valider_donnees_ventes utilise maintenant Vente.model_validate() au lieu de Vente(**).
 - La gestion des erreurs reste inchangée.
- Création du DataFrame et Analyses :**
 - Cette partie reste largement inchangée, car elle dépend principalement de Pandas et Matplotlib.
- Avantages de cette approche avec Pydantic :**
 - Performance améliorée :** Pydantic offre de meilleures performances, cruciales pour de grands ensembles de données.
 - Validations plus robustes :** L'utilisation de Field avec pattern et les nouveaux validateurs

offrent une validation plus précise et flexible.

- **Meilleure lisibilité du code** : Les annotations de type et la structure des validateurs sont plus claires dans Pydantic.
- **Intégrité des données** : Seules les données valides sont incluses dans l'analyse.
- **Flexibilité** : Le modèle Pydantic peut être facilement étendu pour inclure de nouvelles validations ou champs.
- **Traçabilité** : Les erreurs de validation sont clairement identifiées et peuvent être corrigées à la source.

Création d'un Chatbot

mardi 10 juin 2025 17:56

<https://aistudio.google.com/app/apikey?hl=fr&Clé>
Clé API : AlzaSyDlj67WE1QJ8uvlJSp6y3zHwoi5gzx4hxQ

Voir le colab sur la quête creation d'un chatbot
https://colab.research.google.com/drive/1G2v1_YkhC5WUxFhU2aknNQ4YLZlWH8SF?usp=sharing

Voir Colab : https://colab.research.google.com/drive/1G2v1_YkhC5WUxFhU2aknNQ4YLZlWH8SF?usp=sharing

Nous avons vu comment utiliser les LLMs à travers les Chatbots en ligne comme ChatGPT ou Gemini. Cependant, il est possible d'aller beaucoup plus loin avec les LLMs en ne leur demandant pas de générer ou de corriger du code, mais en travaillant directement au sein du programme, via une API. C'est de cette façon que l'on peut créer ses propres Chatbots !



Des prompts pour améliorer la réponse

L'une des caractéristiques des LLMs, que l'on pourrait considérer comme un défaut, est leur propension à produire du texte à la chaîne. Pour citer *Les Tontons flingueurs* : « C'est curieux, chez les marins, ce besoin de faire des phrases. »

L'autre limite, lorsqu'on laisse le LLM répondre sans contexte, est qu'il va produire une donnée « brute », générique. Or, il est souvent nécessaire de préciser le type de réponse que l'on attend. C'est ce que l'on appelle le **contexte**, que l'on peut préciser dans un message « système ».

Reprenons l'exemple précédent : à la question simple de savoir qui a été le dernier roi de France, Gemini a répondu en indiquant son nom et les dates de son règne. Mais supposons que l'on veuille disposer de réponses plus complètes : il faut indiquer cela dans le contexte du prompt.

Pour cela rien de plus simple, il suffit de préciser ces informations dans le contexte que l'on fournira au modèle en même temps que la question :

```
1 from google.genai import types
2
3 sys_instruct="Tu es un spécialiste de l'Histoire de France. Tu donnes des réponses précises et complètes, en les replaçant dans le contexte politique
4
5 prompt = "Quel est le dernier roi de France ?"
6
7 response = client.models.generate_content(
8     model="gemini-2.0-flash",
9     config=types.GenerateContentConfig(
10         system_instruction=sys_instruct),
11     contents=[prompt]
12 )
13
14 print(response.text)
```

De cette façon, on obtiendra une réponse plus pertinente, par exemple :

Le dernier roi de France est Louis-Philippe Ier, qui a régné de 1830 à 1848. Son règne est connu sous le nom de Monarchie de Juillet.

****Contexte politique : ****

Le dernier roi de France est Louis-Philippe Ier, qui a régné de 1830 à 1848. Son règne est connu sous le nom de Monarchie de Juillet.

Contexte politique :

- * **La Restauration (1814-1830):** Après la Révolution française et l'Empire napoléonien, la monarchie est restaurée avec les frères de Louis XVI : Louis XVIII puis Charles X. Cependant, cette Restauration est fragile, car elle oscille entre un retour à l'Ancien Régime et la nécessité de prendre en compte les acquis de la Révolution. Charles X, en particulier, mène une politique de plus en plus réactionnaire.
- * **Les Trois Glorieuses (27-29 juillet 1830):** La politique autoritaire de Charles X conduit à une insurrection populaire à Paris, connue sous le nom des "Trois Glorieuses". Charles X est contraint d'abdiquer.
- * **La Monarchie de Juillet (1830-1848):** Pour éviter une nouvelle république, les élites libérales, menées par Adolphe Thiers et François Guizot, proposent Louis-Philippe d'Orléans, un cousin de Charles X, comme roi. Louis-Philippe accepte et devient "roi des Français" et non plus "roi de France", marquant ainsi une rupture symbolique avec l'Ancien Régime.

Louis-Philippe Ier :

- * Louis-Philippe incarne une monarchie constitutionnelle plus libérale que celle de la Restauration. Il adopte le drapeau tricolore, symbole de la Révolution, et élargit légèrement le droit de vote (suffrage censitaire).
- * Cependant, son régime reste conservateur et favorise les intérêts de la bourgeoisie. Le peuple et les républicains sont déçus par le manque de réformes sociales et politiques.
- * **La Révolution de 1848:** Le mécontentement populaire et les crises économiques conduisent à la Révolution de 1848. Louis-Philippe est renversé et la Deuxième République est proclamée.

En résumé : Louis-Philippe Ier est bien le dernier roi à avoir régné en France. Son règne marque une période de transition entre la Restauration et la Deuxième République, caractérisée par une monarchie constitutionnelle libérale mais socialement conservatrice.

Voilà qui est sans doute plus satisfaisant ! Nous aurions pu préciser la longueur du texte à fournir dans le contexte (« donne une réponse courte » ou « donne une réponse détaillée », voire préciser un format (« donne une réponse sous forme de liste à points », par exemple).

Vous aurez également remarqué la présence de symboles « * » qui sont destinés à la mise en forme au format Markdown, qui n'est pas pris en charge par la fonction `print`.

Le texte encadré par « ** » et « * » est affiché en **gras**, et le texte encadré par « * » et « * » est affiché en *italique*.

Si vous souhaitez bénéficier de cette mise en forme, vous devez utiliser une bibliothèque supplémentaire :

```
1 from IPython.display import display, Markdown
2
3 display(Markdown("**Gras** : *italique*"))
```

Création d'un chatbot

https://colab.research.google.com/drive/1G2v1_YkhC5WUxPhU2aknNQ4YLZlWH8SF?usp=sharing

Pour le moment, nous avons vu comment interagir avec Gemini depuis un programme Python, mais ce n'est pas encore satisfaisant. En effet, le contenu généré ne s'appuie que sur une question et un contexte, mais il n'a aucune mémoire de la conversation globale, ce que l'on appelle l'**historique de conversation**.

Pour cela, il va falloir mémoriser tous les messages envoyés et reçus dans une liste de messages qui sera passée à la fonction. Par chance, le modèle de chat de Google prend en charge automatiquement la gestion de la mémoire !

```
1 # Création du modèle de Chatbot
2 client = genai.Client(api_key=GOOGLE_API_KEY)
3
4 system_prompt = """
5 Tu es un spécialiste de l'intelligence artificielle. Tu donnes des réponses précises pour un public connaissant les bases de l'IA et Python.
6 Si la réponse ne concerne pas l'intelligence artificielle, la programmation, les statistiques et les mathématiques, dis que tu ne réponds qu'à des q
7 """
8
9 history = [
10     {"role": "user", "parts": [{"text": system_prompt}]},
11     {"role": "model", "parts": [{"text": "Compris, je suis prêt à répondre à vos questions sur l'IA, la programmation, les statistiques et les mathém
12 ]
13
14 # Création du chatbot
15 chat = client.chats.create(model="gemini-2.0-flash", history=history)
16
17 # Envoi du prompt système
18 chat.send_message(system_prompt)
19
20 print("Bienvenue dans ce Chat qui vous aidera à mieux comprendre l'histoire de France.")
21 print("Pour quitter le Chat, tapez simplement 'fin'")
22 while True:
23     question = input(">")
24     if question.lower().strip() == 'fin':
25         break
26     response = chat.send_message_stream(question)
27     for chunk in response:
28         print(chunk.text, end="")
```

On obtient bien un véritable Chatbot, comme le montre la session suivante :

Bienvenue dans ce Chat qui vous aidera à mieux comprendre l'histoire de France.

Pour quitter le Chat, tapez simplement 'fin'

>Qui a gagné la bataille de Waterloo ?

La bataille de Waterloo a été gagnée par les forces alliées dirigées par le duc de Wellington (Britanniques, Hollandais, Hanovriens, etc.) et par les Prussiens menés par Gebhard Leberecht von Blücher. Ils ont vaincu l'armée française de Napoléon Bonaparte.

>En quelle année ?.

La bataille de Waterloo a eu lieu en 1815.

>Quelle était ma première question ?

Votre première question était : "Qui a gagné la bataille de Waterloo ?"

>fin

Voilà, chose promise, chose due : vous êtes à présent capable de programmer un Chatbot en Python !

Remarque : Si vous affichez la réponse au fur et à mesure à l'aide de la méthode `send_message_stream`, vous ne pouvez pas utiliser le Markdown dans un Notebook (il existe cependant des fonctions nécessaires dans un framework comme Streamlit).

Utilisation d'un modèle Multimodal (prise en charge Texte et image)

https://colab.research.google.com/drive/1G2v1_YkhCSWUxHfU2aknNQ4YLZIWH8SF?usp=sharing

Analyse d'une image avec Gemini

Considérons l'image suivante :



Qu'en pense Gemini 2.0 ? Pour le savoir, il suffit de lui poser la question...


```

1 import requests
2 from PIL import Image
3 from io import BytesIO
4
5 # URL de l'image
6 url = 'https://i0.wp.com/www.ericcanto.com/wp-content/uploads/2021/06/nirvana-1.jpg'
7 # Télécharger l'image
8 response = requests.get(url)
9 response.raise_for_status() # Vérifie que la requête s'est bien passée
10
11 # Charger l'image avec PIL
12 image = Image.open(BytesIO(response.content))
13
14 client = genai.Client(api_key=GOOGLE_API_KEY)
15
16 response = client.models.generate_content_stream(
17     model="gemini-2.0-flash",
18     contents=[image, "Peux-tu me décrire cette image ?"])
19
20 for chunk in response:
21     print(chunk.text, end="")

```

Il devrait vous fournir votre réponse de ce type :

voici une description de l'image :

L'image est une photo en noir et blanc de la formation originale du groupe de rock américain Nirvana. Le groupe est composé de trois membres, tous vêtus de costumes et de cravates. Ils sont placés côte à côte. Le nom du groupe, "Nirvana", est affiché en grosses lettres noires dans une bordure blanche en haut de l'image.

De gauche à droite, les membres du groupe sont :

1. Dave Grohl : Il a les cheveux longs et foncés et un regard sérieux.
2. Kurt Cobain : Il est au centre et a les cheveux blonds mi-longs et un regard intense.
3. Krist Novoselic : Il est à droite et a les cheveux foncés légèrement plus courts.

Structurer le format de sortie

https://colab.research.google.com/drive/1G2v1_YkhC5WUxFhU2aknNQ4YLZlWH8SF?usp=sharing

5. Structurer le format de sortie

Comme vous l'avez vu, les LLMs génèrent du texte en sortie, mais leur format peut varier : des phrases, des listes à puces ou des listes numérotées, du markdown... Mais comment faire si l'on souhaite obtenir des réponses sous une forme précise, en utilisant le LLM non pas comme un chatbot, mais comme une fonction interne à son programme ?

Il faut pour cela imposer un format de sortie, ce qui s'effectue à l'aide d'un prompt, système ou utilisateur. Par exemple :

```

1 from google import genai
2 from pydantic import BaseModel
3
4 class LLMs(BaseModel):
5     nom: str
6     propriétaire: str
7     capacités: list[str]
8
9 client = genai.Client(api_key=GOOGLE_API_KEY)
10 response = client.models.generate_content(
11     model="gemini-2.0-flash",
12     contents="Donne la liste de plusieurs LLMs. Ajoute bien les capacités propres à chacun, comme la classification, l'analyse de texte, de vidéo, etc.",
13     config={
14         'response_mime_type': 'application/json',
15         'response_schema': list[LLMs],
16     },
17 )
18
19 llms = eval(response.text)
20 print(llms[0])

```

Ce qui retourne un résultat comme celui-ci :

```
{'nom': 'GPT-4', 'propriétaire': 'OpenAI', 'capacités': ['Génération de texte', 'Réponse aux questions', 'Traduction', 'Classification', 'Analyse de texte', 'Génération de code', 'Compréhension de documents', 'Chatbot', 'Création de contenu']}
```

Web Scrapping

mar 10 juin 2025 08:46

```
import requests
from bs4 import BeautifulSoup
from time import sleep
```

1. récupérer l'url "
2. récupérer le code html [response = requests.get(url)]
3. Parser le code html avec un parseur pour sélection des éléments dans le code html (BeautifulSoup, scrapy, selenium)

1. le contenu de la page html
url = "https://en.wikipedia.org/wiki/Web_scraping"
html = requests.get(url)
html.status_code

2. pour afficher le contenu de la page
html.text

Le plus fréquent est la protection contre des navigateurs inconnus. Lorsque nous lançons la requête, nous nous identifions comme python, il arrive d'être bloqué. Il suffit alors de s'identifier comme un vrai navigateur, par exemple comme ceci :

```
1 navigator = 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_3)'
2 html = requests.get(url, headers={'User-Agent': navigator})
```

3. Pour retrouver la balise recherchée, il faut **Parser** la page (c'est-à-dire du nettoyage et de la mise en ordre de ce code HTML)
soup = BeautifulSoup(html.text, 'html.parser')
soup

4. Affichage de la class : exemple ... ("class": "mw-headline")
titles_wiki = soup.find_all('span', {"class": "mw-headline"})

len(titles_wiki) pour afficher le nombre d'éléments
titles_wiki[5] pour afficher l'éléments 5
titles_wiki[5].get_text() pour afficher l'éléments 5 sans les balises

Scraper plusieurs page :

```
import requests
from bs4 import BeautifulSoup

# Fonction pour scraper une page
def scraper_page(url):
    ...

Ceci est une fonction pour scraper une page web.
Prend en entrée une chaîne de caractère (str) et retourne une chaîne de caractère
au format "soup".
...
    response = requests.get(url)
    # On vérifie que le site réponde avant d'aller plus loin
    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')
        titles = soup.find_all('h2')
    # On crée une liste vide pour stocker
    data = []
    # On récupère le texte
    for title in titles:
        print(title.text)
        data.append(title.text)

# On programme la sortie de la fonction avec le return : notre texte.
    return data # Sous un format liste
# Autrement, on affiche le message d'erreur.
    else:
        print(f"Erreur lors de la récupération de la page {url}.")
# On peut aussi traiter le message d'erreur avec un "try" "except" si on connaît
le message d'erreur.
```

Pour scraper plusieurs pages, nous allons tout simplement créer une fonction qui va itérer à partir de notre fonction de scraping pour un seul site.

```
# Fonction principale pour scraper plusieurs pages
def scraper_pages(urls):
    for url in urls:
        scraper_page(url)

# Liste d'URLs à scraper
urls_to_scrape = [
    "https://example.com/page1",
    "https://example.com/page2",
    "https://example.com/page3"
]
# Appeler la fonction pour scraper les pages
scraper_pages(urls_to_scrape)
```

Quête WebScraping 1 Maelle : https://colab.research.google.com/drive/2HaqHfAUHv10m-43E6IVMTEQwH3UBH867upg=sharing#scrollTo=57cVp_WCOK

Exemple code sur youtube : (récupérer coordonnées avocat sur 104 pages web) voir code juste coté -->
[\[Tuto Français\] Comment Scraper un Site Web \(+ Sauvegarder les Données\) avec Python et BeautifulSoup](#)



Code de base pour récupérer le code html

```
url = "https://www.randstad.fr/"

headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/113.0.0.0 Safari/537.36"}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    print("🟢 Requête réussie.")
    soup = BeautifulSoup(response.text, 'html.parser')
else:
    print(f"❌ Échec de la requête. Code HTTP : {response.status_code}")
```

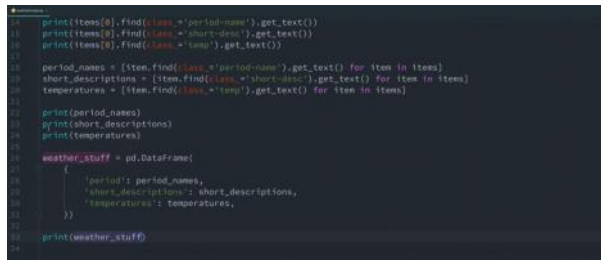
Code vidéo youtube :

```
import requests
from bs4 import BeautifulSoup
import re
# obtenir les urls de pages
def get_all_pages():
    urls = []
    page_number = 1
    for i in range(104): # car 104 pages sur le site
        i = f"https://www.barreaudenice.com/annuaire/avocats/?fwp_paged={page_number}"
        page_number += 1
        urls.append(i)
    return urls

# obtenir les coordonnées des avocats sur 1 page
def parse_attorney(url):
    r = requests.get(url)
    soup = BeautifulSoup(r.content, "html.parser")
    avocats = soup.find_all("div", class_="callout secondary annuaire")
    for avocat in avocats :
        try:
            nom = avocat.find("h3").text.strip() # strip permet de supprimer les espaces gauche et droite
        except AttributeError as e :
            nom = ""
        try:
            adresse = avocat.find("span", class_="adresse").text.strip()
        except AttributeError as e :
            adresse = ""
        try:
            adresse_final = re.sub(r"^(s+", " ", adresse)
        except AttributeError as e :
            adresse_final = ""
        try:
            telephone = avocat.find("span", class_="telephone").text.strip()
        except AttributeError as e :
            telephone = ""
        try:
            email = avocat.find("span", class_="email").a.text.strip()
        except AttributeError as e :
            email = ""
    # obtenir les coordonnées des avocats sur toutes les pages
    def parse_all_attorneys():
        pages = get_all_pages()
        for page in pages:
            parse_attorney()
            print(f"scraping de la page : {page}")

    parse_all_attorneys()
```

Vidéo explicative : [20 - web scraping with python using beautiful soup & requests \(Python tutorial for beginners 2019\)](#)



Le Crawling

mercredi 11 juin 2025 09:34

Le crawling s'effectue donc en deux étapes principales :

1. Tout d'abord il faut faire du scraping sur une première page, et stocker les liens (URL) qui nous intéressent ;
2. Ensuite, il faudra faire du scraping sur tous ces liens, et stocker les résultats.

Démo Code de la Quête

https://colab.research.google.com/drive/1oMfr2M_OoLH3y8uBWx_jp-MmUUw0VrFZ?usp=sharing

Créer une table DATE : très très utile

```
Calendrier = ADDCOLUMNS( CALENDAR(AUTO()),
    "Année", YEAR([Date]),
    "Mois", MONTH([Date]),
    "Jour", DAY([Date]),
    "Nom du Mois", FORMAT([Date], "MMM"),
    "Jour de la Semaine", WEEKDAY([Date]),
    "Nom du Jour de la Semaine", FORMAT([Date], "DDDD"),
    "Numéro de Semaine", WEEKNUM([Date])
)
```

Bonnes pratiques de modélisation :

- 1. Utiliser le Schéma en Étoile avec des relations One (table de dimension) - Many (table de faits).
- 2. Privilégier les Relations Unidirectionnelles de la table de dimension vers la table de faits.
- 3. Inclure uniquement les Données Nécessaires pour éviter la surcharge d'informations.

Fonctions en vrac :

Fonctions sur les colonnes (fonctions Scalaires)

La fonction DISTINCTCOUNT permet de compter le nombre distinct de valeurs dans une colonne
nombre_client = DISTINCTCOUNT(Customer[Customer Name])

fonction COUNTROWS permet de compter le nombre de lignes dans une table
nombre_client = COUNTROWS(Customer)

fonction SUM permet d'additionner les valeurs d'une colonne (MIN, MAX, AVERAGE)
total de quantité commandé total_quantity = SUM(Sales[Units])

La fonction d'arrondi la plus utilisée en DAX est ROUND. (floor, ceiling, roundup)
//calculer le prix moyen de vente
//l'arrondir à 2 chiffres après la virgule
moyenne_vente = ROUND(AVERAGE(Sales[Cost]), 2)

Fonctions sur les Lignes (fonctions Itératives)

Calculer le total de chiffre d'affaires (Minx, Maxx,Averagex)
Total_vente = SUMX(Sales, Sales[Costs] * Sales[Units])

1. ISBLANK
Elle vérifie si une valeur est vide et renvoie TRUE si c'est le cas, sinon FALSE.
// vérifier si total_quantity est vide ou non
check_total_quantity = ISBLANK(total_quantity)
Cette fonction est couramment utilisée pour éviter l'affichage de valeurs blank en les remplaçant par un 0, ce qui rend le visuel plus propre.

2. ISNUMBER
// vérifie si total quantity renvoie un nombre
Check_is_number = ISNUMBER([total_quantity])

Fonctions de Conversion

Elles permettent de changer le type d'une variable ou de formater son affichage. La fonction la plus utilisée est FORMAT (convert, trunc, text)

convertir le type de la mesure total vente en type currency
total_vente_currency = FORMAT([total_vente], "Currency")

Fonctions Logiques

1. IF
En utilisant la mesure total_quantity, nous souhaitons catégoriser les ventes
- "High sales quantity" si total_quantity > 100,
- sinon "Low sales quantity".
classe_vente = IF(SUM([total_quantity]) > 100, "High sales", "Low sales")

2. SWITCH
Si nous voulons maintenant définir trois classes, l'utilisation de IF devient plus complexe.
C'est ici qu'intervient SWITCH, une alternative plus lisible dans ce cas :
Classe_vente_2 = SWITCH(TRUE(), SUM(Sales[Costs]) > 100, "High", SUM(Sales[Costs]) >= 50 && SUM(Sales[Costs]) <= 100, "Medium", "Low")

Quête DAX

- dax advanced part 1 2
- dax advanced part 1 3
- dax advanced part 1 4
- dax advanced part 1 5
- dax advanced part 1 6
- dax advanced part 1 7



Quête DAX avancée



Formule de la quête DAX avancées:

```
total_vente = sum(Sales[Sales Amount])

total_vente_cumule = TOTALYTD([total_vente_france], 'Date'[Date].[date])

total_vente_france = CALCULATE([total_vente], Customer[Country-Region] = "France")

total_vente_france_accessories = CALCULATE([total_vente], Customer[Country-Region] = "France", 'Product'[Category] = "Accessories")

total_vente_france_bikes = CALCULATE([total_vente], Customer[Country-Region] = "France", 'Product'[Category] = "Bikes")

total_vente_france_clothing = CALCULATE([total_vente], Customer[Country-Region] = "France", 'Product'[Category] = "Clothing")

total_vente_lastyear = calculate([total_vente_france], SAMEPERIODLASTYEAR('Date'[Date].[Date]))

percent_sales_country = var total_vente_fixe = CALCULATE([total_vente], All(Customer)) var total_vente_variable = [total_vente] var percent_ = (total_vente_variable * 100) / total_vente_fixe return percent_

percent_total_vente_same_month_last_year = var total_vente_last_year_month = CALCULATE([total_vente_france], SAMEPERIODLASTYEAR('Date'[Date].[Date])) var diff = ('Mesures'[total_vente_france]-total_vente_last_year_month) var ratio = divide(diff, total_vente_last_year_month,0)*100 return ratio

percent_total_vente_two_last_month = var total_vente_two_last_month = CALCULATE([total_vente_france], DATEADD('Date'[Date].[date],2, MONTH)) var diff = ('Mesures'[total_vente_france]-total_vente_two_last_month) var ratio = divide(diff, total_vente_two_last_month,0)*100 return ratio

percent_vente_year_last_month = var total_vente_month = CALCULATE([total_vente_france], PREVIOUSMONTH('Date'[Date].[date])) var diff = ('Mesures'[total_vente_france]-total_vente_month) var ratio = divide(diff, total_vente_month,0)*100 return ratio

percent_vente_year_to_year = var diff = [total_vente_france] - [total_vente_lastyear] var ratio = DIVIDE(diff, [total_vente_lastyear]) * 100 RETURN ratio
```

1.3 DAX - La fonction Calculate

La fonction **CALCULATE** se compose de deux parties essentielles : l'expression à évaluer et les filtres à appliquer. Voici une structure de base de cette fonction

```
1 CALCULATE (
2     expression, //votre calcul tel que total vente, moyenne cout
3     [filter1], //conditions qui modifient le contexte du calcul
4     [filter2],
5     ...
6 )
7
```

Maintenant, explorons quelques exemples d'utilisation de la fonction **CALCULATE** dans différents contextes :

- Total des chiffre d'affaire pour le magasin du canada :

```
1 CALCULATE (
2     SUMX (
3         Sales,
4         Sales[Costs] * Sales[Units]
5     ), // mesure : calcul de total de chiffre d'affaire
6     Stores[Country] = "Canada" // contexte modifié pour la mesure
7 )
8
```

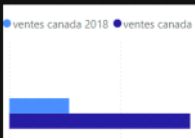
Nous pouvons comparer les ventes du magasin canadien par rapport aux ventes de l'entreprise



- Avec le **CALCULATE** , nous pouvons appliquer plusieurs filtres. Ici nous souhaitons calculer les ventes du magasin canadien pour l'année 2018.

```
1 CALCULATE (
2     SUMX (
3         Sales,
4         Sales[Costs] * Sales[Units]
5     ),
6     Stores[Country] = "Canada" // premier filtre
7     YEAR('Date'[Date]) = 2018 // deuxième filtre. YEAR étant une fonction pour extraire l'année
8 )
```

Ainsi nous pouvons comparer les ventes du magasin Canadien de 2018 par rapport à toute les ventes



Fonctions de modification de contexte

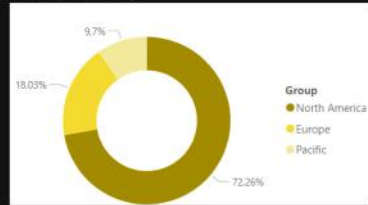
Ces fonctions sont utilisées avec **CALCULATE** pour altérer ou réinitialiser le contexte d'évaluation

1. All

Elle élimine tous les filtres existants sur une colonne ou une table.
Par exemple, **ALL('Stores')** supprime tous les filtres appliqués à la table des Stores.

```
1 //pourcentage de vente de chaque magasin
2 Part_vente =
3 CALCULATE(
4     //fonction pour diviser
5     DIVIDE(
6
7         //total vente qui varie en fonction du contexte
8         [Total_vente],
9
10        //total de vente fixe peu importe le filtre sur la table Stores
11        CALCULATE(
12            [Total_vente],
13
14            // ignorer les filtres appliqués à la table des Stores
15            ALL('Stores')
16        )
17    )
18 )
```

Nous pouvons ainsi voir la répartition des ventes entre les différentes régions des magasins



⚠ remarque : dans certains exemples vous trouverez **removefilters** au lieu du **all**, c'est juste un alias, le résultat reste inchangé.

2. keepfilters

KEEPFILTERS permet de conserver les filtres existants tout en ajoutant un nouveau filtre.

```
1 // calculer total vente, en ajoutant un filtre pour ne considérer que les années 2022 et 2023
2 CALCULATE(
3     [Total_vente],
4     //keepfilter permet de conserver les filtres
5     KEEPFILTERS(year(Date[Date]) in [2022, 2023])
6 )
```

⚠ Il existe d'autres fonctions de modification de contexte :

allexcept : permet de supprimer les filtres sur toutes les colonnes, à l'exception d'une colonne particulière.

allexcepted : supprime tous les filtres sauf ceux définis par l'utilisateur via les slicers ou autres éléments interactifs.

Pour ceux qui veulent en savoir plus, vous pouvez regarder cette [video](#)

Fonction time intelligence

Les fonctions de Time Intelligence en DAX permettent de faire des calculs basés sur des périodes de temps. Elles aident à comparer une mesure actuelle avec la même mesure sur une période précédente.

Voici quelques exemples couramment utilisés :

- **Previousmonth** :

```
1 //calculer le total vente pour le mois précédent
2 Ventes_Mois_Precedent =
3 CALCULATE(
4     [Total_vente],
5     PREVIOUSMONTH('Date'[Date])
6 )
```

Nous pouvons ainsi comparer les ventes du mois en cours et du mois précédent pour les années 2018 et 2019

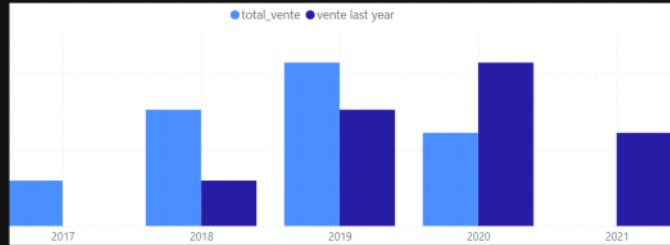


⚠ il existe d'autre fonction similaire mais pour traiter d'autre périodes : **previousday**, **previousyear**, **previousquarter** qui permettent respectivement de faire un calcul sur le **jour**, **année**, **trimestre précédent**

- Sameperiodlastyear :

```
1 // calculer le total vente pour la même période mais de l'année dernière
2 Ventes_Meme_Period_Last_Year =
3 CALCULATE(
4     [Total_vente],
5     SAMEPERIODLASTYEAR('Date'[Date])
6 )
7
```

Nous pouvons donc comparer les ventes de l'année encours et l'année dernière



- Dateadd :

La fonction DATEADD permet de décaler une date par rapport à une période donnée.

```
1 // calculer le total des ventes pour les 3 derniers mois
2 Ventes_Mois_Dernier =
3 CALCULATE(
4     [Total_vente],
5     DATEADD('Date'[Date], -3, MONTH)
6 )
```

Nous pouvons stocker cette information sous format de carte

6,94M
total vente last 3 month

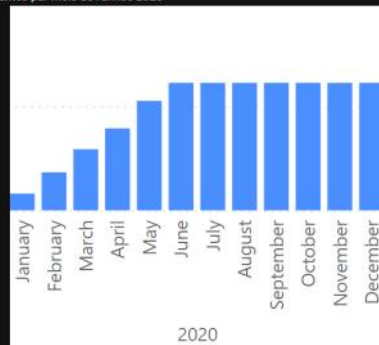
⚠ On peut adapter le paramètre de période en remplaçant "MONTH" par "WEEK", "YEAR", "QUARTER", "DAY"

- Totalmtd, Totalytd, Totalqtd (Total cumulé) :

Les fonctions **TOTALMTD**, **TOTALYTD**, et **TOTALQTD** calculent le total cumulé respectivement par mois, année, et trimestre.

```
1 // total des vente cumulé par mois
2 TotalMTD_Ventes =
3 CALCULATE(
4     [Total_vente],
5     TOTALMTD('Date'[Date])
6 )
```

à titre d'exemple nous pouvons suivre le total cumulé des ventes par mois de l'année 2020



- DATESYTD (Année à ce jour) :

```
1 // Calculer le cumul du total de ventes depuis le début de l'année jusqu'à la date spécifiée
2 Ventes_YTD =
3 CALCULATE(
4     [Total_vente],
5     DATESYTD('Date'[Date])
6 )
```

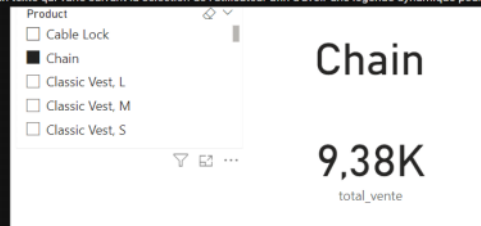
⚠ de meme **DATETMD** et **DATEQTD** permettent respectivement d'effectuer le calcul du début du mois ou du début du trimestre jusqu'à la date spécifiée.

Fonctions DAX pour interagir avec l'utilisateur

L'idée est d'inclure dans nos formules DAX les données fournies par l'utilisateur. Nous utiliserons souvent la fonction `SELECTEDVALUE`, qui permet de récupérer la valeur sélectionnée dans un segment.

```
1
2 // sélectionner le nom de produit choisi en filtre par l'utilisateur
3 ProduitSelectionne =
4     SELECTEDVALUE('Product'[ProductName])
5
```

En utilisant cette mesure nous pouvons afficher un texte qui varie suivant la sélection de l'utilisateur afin d'avoir une légende dynamique pour le total des ventes



Ecriture des formules DAX avec Var et Return

Lors de la création de mesures complexes, il peut être difficile de lire le calcul sur une seule ligne. Pour simplifier la lisibilité du code, nous utilisons `VAR` pour définir des variables intermédiaires et `RETURN` pour renvoyer le résultat final.

```
1 // Calculer le pourcentage des ventes
2 PourcentageVentesTotal =
3
4 // Calcul du total des ventes en ignorant les filtres
5 VAR all_sales = CALCULATE(SUMX(ALL('Sales'),
6     'Sales'[Cost] * 'Sales'[Units]))
7 // Calcul des ventes actuelles (change suivant le contexte)
8 VAR current_sales = SUMX('Sales',
9     'Sales'[Cost] * 'Sales'[Units])
10 // Calcul du pourcentage des ventes par rapport au total
11 VAR ratio = current_sales / all_sales
12
13 // retourner les résultat
14 RETURN ratio
```

Le fameux "Dockerfile"

Un **Dockerfile** est un script de configuration utilisé pour automatiser le processus de création d'une image Docker, qui servira de base à la création de conteneurs. Il contient une série de commandes et d'instructions que Docker utilisera pour assembler l'image.

Si vous souhaitez pousser plus en détail, nous en parlons aussi très bien dans cette quête :

Dans notre Dockerfile, nous trouverons :

- **Base de l'image :**

Commence généralement par une instruction **FROM** qui définit l'image de base (par exemple, un système d'exploitation comme Ubuntu ou une version spécifique de Python).

Exemple :

```
1 FROM python:3.9-slim
2 # Cette ligne indique l'image de base à utiliser pour la construction de l'image Docker.
3 # Ici, l'image de base est python:3.9-slim, qui est une version allégée de l'image officielle de Python 3.9.
4 # Elle contient les outils et bibliothèques essentiels pour exécuter des applications Python,
5 # mais est optimisée pour être plus petite que les images Python standard.
```

- **Configuration de l'environnement :**

Inclut des instructions pour configurer le système, comme **RUN** pour exécuter des commandes Shell (mise à jour de packages, installation de logiciels, etc.).

exemple:

```
1 RUN apt-get update && apt-get install -y git && apt-get install -y build-essential && rm -rf /var/lib/apt/lists/*
2 # Cette commande est exécutée lors de la construction de l'image.
3 # Elle met à jour la liste des paquets disponibles,
4 # installe Git dans le conteneur (ce qui peut être nécessaire pour les dépendances ou scripts qui utilisent Git),
5 # puis supprime les fichiers de liste de paquets pour réduire la taille de l'image.
6 # Les options utilisées sont :
7 # * apt-get update : met à jour la liste des paquets.
8 # * apt-get install -y git : installe Git sans demander de confirmation grâce à l'option -y.
9 # * apt-get install -y build-essential : installe gcc qui est nécessaire au projet
10 # * rm -rf /var/lib/apt/lists/* : supprime les fichiers de liste de paquets pour garder l'image légère.
11
12 RUN pip install --upgrade pip setuptools wheel
13 # Mise à jour de pip, setuptools et wheel.
14 # Setuptools et wheel facilitent la distribution et l'installation des packages Python.
15
16 RUN pip install --no-cache-dir -r requirements.txt
17 # Exécute pip install pour installer les paquets Python listés dans le fichier requirements.txt.
18 # L'option --no-cache-dir empêche pip de garder une copie des fichiers de paquet téléchargés,
19 # ce qui est une autre mesure pour garder l'image Docker légère.
20 # L'option -r indique à pip de lire à partir du fichier de requirements spécifié (requirements.txt dans ce cas).
```

- **Copie de fichiers :**

Utilise **COPY** pour transférer des fichiers du système hôte vers l'image, comme le code source de l'application ou des fichiers de configuration.

exemple:

```
1 COPY requirements.txt requirements.txt
2 # Copie le fichier requirements.txt de votre projet (situé sur votre machine ou dans votre répertoire source)
3 # dans le répertoire /app du conteneur, en conservant le nom du fichier: requirements.txt.
4 COPY . .
5 # Copie tous les autres fichiers et dossiers du répertoire courant (de votre machine ou répertoire source)
6 # dans le répertoire de travail actuel du conteneur Docker (défini par WORKDIR /<mon dossier>).
```

- **Paramètres de lancement :**

Utilise **CMD** ou **ENTRYPOINT** pour définir la commande qui sera exécutée au démarrage du conteneur. Cela spécifie comment l'application doit être lancée.

exemple:

```
1 CMD ["mage", "start"]
2 # Ici, "mage" est l'exécutable configuré dans l'image, que vous souhaitez exécuter avec l'argument "start".
3 # Cette commande n'est exécutée que lorsque le conteneur est lancé, pas au moment de la construction de l'image.
4
```

- **Exposition de ports et autres :**

Peut inclure des instructions comme **EXPOSE** pour indiquer les ports à ouvrir, **ENV** pour définir des variables d'environnement, et **WORKDIR** pour définir le répertoire de travail.

exemple:

```
1 WORKDIR /<mon dossier>
2 # Définit le répertoire de travail dans le conteneur Docker.
3 # Cela signifie que toutes les actions ultérieures,
4 # comme la copie de fichiers ou l'exécution de commandes,
5 # seront effectuées dans le chemin /<mon dossier> à l'intérieur du conteneur.
```

Lorsque vous lancez un build Docker avec ce **Dockerfile**, chaque ligne d'instruction est exécutée dans un ordre séquentiel, créant une image qui peut être utilisée pour démarrer un nouveau conteneur. Ce processus assure que l'environnement est reproductible et isolé, facilitant ainsi le déploiement et la mise à l'échelle des applications.

Le fichier de variables d'environnement .env: Le gout du travail bien fait !

Les fichiers **.env** jouent un rôle crucial dans la gestion des configurations d'applications dans de nombreux environnements de développement. Ces fichiers permettent de stocker des configurations qui peuvent varier entre différents environnements (développement, test, production) sans changer le code source.

Structure et Usage

Format: Les fichiers **.env** sont généralement des fichiers texte simples contenant des paires clé-valeur, une paire par ligne.

Utilisation: Les valeurs stockées dans le fichier **.env** sont chargées dans l'environnement d'exécution de l'application au démarrage. Ces valeurs peuvent être utilisées pour configurer des paramètres tels que les chaînes de connexion aux bases de données, clés API, ports, et autres configurations nécessaires pour l'exécution de l'application.

Avantages

- **Sécurité:** permet de séparer les informations sensibles du code source, réduisant ainsi le risque d'exposer des mots de passe, des tokens API, etc. Cela est particulièrement important pour le code partagé via des répertoires publics ou privés.
- **Flexibilité:** Facilite la gestion des configurations qui varient entre les environnements sans nécessiter de modifications du code. Cela simplifie le déploiement et la maintenance des applications.
- **Simplicité:** Faciles à utiliser et à comprendre, ces fichiers sont largement adoptés dans la communauté de développement.

Exemple:

```
1 # Projet
2 BDD_NAME=PostgresDB_container
3 CONT_1_NAME=MageAI_container
4
5 # PostgreSQL
6 POSTGRES_DBNAME=my_pg_db
7 POSTGRES_USER=my_user
8 POSTGRES_PASSWORD=my_password
9 POSTGRES_PORT=5432
10 POSTGRES_HOST=5432
11 POSTGRES_SCHEMA=public
12
13 # MageAI
14 ENV=my_profil_custom
15
16 # PostgreSQL Host pour MageAI
17 POSTGRES_HOST=my_postgres_db
18 POSTGRES_SCHEMA=public
```

📄 Bonnes pratiques :

- **Ne jamais les inclure dans le contrôle de version !** Les fichiers `.env` doivent être exclus (par exemple, via `.gitignore` dans les projets Git) pour éviter la fuite de données sensibles.
- **Utiliser des fichiers `.env` spécifiques à l'environnement :** Pour des environnements multiples (développement, test, production), il peut être judicieux d'utiliser des fichiers distincts comme `.env.development`, `.env.test`, et `.env.production`. Certains frameworks et outils peuvent charger automatiquement le fichier pertinent en fonction de l'environnement d'exécution.
- **Intégration avec Docker :** Les fichiers `.env` peuvent être spécifiés dans le fichier `docker-compose.yml` pour charger automatiquement les variables d'environnement dans les services définis.

Docker-compose.yml: Musique maestro !

Le fichier `docker-compose.yml` est un fichier YAML utilisé pour définir et configurer les services, réseaux et volumes pour les applications Docker **multi-conteneurs**. Il permet de gérer et de déployer plusieurs conteneurs Docker simultanément, avec une seule commande au lieu de construire et exécuter chaque conteneur séparément avec des commandes Docker individuelles.

Si vous souhaitez en savoir plus, nous en parlons également très bien dans cette quête :

Fonctionnement

• Définition des services :

Chaque conteneur dans l'application est défini comme un "service" dans ce fichier. Ces services peuvent être des applications web, des bases de données, des caches en mémoire, etc. Pour chaque service, vous spécifiez l'image à utiliser, les commandes à exécuter, les ports à exposer, et d'autres configurations nécessaires.

Exemple:

```
1 services:
2
3   my_postgres_db:           # Nom du mon premier service.
4     image: postgres:14      # Utilise l'image officielle PostgreSQL version 14 comme base pour ce service.
5
6     ...                     # Les autres paramètres de ce service.
7
8   mage_ai:
9
10    build:
11      context: .              # Indique service sera construit à partir d'une image Docker définie dans le répertoire courant (indiqué par le .)
12      dockerfile: Dockerfile  # Construit l'image de l'application à partir du fichier "Dockerfile"
13
14    ...
15
```

- **Variables d'environnement :**

Les variables d'environnement peuvent être définies dans le fichier **docker-compose.yml** pour configurer les services selon les besoins.

Exemple:

```
1  services:
2
3  my_postgres_db:
4
5    ...
6
7  env_file:
8    - .env
9  # Charge les variables d'environnement définies dans le fichier .env situé dans le même dossier que docker-compose.yml.
10
11  container_name: ${BDD_NAME}
12
13  environment:
14    POSTGRES_DB: ${POSTGRES_DBNAME}
15    POSTGRES_USER: ${POSTGRES_USER}
16    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
17
18    ...
19
20  mage_ai:
21
22    ...
23
24  env_file:
25    - .env
26
```

```
26
27    ...
28
29  container_name: ${CONT_1_NAME}
30
31  environment:
32    ENV: ${ENV}
33    POSTGRES_DBNAME: ${POSTGRES_DBNAME}
34    POSTGRES_SCHEMA: ${POSTGRES_SCHEMA}
35    POSTGRES_USER: ${POSTGRES_USER}
36    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
37    POSTGRES_HOST: ${POSTGRES_HOST}
38    POSTGRES_PORT: ${POSTGRES_PORT}
39  # Définit des variables d'environnement utilisées par MageAI pour se connecter et interagir avec la base de données.
40
41    ...
```

❗ La syntaxe **\$(NOM_DE_MA_VARIABLE)** permet de récupérer le contenu référencé dans le fichier **.env** présent dans le dossier du projet.

- Réseau entre conteneurs :

Le fichier `docker-compose.yml` permet de définir des réseaux internes pour faciliter la communication entre les conteneurs sans exposer les ports au monde extérieur. Les conteneurs peuvent communiquer entre eux à travers ces réseaux en utilisant des noms de service au lieu d'adresses IP.

Si vous souhaitez creuser le sujet, nous en parlons aussi dans cette quête :

Exemple :

```
1 services:
2
3   my_postgres_db:
4
5     ...
6
7   ports:
8     - "${POSTGRES_PORT}:5432"
9
10  mage_ai:
11
12    ...
13
14    ports:
15      - "6789:6789"
16
17    ...
18
```

- Dépendances entre services :

Vous pouvez définir l'ordre de démarrage des services et spécifier des dépendances en utilisant des options comme `"depends_on"`.

Exemple :

```
1 services:
2
3   my_postgres_db:
4
5     ...
6
7   mage_ai:
8
9     ...
10
11    depends_on:
12      - my_postgres_db    # Spécifie que ce service dépend du service my_postgres_db, assurant que celui-ci soit démarré avant que mage_ai ne comm
13      ...
14
```

🕒 Construction vs. Orchestration : la récap'

Dockerfile : Priorise la **construction** de l'image. Il doit être complet et correct pour que l'image fonctionne comme prévu.

Docker Compose : Priorise l'**orchestration**, en s'assurant que les conteneurs basés sur ces images fonctionnent ensemble de manière optimale.

- Utilisation dans Docker Compose :

Si un service dans `docker-compose.yml` spécifie un **build**, cela indique à Docker Compose de **construire l'image à partir du Dockerfile** spécifié avant de démarrer les conteneurs.

- Variables d'environnement et configuration :

Docker Compose permet de configurer les conteneurs de manière plus flexible en utilisant des variables d'environnement et des fichiers d'environnement externes, ce qui peut compléter ou surcharger des configurations définies dans le **Dockerfile**.

- **Configuration des volumes :**

Les volumes peuvent être définis pour stocker et maintenir les données générées par votre pipeline, permettant ainsi une persistance des données au-delà de la durée de vie des conteneurs.

Si vous souhaitez creuser le sujet, nous en parlons également dans cette quête:

Exemple:

```
1 services:
2
3   my_postgres_db:
4
5     ...
6
7   mage_ai:
8
9     ...
10
11 volumes:          # À placer au même niveau que les services.
12   postgres_data:   # Indique le volume nommé postgres_data.
13
14   # Cela indique que le volume nommé postgres_data est monté dans le conteneur à l'emplacement /var/lib/postgresql/data.
```

① Sans ce mappage, toutes les données seraient perdues lorsque le conteneur serait détruit.

MageAI : Do your magic !

Pour déployer notre container MageAI, nous allons constituer un lot de librairies nécessaires au bon fonctionnement de notre projet MageAI. Pour ce faire, un moyen simple:

le fichier requirements.txt: Un grand classique

Exemple :

```
1 mage-ai           # Notre ETL préféré.
2 python-dotenv      # Pour lire nos variables d'environnement depuis le fichier .env.
3 ssh tunnel         # Création de tunnels SSH pour accéder de manière sécurisée à des services qui ne sont pas directement exposés sur le réseau.
4 psycopg2-binary   # Un adaptateur Python pour PostgreSQL.
5 pandas            # Pour le traitement de nos données dans notre pipeline.
6 datetime          # Toujours pratique pour formater et effectuer des calculs avec des objets date et heure.
7 boto3             # Interactions avec les services AWS.
8 dask[dataframe]   # Permet de manipuler de grands ensembles de données avec des DataFrames, avec des performances optimisées pour le traitement parallèle.
```


Le fichier io_config.yaml de MageAI

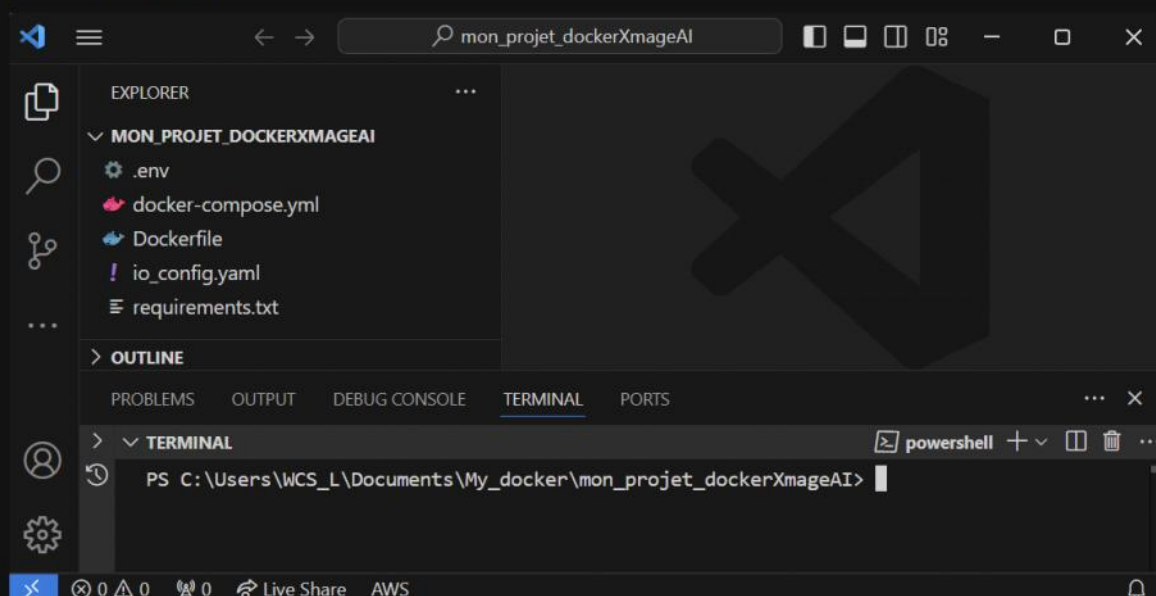
Ce fichier contient les informations de connexion **nécessaires à MageAI pour se connecter aux différentes bases de données, Data Warehouses, Data Lakes et services cloud.**

Ce fichier de paramètres par défaut est fourni dans [la documentation de MageAI](#).

Exemple :

```
1 version: 0.1.1
2
3 mon_profil_custom:
4
5   POSTGRES_CONNECT_TIMEOUT: 10
6   POSTGRES_DBNAME: {{ env_var('POSTGRES_DBNAME') }}
7   POSTGRES_SCHEMA: {{ env_var('POSTGRES_SCHEMA') }}
8   POSTGRES_USER: {{ env_var('POSTGRES_USER') }}
9   POSTGRES_PASSWORD: {{ env_var('POSTGRES_PASSWORD') }}
10  POSTGRES_HOST: {{ env_var('POSTGRES_HOST') }}
11  POSTGRES_PORT: {{ env_var('POSTGRES_PORT') }}
12
13 default:
14 # PostgreSQL
15   POSTGRES_CONNECT_TIMEOUT: 10
16   POSTGRES_DBNAME: postgres
17   POSTGRES_SCHEMA: public # Optional
18   POSTGRES_USER: username
19   POSTGRES_PASSWORD: password
20   POSTGRES_HOST: hostname
21   POSTGRES_PORT: 5432
22
23 ...
24
```

Une fois vos cinq fichiers correctement configurés,



il ne restera plus qu'à exécuter dans votre terminal la commande qui lancera le processus de déploiement :

```
1 docker-compose up --build --detach
```

Quelques explications :

–build :

Ce flag (option) indique à Docker Compose de reconstruire les images des conteneurs définis dans le fichier **docker-compose.yml** avant de démarrer les conteneurs. Cela signifie que même si une image Docker existe déjà pour un service, Docker Compose va ignorer cette image existante et la reconstruire à partir du **Dockerfile** ou des configurations de build définies. Ceci est particulièrement utile si vous avez modifié le Dockerfile ou d'autres fichiers nécessaires à la construction de l'image et que vous souhaitez vous assurer que les conteneurs utilisent les dernières versions de ces modifications.

–detach (ou -d) :

Dit à Docker Compose de démarrer les conteneurs en mode "détaché", ce qui signifie que les conteneurs seront lancés en arrière-plan. Lorsque cette option est utilisée, **vous récupérez immédiatement le contrôle de la ligne de commande après avoir lancé les conteneurs**. Vous n'aurez pas besoin de garder le terminal ouvert pour que les conteneurs continuent de fonctionner. Les logs de démarrage et de fonctionnement des conteneurs ne seront pas affichés directement dans le terminal ; vous pouvez les visualiser plus tard en utilisant des commandes comme **docker-compose logs**.

Et voilà !

Vos conteneurs vont apparaître dans votre client Docker comme par... magie.

The screenshot shows the Docker Desktop application window. The top bar is dark blue with the Docker logo and 'PERSONAL' label. A search bar is present with the text 'Search for images, containers, volumes... Ctrl+K'. The main area is titled 'Containers' and shows system metrics: 'Container CPU usage 0.09% / 1200% (12 CPUs available)' and 'Container memory usage 377.92MB / 7.4GB'. A table lists three running containers:

Name	Image	Status	Port(s)	CPU (%)	Last started	Actions
docker_mage_tuto		Running (2/2)		0.09%	2 hours ago	[Stop] [Refresh] [Delete]
PostgresDB_container	postgres:14	Running	5432:5432	0%	2 hours ago	[Stop] [Refresh] [Delete]
MageAI_container	docker_mage_tuto-mage_ai: d081fba37e03	Running	6789:6789	0.09%	2 hours ago	[Stop] [Refresh] [Delete]

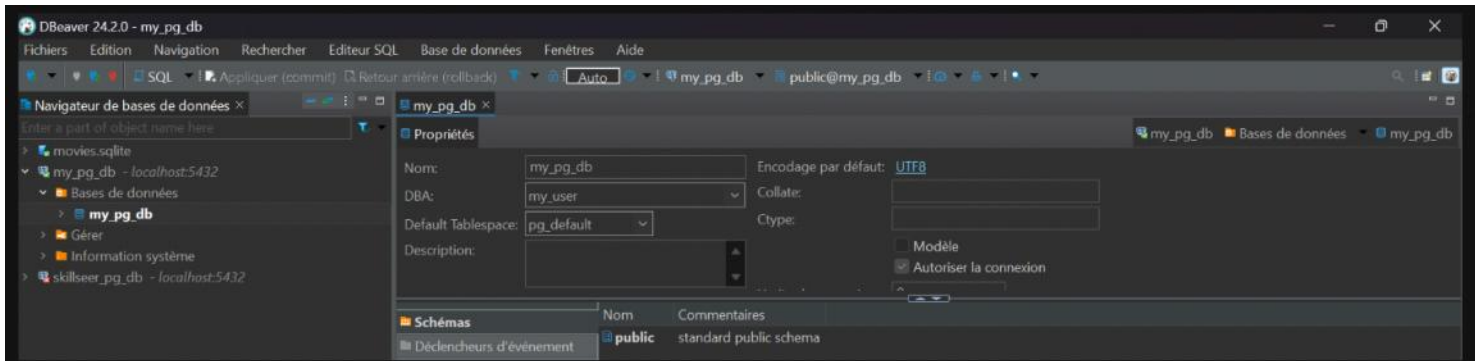
At the bottom, system status is shown: 'RAM 7.81 GB CPU 2.33% Disk ... GB avail. of ... GB'. A 'Terminal' button is visible in the bottom right corner.

Maintenant allons vérifier que tout fonctionne bien :

Pour mon conteneur MageAI:

The screenshot shows the 'Containers / MageAI_container' view in Docker Desktop. The container name 'MageAI_container' is prominently displayed. Below it, the image ID 'd081fba37e03' and the image name 'docker_mage_tuto-mage_ai:latest' are shown. The port mapping '6789:6789' is highlighted with a pink box. At the bottom, the 'Logs' tab is selected, showing the following log entries:

```
INFO:mage_ai.server.scheduler_manager:Scheduler status: running.
INFO:mage_ai.orchestration.pipeline_scheduler_original:Active pipel
INFO:mage_ai.orchestration.pipeline_scheduler_original:Active pipel
```



Le pipeline est en place, reste à lui dire quoi faire maintenant ! Mais

Création d'un Pipeline avec MageAI

Introduction

Dans ce cours, nous explorerons la création d'un pipeline de données en utilisant MageAI. Un pipeline de données est une série d'étapes automatisées qui permettent de collecter, transformer et charger des données de manière efficace et reproductible.

Objectifs

- À la fin de ce cours, vous serez capable de :
- Extraire des données à l'aide du bloc Data Loader de MageAI
 - Transformer et nettoyer les données avec le bloc Transformer
 - Charger les données dans une base PostgreSQL via le bloc Data Exporter
 - Créer un pipeline de données complet et fonctionnel

Le bloc Data Loader

Le Data Loader est responsable de l'extraction des données depuis diverses sources. Voici sa structure de base :

```
1 import io
2 import pandas as pd
3 import requests
4
5 @data_loader
6 def load_weather_data(*args, **kwargs):
7     """
8     Charge les données météo de plusieurs villes depuis une API
9     """
10    import requests
11    import pandas as pd
12
13    cities = ['Paris', 'Lyon', 'Marseille']
14    weather_data = []
15
16    # Simulation d'une API météo
17    for city in cities:
18        data = {
19            'city': city,
20            'date': pd.Timestamp.now(),
21            'temperature': 20, # En °C
22            'humidity': 75,    # En %
23            'rain': 0          # En mm
24        }
25        weather_data.append(data)
26
27    return pd.DataFrame(weather_data)
```

```
import io
import pandas as pd
import requests
@data_loader
def load_weather_data(*args, **kwargs):
    """
    Charge les données météo de plusieurs villes depuis une API
    """
    import requests
    import pandas as pd

    cities = ['Paris', 'Lyon', 'Marseille']
    weather_data = []

    # Simulation d'une API météo
    for city in cities:
        data = {
            'city': city,
            'date': pd.Timestamp.now(),
            'temperature': 20, # En °C
            'humidity': 75,    # En %
            'rain': 0          # En mm
        }
        weather_data.append(data)

    return pd.DataFrame(weather_data)
```

Le décorateur @data_loader est un design pattern Python qui permet de modifier ou d'enrichir une fonction sans modifier son code source.

Le bloc Transformer

Le Transformer permet de modifier et nettoyer les données. Voici sa structure :

```
1 @transformer
2 def analyze_weather(data, *args, **kwargs):
3     """
4     Analyse les données météo et ajoute des indicateurs
5     """
6     df = data.copy()
7
8     # Ajoute des indicateurs
9     df['weather_status'] = df['temperature'].apply(
10         lambda t: 'Froid' if t < 15
11         else 'Tempéré' if t < 25
12         else 'Chaud'
13     )
14
15     # Calcule des moyennes par ville
16     df['city'] = df['city'].str.upper()
17
18     return df
```

```
@transformer
def analyze_weather(data, *args, **kwargs):
    """
    Analyse les données météo et ajoute des indicateurs
    """
    df = data.copy()

    # Ajoute des indicateurs
    df['weather_status'] = df['temperature'].apply(
        lambda t: 'Froid' if t < 15
        else 'Tempéré' if t < 25
        else 'Chaud'
    )

    # Calcule des moyennes par ville
    df['city'] = df['city'].str.upper()

    return df
```

Il est recommandé de créer plusieurs blocs Transformer, chacun ayant une responsabilité unique, pour faciliter la maintenance et le débogage.

Le bloc Data Exporter

Le Data Exporter permet de sauvegarder les données transformées :

```
1 @data_exporter
2 def export_weather_data(df: DataFrame, **kwargs) -> None:
3     """
4     Exporte les données météo vers PostgreSQL
5     """
6     config_path = path.join(get_repo_path(), 'io_config.yaml')
7     config_profile = 'MyConfigProfile'
8     schema_name = 'public'
9     table_name = 'weather_data'
10
11     with Postgres.with_config(ConfigFileLoader(config_path, config_profile)) as loader:
12         loader.export(
13             df,
14             schema_name,
15             table_name,
16             index=False,
17             if_exists='replace',
18         )

@data_exporter
def export_weather_data(df: DataFrame, **kwargs) -> None:
    """
    Exporte les données météo vers PostgreSQL
    """
    config_path = path.join(get_repo_path(), 'io_config.yaml')
    config_profile = 'MyConfigProfile'
    schema_name = 'public'
    table_name = 'weather_data'
    with Postgres.with_config(ConfigFileLoader(config_path, config_profile)) as loader:
        loader.export(
            df,
            schema_name,
            table_name,
            index=False,
            if_exists='replace',
        )
```


Création d'un Pipeline avec MageAI/PostgreSQL

Introduction

Dans ce cours, nous explorerons la création d'un pipeline de données en utilisant MageAI, en mettant l'accent sur les problèmes courants et leurs solutions. Un pipeline de données est une série d'étapes automatisées qui permettent de collecter, transformer et charger des données de manière efficace et reproductible.

Objectifs

- À la fin de ce cours, vous serez capable de :
- ✓ Extraire des données avec gestion des problèmes d'encodage
 - ✓ Transformer et nettoyer les données efficacement
 - ✓ Configurer correctement une connexion PostgreSQL
 - ✓ Gérer les erreurs courantes dans un pipeline

Initiatisation

Après avoir lancé Docker Desktop et que vous vous assurez que les container Docker et Postgre qu'on a crée pendant la quête précédente sont exécutés comme dans l'image suivante :

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage

0.91% / 1200% (12 CPUs available)

Container memory usage

1.79GB / 15.21GB

Show charts

Q Search

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	illad	-	-	-	0.87%	4 hours ago	
☐	mage-zoomcamp	-	-	-	0.04%	3 hours ago	
☐	wild_mage_project_mageai	b0c26a296b0e	mage-zoom	6789:6789	0.04%	3 hours ago	
☐	wild_mage_project_postgres	417cead03fca	postgres:14	5432:5432	0%	3 hours ago	

Ouvrez votre navigateur et mettez le lien : <http://localhost:6789/>

Sélectionnez "New" puis "Standard (batch)" comme dans l'image suivante puis mettez le nom qui vous convient à votre pipeline:

Pipelines

New

Filter

Group

A. Data integration

B. Streaming

C. From a template

D. Import pipeline zip

E. Using AI (beta)

Description	Type	Updated at	Created at	Tags	Blocks	Triggers
A. Data integration	Standard	2023-01-29 22:06:26	2023-01-29 22:11:19		5	0

Les étapes suivantes vont vous permettre de construire les blocs comme expliquer dans la quête suivez les instructions et suivez les explications pour comprendre l'intérêt de chaque étape.

Le bloc Data Loader

Le Data Loader est responsable de l'extraction des données. Voici un exemple robuste avec gestion des erreurs :

```
1 import io
2 import pandas as pd
3 import requests
4
5 @data_loader
6 def load_data_from_api(*args, **kwargs):
7     """
8     Charge les données avec gestion des erreurs et de l'encodage
9     """
10    url = 'https://www.data.gouv.fr/fr/datasets/r/dbe8a621-a9c4-4bc3-9cae-be1699c5ff25'
11
12    try:
13        response = requests.get(url)
14        response.raise_for_status()
15
16        # Test de différents encodages
17        encodings = ['latin1', 'utf-8', 'iso-8859-1']
18        for encoding in encodings:
19            try:
20                df = pd.read_csv(io.StringIO(response.content.decode(encoding)), sep=',')
21                # Vérification rapide de l'encodage
22                if not df['nom_commune_complet'].str.contains('À').any():
23                    return df
24            except UnicodeDecodeError:
25                continue
26
27        # Si aucun encodage n'a fonctionné, utiliser latin1 avec gestion spécifique
28        return pd.read_csv(io.StringIO(response.content.decode('latin1')), sep=',')
29
30 except requests.exceptions.RequestException as e:
31     raise Exception(f"Erreur lors de la récupération des données : {e}")
32
33 @test
34 def test_output(output, *args) -> None:
35     """
36     Vérifie la validité des données et l'encodage
37     """
38     assert output is not None, 'Les données sont vides'
39     assert 'nom_commune_complet' in output.columns, 'Colonne nom_commune_complet manquante'
40     assert not output['nom_commune_complet'].str.contains('À').any(), "Problème d'encodage détecté"
```

```
import io
import pandas as pd
import requests
@data_loader
def load_data_from_api(*args, **kwargs):
    """
    Charge les données avec gestion des erreurs et de l'encodage
    """
    url = 'https://www.data.gouv.fr/fr/datasets/r/dbe8a621-a9c4-4bc3-9cae-be1699c5ff25'

    try:
        response = requests.get(url)
        response.raise_for_status()

        # Test de différents encodages
        encodings = ['latin1', 'utf-8', 'iso-8859-1']
        for encoding in encodings:
            try:
                df = pd.read_csv(io.StringIO(response.content.decode(encoding)), sep=',')
                # Vérification rapide de l'encodage
                if not df['nom_commune_complet'].str.contains('À').any():
                    return df
            except UnicodeDecodeError:
                continue

        # Si aucun encodage n'a fonctionné, utiliser latin1 avec gestion spécifique
        return pd.read_csv(io.StringIO(response.content.decode('latin1')), sep=',')

    except requests.exceptions.RequestException as e:
        raise Exception(f"Erreur lors de la récupération des données : {e}")

@test
def test_output(output, *args) -> None:
    """
    Vérifie la validité des données et l'encodage
    """
    assert output is not None, 'Les données sont vides'
    assert 'nom_commune_complet' in output.columns, 'Colonne nom_commune_complet manquante'
    assert not output['nom_commune_complet'].str.contains('À').any(), "Problème d'encodage détecté"
```

❗ Problèmes courants du Data Loader :

1. Erreur d'encodage : Les caractères spéciaux apparaissent mal (ex: 'Ã©' au lieu de 'é')
2. Colonnes manquantes : Vérifiez les noms exacts des colonnes
3. Erreurs HTTP : Vérifiez l'URL et votre connexion

Le bloc Transformer

Le Transformer est essentiel pour le nettoyage des données. Voici comment gérer les problèmes d'encodage :

```
1 @transformer
2 def transform_commune_data(data, *args, **kwargs):
3     """
4     Nettoie et formate les données des communes
5     """
6     df_clean = data.copy()
7
8     # Correction d'encodage (si nécessaire après le Data Loader)
9     for column in ['nom_commune_complet', 'nom_region']:
10         df_clean[column] = df_clean[column].apply(
11             lambda x: x.encode('latin1', errors='ignore').decode('utf-8', errors='ignore')
12             if isinstance(x, str) else x
13         )
14
15     # Mise en majuscules des noms de régions
16     df_clean['nom_region'] = df_clean['nom_region'].str.upper()
17
18     # Sélection des colonnes nécessaires
19     df_clean = df_clean[['code_postal', 'nom_commune_complet', 'nom_region']]
20
21     return df_clean
22
23 @test
24 def test_transform_output(output, *args) -> None:
25     """
26     Vérifie la transformation avec diagnostic
27     """
28     # Affichage pour diagnostic
29     print("Exemple de communes :", output['nom_commune_complet'].head())
30     print("Exemple de régions :", output['nom_region'].head())
31
32     # Vérifications
33     assert set(output.columns) == {'code_postal', 'nom_commune_complet', 'nom_region'}, \
34         "Colonnes incorrectes"
35     assert output['nom_region'].str.isupper().all(), \
36         "Certaines régions ne sont pas en majuscules"
37     assert not output['nom_commune_complet'].str.contains('Ã').any(), \
38         "Problème d'encodage dans les communes"
```

```
@transformer
def transform_commune_data(data, *args, **kwargs):
    """
    Nettoie et formate les données des communes
    """
    df_clean = data.copy()

    # Correction d'encodage (si nécessaire après le Data Loader)
    for column in ['nom_commune_complet', 'nom_region']:
        df_clean[column] = df_clean[column].apply(
            lambda x: x.encode('latin1', errors='ignore').decode('utf-8', errors='ignore')
            if isinstance(x, str) else x
        )

    # Mise en majuscules des noms de régions
    df_clean['nom_region'] = df_clean['nom_region'].str.upper()

    # Sélection des colonnes nécessaires
    df_clean = df_clean[['code_postal', 'nom_commune_complet', 'nom_region']]

    return df_clean

@test
def test_transform_output(output, *args) -> None:
    """
    Vérifie la transformation avec diagnostic
    """
    # Affichage pour diagnostic
    print("Exemple de communes :", output['nom_commune_complet'].head())
    print("Exemple de régions :", output['nom_region'].head())

    # Vérifications
    assert set(output.columns) == {'code_postal', 'nom_commune_complet', 'nom_region'}, \
        "Colonnes incorrectes"
    assert output['nom_region'].str.isupper().all(), \
        "Certaines régions ne sont pas en majuscules"
    assert not output['nom_commune_complet'].str.contains('Ã').any(), \
        "Problème d'encodage dans les communes"
```

❗ Si les problèmes d'encodage persistent :

1. Vérifiez l'encodage dès la lecture dans le Data Loader
2. Utilisez des outils de diagnostic (print des données)
3. Testez différentes combinaisons d'encode/decode

Le bloc Data Exporter

La configuration de PostgreSQL est une étape critique. Voici comment procéder :

1. Créez ou modifiez le fichier `io_config.yaml` dans le répertoire de votre projet :

```
1 MyConfigProfile:
2   POSTGRES_CONNECT_TIMEOUT: 10
3   POSTGRES_DBNAME: mage          # Nom de votre base de données
4   POSTGRES_HOST: postgres        # Nom du service dans docker-compose
5   POSTGRES_PORT: 5432           # Port PostgreSQL standard
6   POSTGRES_USER: mage           # Votre utilisateur
7   POSTGRES_PASSWORD: mage       # Votre mot de passe
8
9   version: 0.1.1
```

```
MyConfigProfile:
  POSTGRES_CONNECT_TIMEOUT: 10
  POSTGRES_DBNAME: mage          # Nom de votre base de données
  POSTGRES_HOST: postgres        # Nom du service dans docker-compose
  POSTGRES_PORT: 5432           # Port PostgreSQL standard
  POSTGRES_USER: mage           # Votre utilisateur
  POSTGRES_PASSWORD: mage       # Votre mot de passe
  version: 0.1.1
```

2. Code du Data Exporter avec gestion d'erreurs :

```
1 from mage_ai.settings.repo import get_repo_path
2 from mage_ai.io.config import ConfigFileLoader
3 from mage_ai.io.postgres import Postgres
4 from pandas import DataFrame
5 from os import path
6
7 @data_exporter
8 def export_commune_data(df: DataFrame, **kwargs) -> None:
9     """
10     Exporte les données vers PostgreSQL avec gestion d'erreurs
11     """
12     config_path = path.join(get_repo_path(), 'io_config.yaml')
13
14     # Vérification du fichier de configuration
15     if not path.exists(config_path):
16         raise FileNotFoundError(f"Le fichier de configuration n'existe pas : {config_path}")
17
18     try:
19         config_profile = 'MyConfigProfile'
20         schema_name = 'public'
21         table_name = 'communes_france'
22
23         with Postgres.with_config(ConfigFileLoader(config_path, config_profile)) as loader:
24             loader.export(
25                 df,
26                 schema_name,
27                 table_name,
28                 index=False,
29                 if_exists='replace',
30             )
31             print(f"Données exportées avec succès dans {schema_name}.{table_name}")
32
33     except KeyError as e:
34         print(f"Erreur de configuration : {e}")
35         print("Vérifiez que votre fichier io_config.yaml contient une section 'MyConfigProfile'")
36         raise
37     except Exception as e:
38         print(f"Erreur lors de l'export : {e}")
39         print("Vérifiez vos paramètres de connexion et que PostgreSQL est accessible")
40         raise
```

```
from mage_ai.settings.repo import get_repo_path
from mage_ai.io.config import ConfigFileLoader
from mage_ai.io.postgres import Postgres
from pandas import DataFrame
from os import path

@data_exporter
def export_commune_data(df: DataFrame, **kwargs) -> None:
    """
    Exporte les données vers PostgreSQL avec gestion d'erreurs
    """
    config_path = path.join(get_repo_path(), 'io_config.yaml')

    # Vérification du fichier de configuration
    if not path.exists(config_path):
        raise FileNotFoundError(f"Le fichier de configuration n'existe pas : {config_path}")

    try:
        config_profile = 'MyConfigProfile'
        schema_name = 'public'
        table_name = 'communes_france'
        with Postgres.with_config(ConfigFileLoader(config_path, config_profile)) as loader:
            loader.export(
                df,
                schema_name,
                table_name,
                index=False,
                if_exists='replace',
            )
            print(f"Données exportées avec succès dans {schema_name}.{table_name}")

    except KeyError as e:
        print(f"Erreur de configuration : {e}")
        print("Vérifiez que votre fichier io_config.yaml contient une section 'MyConfigProfile'")
        raise
    except Exception as e:
        print(f"Erreur lors de l'export : {e}")
        print("Vérifiez vos paramètres de connexion et que PostgreSQL est accessible")
        raise
```

❗ Erreurs courantes PostgreSQL :

1. `KeyError 'MyConfigProfile'` : Votre `io_config.yaml` est mal configuré
2. `Connection refused` : PostgreSQL n'est pas accessible
3. `Authentication failed` : Identifiants incorrects

Guide de dépannage

1. Problèmes d'encodage

Si vous voyez des caractères comme "Ã©", "Ã", etc. :

- Vérifiez l'encodage dans le Data Loader en premier
- Testez différents encodages (latin1, utf-8, iso-8859-1)
- Utilisez les prints de diagnostic pour voir les données

2. Erreurs PostgreSQL

Si vous avez des erreurs de connexion :

1. Vérifiez votre fichier `io_config.yaml` :
 - Il doit être dans le bon répertoire
 - La section 'MyConfigProfile' doit être présente
 - Les informations de connexion doivent être correctes
2. Vérifiez que PostgreSQL est accessible :
 - Les services Docker sont-ils lancés ?
 - Les ports sont-ils corrects ?
 - Les identifiants sont-ils bons ?

Mode opératoire

Créez un pipeline complet qui :

1. Récupère les données des communes françaises
2. Nettoie et formate les données
3. Exporte vers PostgreSQL

Étapes suggérées

1. Commencez par le Data Loader avec gestion d'encodage
2. Testez la transformation sur un petit échantillon
3. Vérifiez la configuration PostgreSQL avant l'export

Critères d'acceptation

- Les données sont correctement encodées (pas de caractères spéciaux mal formés)
- Les trois colonnes requises sont présentes et bien formatées
- Les régions sont en majuscules
- Les données sont accessibles dans PostgreSQL

Correction checkpoint

mardi 24 juin 2025 16:49

Checkpoint 1

Sql
Business Intelligence
Algo python
Opérateur logique (if, elif, etc ...)

<https://colab.research.google.com/drive/1i254OSWPzFCvbw08uCXvIwVu8ETo3J-?usp=sharing>

Checkpoint 2

Eda (analyse exploratoire des données)
Transformation avec apply pour un df ou map sur une serie
Remplacement des valeur manquantes
Visualisation matplotlib
Correlation + heatmap de correlation avec mask
Nuage de point
Box plot - boite a moustache
Graph interactif avec plotly.express
Machine learning :

- classification RandomForestClassifier
 - o Matrice de confusion
- Regression lineaire avec pipeline
 - o Avaluation du modèle R2 Score et RMSE

<https://colab.research.google.com/drive/1ZuiFaPDd1pwgCPtMVXm8ewBKvXWsUTI>

Checkpoint 3

Requête API
Géolocalisation
NLP
Régression logistique + évaluation du modèle
Arbre de décision + évaluation du modèle
GridSearch

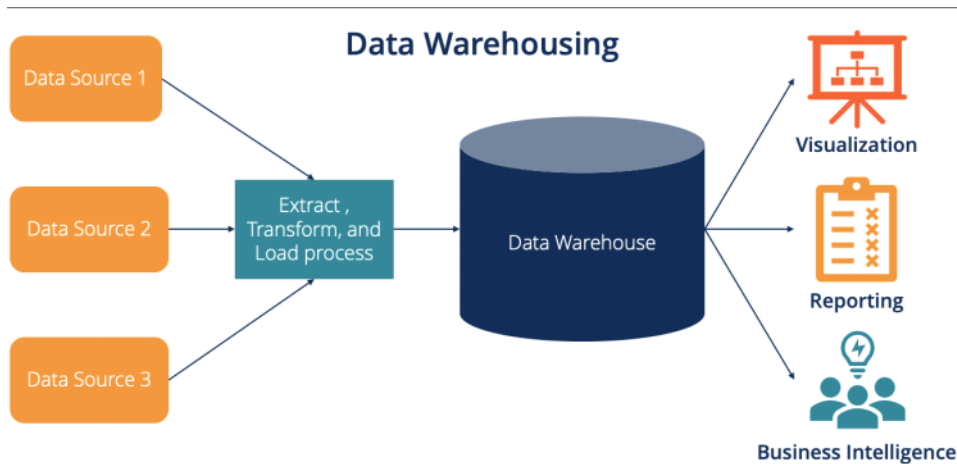
<https://colab.research.google.com/drive/1cCrGo41sor5d1i1dN40FwfT1i6zDYGax>

Mission Data

Exemple de réalisation

Préparation des données
Matrice de corrélation
Heatmap sans mask
Pairplot (génère des graph sur col numérique)
ML supervisé - KNeighborsClassifier
Prédiction

https://colab.research.google.com/drive/1gH_6fOKS3QCg4pg1qUud2arznzUhz-4



Transformation - préparer la donnée pour stockage dans le Data warehouse

Nettoyage des données

Le nettoyage des données supprime les erreurs et aligne les données sources sur le format des données cibles. Par exemple, vous pouvez remplacer les valeurs manquantes par des zéros, ou convertir les valeurs "Homme" en "H" et "Femme" en "F".

Déduplication des données

La déduplication identifie et supprime les enregistrements en double.

Révision du format des données

La révision du format convertit les données telles que les chaînes de caractères, les unités de mesure et les valeurs de date/heure en un format cohérent. Par exemple, une entreprise alimentaire peut avoir différentes bases de données de recettes où les ingrédients sont mesurés en kilogrammes ou en livres. L'ETL convertira tout dans une seule unité de mesure (soit en livres, soit en kilogrammes, mais pas les deux).

Chargement (Load)

Le chargement consiste à déplacer les données transformées depuis l'emplacement de stockage temporaire vers le data warehouse cible. Pour la plupart des organisations, ce processus est automatisé.

Alternative en utilisant uv

je vous conseille de laisser tomber requirements.txt et pip install.
 Faites : uv init
 (si uv n'est pas installé : pip install uv)
 uv venv
 source .venv/Scripts/activate (PC) ou source .venv/bin/activate (Mac)
 uv add dbt-core dbt-mysql pandas pymysql sqlalchemy sqlalchemy_utils
 dbt init

Installation de DBT

DBT Core est un package Python, il nécessite donc que Python soit installé sur votre machine.

Pour connecter **DBT Core** à une base de données, il est également nécessaire d'installer un adaptateur spécifique.

Dans notre exemple, nous allons connecter **DBT** à une base de données **MySQL**.

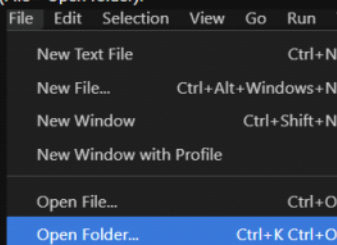
1 Création & vérification du bon fonctionnement d'un environnement virtuel.

⚠ L'environnement virtuel doit être créé en local sur ta machine. Donc tu ne dois pas utiliser Google Colab.

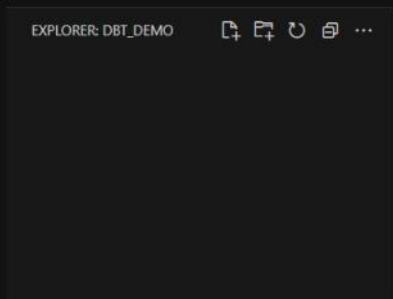
- Créer un nouveau dossier sur votre ordinateur, ici je le nomme DBT_DEMO. Mais tu peux l'appeler, comme bon te semble :

📁 > Documents > DBT_DEMO

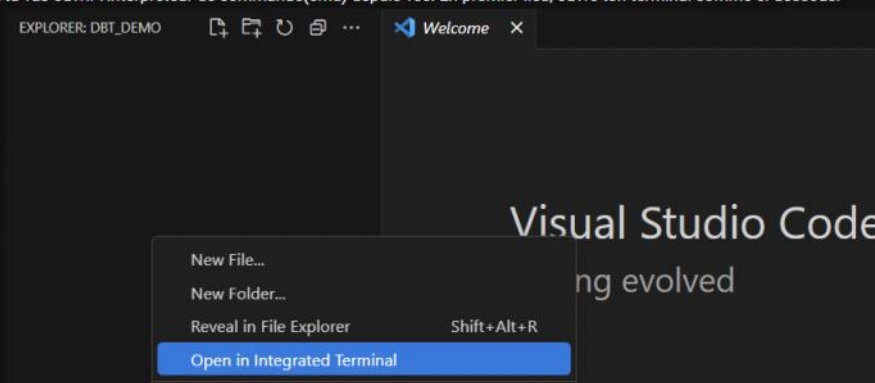
- Ouvrir, Visual Studio Code. Dans ce dernier tu dois ouvrir un dossier (File > Open folder).



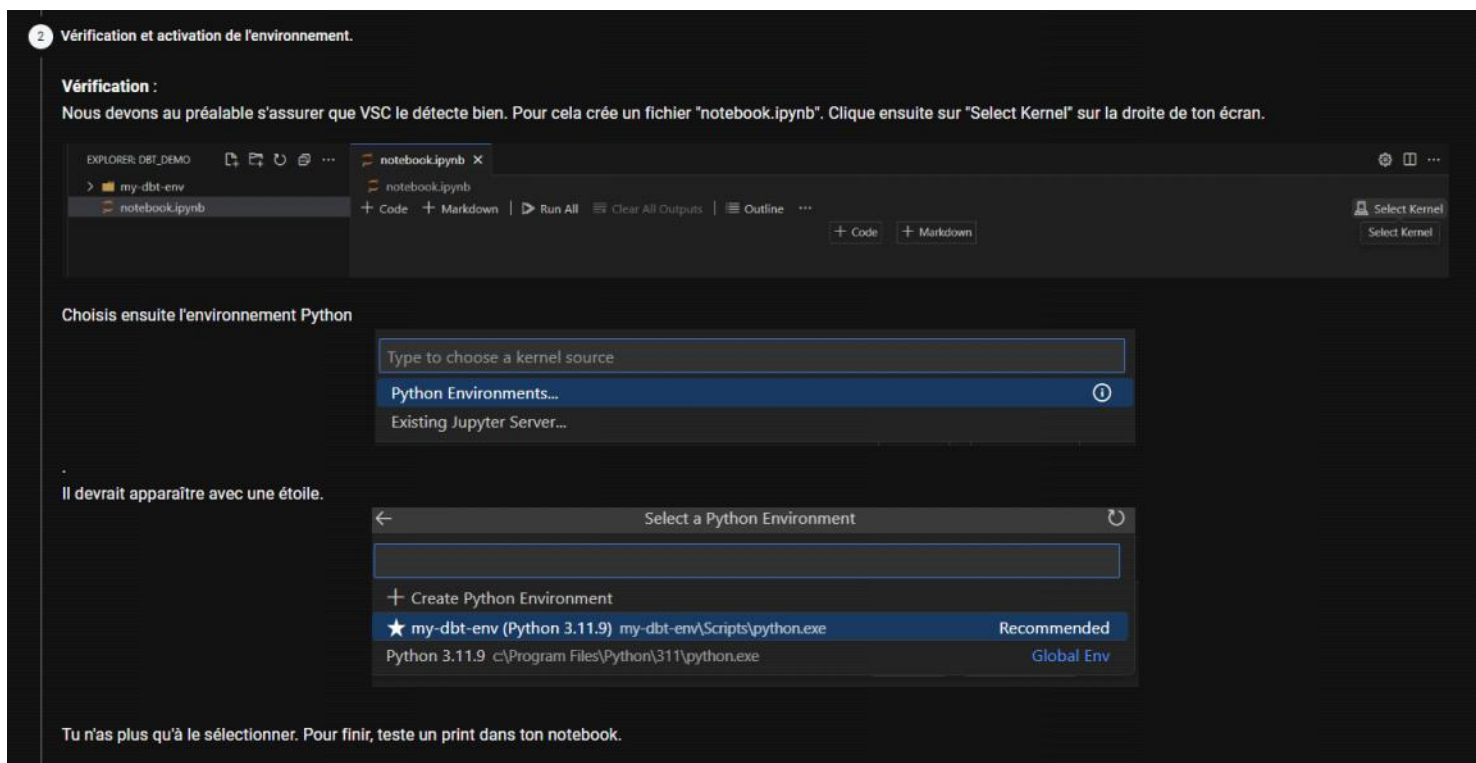
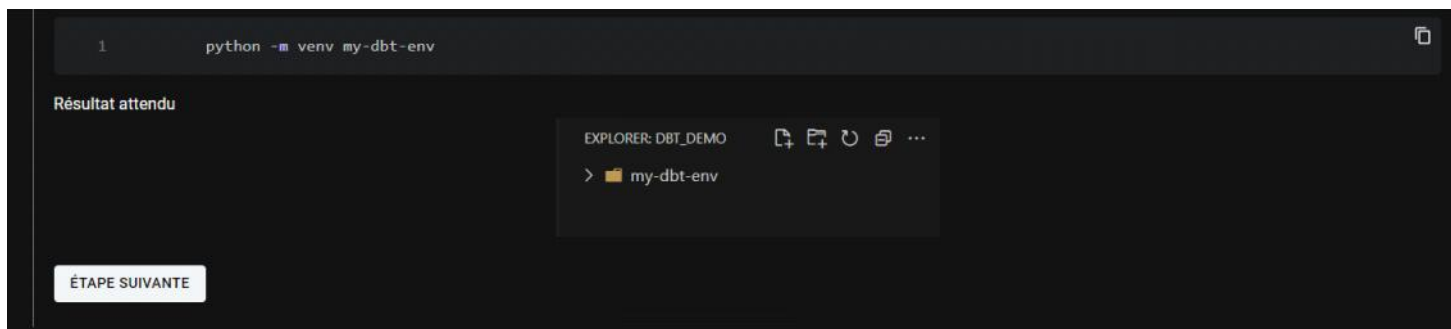
Tu sélectionnes le dossier que tu viens de créer. Résultat attendu :



- Tu es à la racine de ton dossier, tu vas ouvrir l'interpréteur de commande(cmd) depuis vsc. En premier lieu, ouvre ton terminal comme ci-dessous.



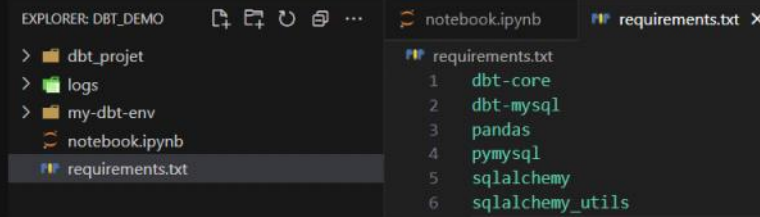
Ouvre ensuite un terminal(Command Prompt), VSC pourra ainsi détecter le nouvel environnement que tu vas créer. Maintenant que tu es dans command Prompt, tu vas pouvoir exécuter la commande ci dessous.



✓ Vérification et activation de l'environnement.

3 Installation des dépendances

A la racine de ton dossier, crée maintenant un fichier "requirements.txt" qui va contenir toutes les dépendances à installer pour cette quête. Le fichier devra se présenter comme ci-dessous.



```
EXPLORER: DBT_DEMO
> dbt_projet
> logs
> my-dbt-env
notebook.ipynb
requirements.txt

notebook.ipynb
requirements.txt
1 dbt-core
2 dbt-mysql
3 pandas
4 pymysql
5 sqlalchemy
6 sqlalchemy_utils
```

Tu peux maintenant exécuter le code suivant pour les installer

```
1 pip install -r requirements.txt
```

RETOUR

ÉTAPE SUIVANTE

4 vérifier que DBT est correctement installé.

```
1 dbt --version
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(my-dbt-env) C:\Users\phili\Documents\DBT_DEMO>dbt --version
Core:
- installed: 1.7.9
- latest: 1.9.1 - Update available!

Your version of dbt-core is out of date!
You can find instructions for upgrading here:
https://docs.getdbt.com/docs/installation

Plugins:
- mariadb: 1.7.0 - Could not determine latest version
- mysql: 1.7.0 - Up to date!
- mysqls: 1.7.0 - Could not determine latest version
```

RETOUR

ÉTAPE SUIVANTE

5 Initialisation d'un projet DBT

Toujours dans ton terminal (Command Prompt), exécute le code suivant:

```
1 dbt init
```

```
(my-dbt-env) C:\Users\phili\Documents\DBT_DEMO>dbt init
15:39:49 Running with dbt=1.7.9
Enter a name for your project (letters, digits, underscore):
```

- Renseigne le nom de ton projet

```
Happy modeling!

15:42:02 Setting up your profile.
Which database would you like to use?
[1] mariadb
[2] mysql
[3] mysql5

(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)
Enter a number:
```

- Renseigne le numéro correspondant à mysql (dans mon cas, c'est le numéro 2)

Un nouveau dossier portant le nom du projet que tu as indiqué sera créé, avec plusieurs sous-dossiers.

```
EXPLORER: DBT_DEMO
> dbt_projet
> logs
> my-dbt-env
notebook.ipynb
requirements.txt
```

Bon jusque-là, **TOUT VA BIEN**. Nous allons maintenant pouvoir utiliser **SQL Alchemy**. Par ici, la découverte en détails dans cette [quête](#).

Ce script va permettre de créer une connexion vers MySQL Server installé sur votre ordinateur, créer une base de données et rajouter des données depuis un CSV vers une table.

```
1 import pandas as pd
2 from sqlalchemy import create_engine
3 from sqlalchemy_utils import database_exists, create_database
4
5 # Paramètres de connexion.
6 username = "root"
7 password = "password"
8 host = "localhost"
9 port = 3306
10 database = "my_dbt_db"
11
12 # On crée la connexion vers la base de données.
13 DATABASE_URI = f'mysql+pymysql://{username}:{password}@{host}:{port}/{database}'
14 engine = create_engine(DATABASE_URI)
15
16 # On crée la base de données si elle n'existe pas.
17 if not database_exists(engine.url):
18     create_database(engine.url)
19
20 # On crée un DataFrame pour chaque fichier CSV de la base de données.
21 liste_tables = ["customers", "items", "orders", "products", "stores", "supplies"]
22
23 for table in liste_tables:
24     csv_url = f"https://raw.githubusercontent.com/dsteddy/jaffle_shop_data/main/raw_{table}.csv"
25     df = pd.read_csv(csv_url)
26     # On ajoute les informations de dataframe à la table dans MySQL.
27     df.to_sql(f"raw_{table}", engine, if_exists="replace", index=False)
```

❗ **Remarque :** Pensez à remplacer **root** et **password** par le nom d'utilisateur et le mot de passe que vous avez définis sur votre machine lorsque vous avez installé MySQL Server.

❗ Si votre mot de passe contient un caractère spécial tel que `@`, vous devrez peut-être encoder ce caractère pour l'utiliser correctement. Par exemple, remplacez `@` par `%40`. Vous pouvez utiliser un outil comme [URL Encoder](#) pour encoder votre mot de passe.

❗ Si vous rencontrez une erreur, vous aurez peut-être besoin d'installer la librairie 'cryptography'.

Une fois la base de données et les tables créées, vous devrez indiquer les informations de connexion à la base de données dans **DBT**.

Pour cela, modifiez le fichier `profiles.yml` qui a été créé lors de l'exécution de `dbt init`.

- Sur Windows, voici le chemin d'accès :

```
1 C:\Users\NomDuUser\.dbt\profiles.yml
```

- Sur Mac, le code à exécuter :

```
1 code ~/.dbt/profiles.yml
```

Par défaut, ce fichier ressemble à ceci :

```
1 dbt_quest:
2   target: dev
3   outputs:
4     dev:
5       type: mysql
6       server: localhost
7       port: 3306
8       database: my_dbt_db
9       schema: my_dbt_db
10      username: root
11      password: password
12      driver: MySQL ODBC 8.0 ANSI Driver
```

Pour cette quête, vous n'avez besoin de modifier que la partie dev.

Remplacez `server`, `port`, `my_dbt_db`, `root`, et `password` par les mêmes informations que celles utilisées pour créer la base de données avec **SQLAlchemy**.

Vous êtes maintenant prêt à utiliser **DBT Core** pour gérer vos transformations de données !

Structure d'un projet DBT :

Vous pouvez maintenant revenir au dossier créé lors de `dbt init` et inspecter son contenu. |

Dossier/Fichier	Description
<code>./dbt_project.yml</code>	Le fichier de configuration principal du projet dbt, contenant les paramètres globaux, les noms des dossiers et les configurations spécifiques aux modèles et tests.
<code>./models</code>	Contient les modèles SQL définissant les transformations de données. Ces modèles créent des vues ou des tables matérialisées dans le data warehouse.
<code>./macros</code>	Contient les macros dbt, des fonctions réutilisables écrites en Jinja pour faciliter l'écriture de code SQL dans les modèles ou tests.
<code>./tests</code>	Dossier dédié aux tests personnalisés, qui vérifient la validité et l'intégrité des données selon les règles définies.
<code>./analyses</code>	Stocke les fichiers SQL pour les analyses exploratoires ou ponctuelles qui ne créent pas de nouvelles tables ou vues dans le data warehouse.
<code>./snapshots</code>	Contient les fichiers de snapshots pour capturer l'état d'une table à un moment donné, permettant de suivre les changements au fil du temps.
<code>./seeds</code>	Contient des fichiers CSV qui sont directement intégrés dans le data warehouse. Utilisé pour des petits ensembles de données statiques ou semi-statiques.
<code>./logs</code>	Stocke les fichiers de logs générés lors de l'exécution des commandes dbt telles que <code>dbt run</code> , <code>dbt test</code> , <code>dbt compile</code> , etc.

Organisation des modèles

Il est recommandé de structurer le dossier `models` en sous-dossiers pour une meilleure organisation. Cette structuration peut varier en fonction de la taille du projet et des équipes impliquées.

Voici quelques exemples d'organisations courantes :

- **Par domaine fonctionnel (domain-driven)** : Créer des sous-dossiers pour chaque domaine d'activité ou sujet (ex. : `sales`, `marketing`, `finance`).
- **Par type de transformation** : Organiser les modèles en fonction du type de transformation effectuée (ex. : `staging`, `intermediate`, `mart`).
- **Par équipe** : Structurer les dossiers en fonction des équipes responsables de chaque partie du projet (ex. : `analytics_team`, `data_engineering_team`).

Exemple de modèle dans DBT

Prenons l'exemple de la base de données `my_dbt_db` sur **MySQL**, vue dans la quête précédente.

Nous avons plusieurs tables, dont la table des clients `raw_customers`.

Imaginons que nous souhaitions analyser les données clients pour obtenir le nombre total de commandes passées par chaque client.

Dans le dossier `models`, vous trouverez un dossier `example`, que vous pouvez supprimer et remplacer par un dossier `staging`.

La partie **staging** est dédiée à la préparation des données : nettoyage, normalisation, et mise en forme avant utilisation dans les modèles finaux.

Ensuite, créez un fichier `sources.yml` dans ce dossier. Ce fichier permettra de définir les tables sources utilisées dans les modèles de **staging**, facilitant ainsi la gestion des dépendances et la mise en place des tests.

```
1 version: 2
2
3 sources:
4   - name: my_dbt_db
5     schema: my_dbt_db
6
7   tables:
8     - name: raw_customers
```

Détails du fichier `sources.yml` :

- **version: 2** : Indique la version de la spécification YAML utilisée (v2 est la version actuelle pour DBT).
- **sources** : Définit la section contenant les sources de données.
- **name: my_dbt_db** : Le nom de la source de données, souvent lié au projet ou à l'entrepôt de données.
- **schema** : Le nom du schéma où sont stockées les tables brutes (dans cet exemple, il s'agit du schéma de la base de données MySQL).
- **tables** : Liste des tables brutes référencées dans vos modèles.
 - **name: raw_customers** : Le nom de la table brute.

Ensuite, dans le dossier `staging`, créez un fichier `stg_customers.sql` dans lequel vous utiliserez la source définie dans le fichier `sources.yml` :

```
1 with source as (  
2   select * from {{ source('my_dbt_db', 'raw_customers') }}  
3 ),  
4 renamed as (  
5   select  
6     id as customer_id,  
7     name as customer_name  
8   from source  
9 )  
10 select * from renamed
```

Explication du fichier SQL :

- La CTE `source` récupère toutes les colonnes de la table brute `raw_customers`. Cela centralise l'accès aux données brutes, simplifiant ainsi les futures références.
- La CTE `renamed` applique une transformation en renommant les colonnes pour les rendre plus descriptives. Cette étape est importante pour préparer les données à des transformations ou analyses ultérieures.
- La requête finale sélectionne toutes les colonnes de la CTE `renamed`, offrant une approche modulaire pour appliquer des transformations intermédiaires tout en conservant un modèle final clair et simple.

Ensuite, vous pouvez ajouter un fichier `stg_customers.yml` pour documenter le modèle correspondant. Pour le moment, on va juste documenter le nom des colonnes, dans une prochaine quête, on rajouter des descriptions.

```
1 version: 2  
2  
3 models:  
4   - name: stg_customers  
5     columns:  
6       - name: customer_id  
7       - name: customer_name
```

Détails du fichier `stg_customers.yml` :

- `models` : Contient la liste des modèles documentés.
 - `name` : Le nom du modèle, correspondant au fichier SQL.
- `columns` :
 - `name` : Le nom de la colonne du modèle.

Une fois le modèle créé et documenté, il ne reste plus qu'à le configurer dans le fichier `dbt_project.yml`.

En général, la première partie de ce fichier est déjà bien configurée. Ce qui nous intéresse, c'est la section où il est indiqué `models:`.

Remplacez la partie `example` par `staging`, car c'est le modèle que nous venons d'ajouter.

```
models:  
  dbt_quest:  
    # Config indicated by + and applies to all files under models/example/  
    staging:  
      +materialized: view
```

La ligne `+materialized: view` signifie que les requêtes SQL généreront des **vues** plutôt que des **tables physiques**. Une vue est particulièrement adaptée pour la phase de **staging**, car elle permet de transformer les données de manière dynamique sans occuper d'espace supplémentaire dans la base de données.

Lorsqu'on créera un modèle de **mart** pour l'analyse, il suffira de l'ajouter sous la section appropriée et de modifier cette ligne en `+materialized: table` pour créer une table physique dans la base de données.

Vérification du modèle

Pour vérifier que le modèle `stg_customers` fonctionne correctement, exécutez la commande suivante dans votre terminal :

```
1 dbt run
```

Si tout se déroule comme prévu, vous devriez voir un résultat similaire à ceci :

```
-----
15:07:46 Found 1 model, 3 tests, 1 source, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
15:07:46
15:07:46 Concurrency: 1 threads (target='dev')
15:07:46
15:07:46 1 of 1 START sql view model my_dbt_db.stg_customers ..... [RUN]
15:07:46 1 of 1 OK created sql view model my_dbt_db.stg_customers ..... [SUCCESS 0 in 0.12s]
15:07:46
15:07:46 Finished running 1 view model in 0 hours 0 minutes and 0.20 seconds (0.20s).
15:07:46
15:07:46 Completed successfully
15:07:46
15:07:46 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
-----
```

❗ La commande `dbt run` exécute tous les modèles de votre projet DBT. Cela peut ne pas être optimal pour un projet contenant de nombreux modèles.

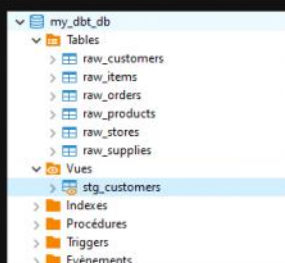
- Pour exécuter un seul modèle, utilisez la commande suivante :
`dbt run --select stg_customers.`
- Pour exécuter tous les modèles dans un dossier spécifique :
`dbt run --select models/staging/*`
- Vous pouvez également ajouter un **tag** dans le fichier `.yaml` correspondant au modèle, puis exécuter tous les modèles associés à ce tag. Cela est particulièrement utile dans les projets avec un grand nombre de modèles.
`dbt run --select tag:my_tag`

Si vous souhaitez exécuter un modèle en incluant ses dépendances en amont (modèles dont il dépend) ou en aval (modèles qui dépendent de lui), vous pouvez utiliser les opérateurs suivants :

- Pour exécuter un modèle et ses dépendances en amont :
`dbt run --select +stg_customers`
- Pour exécuter un modèle et ses dépendances en aval :
`dbt run --select stg_customers+`
- Pour exécuter un modèle avec à la fois ses dépendances en amont et en aval :
`dbt run --select +stg_customers+`

Ces options vous permettent d'exécuter uniquement les parties du projet nécessaires en fonction des relations entre les modèles.

En consultant votre base de données, vous constaterez qu'une vue a été créée :



Type de tests DBT

DBT propose plusieurs types de tests qui permettent de vérifier la qualité et l'intégrité des données.

1. Tests intégrés (built-in)

DBT propose plusieurs **tests intégrés** que vous pouvez ajouter directement dans vos fichiers YAML pour vérifier la qualité des données. Voici quelques exemples courants :

- **unique** : Vérifie que chaque valeur d'une colonne est unique.
- **not_null** : S'assure qu'une colonne ne contient pas de valeurs nulles.
- **accepted_values** : Vérifie que les valeurs d'une colonne appartiennent à un ensemble spécifique.
- **relationships** : Assure l'intégrité référentielle entre deux tables.

Exemple :

Dans le fichier `stg_customers.yml`, vous pouvez ajouter ces tests intégrés pour vérifier la qualité des colonnes `customer_id` et `customer_name`.

```
1 version: 2
2
3 models:
4   - name: stg_customers
5     columns:
6       - name: customer_id
7         tests:
8           - not_null
9           - unique
10      - name: customer_name
11        tests:
12          - not_null
```

❗ La version de DBT Core utilisée pour cette quête est une ancienne version qui est nécessaire pour fonctionner avec l'adaptateur MySQL.
Dans les nouvelles versions de DBT Core, les tests sont remplacés par des **data_tests**.

2. Tests personnalisés

Il est aussi possible de créer des tests sur mesure en écrivant des requêtes SQL spécifiques. Ces tests sont placés dans le dossier `tests`.

Exemple de test personnalisé :

Imaginons que vous souhaitez vérifier que la colonne `order_date` de la vue `stg_orders` que vous avez créé dans la quête précédente ne contient pas de dates futures.

❗ Si vous n'avez plus la vue de la quête précédente, vous pouvez trouver les 3 fichiers nécessaires [ici](#).
N'oubliez pas de lancer `dbt run` pour générer la vue `stg_orders`.

Vous pouvez créer un test SQL comme suit :

- Créez un fichier `order_dates_in_the_past.sql` dans le dossier `tests`.

```
1 with invalid_orders as (
2   select *
3   from {{ ref('stg_orders') }}
4   where ordered_at > current_date
5 )
6 select count(*)
7 from invalid_orders
8 having count(*) > 0
```

❗ Cette requête affiche le nombre de dates de commandes postérieures à la date du jour. Si le résultat est 0, la requête ne renverra aucun résultat, ce qui signifie que le test sera réussi.

Exécution des tests

Une fois que les tests sont configurés dans vos fichiers, vous pouvez les exécuter avec la commande suivante :

```
1 dbt test
```

Cette commande exécutera tous les tests définis dans les fichiers YAML ainsi que les tests personnalisés. Si un test échoue, DBT fournira un rapport détaillé pour vous permettre de corriger les problèmes.

```
Running with dbt=1.7.18
Registered adapter: mysql-1.7.0
Found 2 models, 5 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models

Concurrency: 1 threads (target='dev')

1 of 5 START test not_null_stg_customers_customer_id ..... [RUN]
1 of 5 PASS not_null_stg_customers_customer_id ..... [PASS in 0.04s]
2 of 5 START test not_null_stg_customers_customer_name ..... [RUN]
2 of 5 PASS not_null_stg_customers_customer_name ..... [PASS in 0.02s]
3 of 5 START test not_null_stg_orders_order_date ..... [RUN]
3 of 5 PASS not_null_stg_orders_order_date ..... [PASS in 0.05s]
4 of 5 START test order_dates_in_the_past ..... [RUN]
4 of 5 PASS order_dates_in_the_past ..... [PASS in 0.07s]
5 of 5 START test unique_stg_customers_customer_id ..... [RUN]
5 of 5 PASS unique_stg_customers_customer_id ..... [PASS in 0.03s]

Finished running 5 tests in 0 hours 0 minutes and 0.29 seconds (0.29s).

Completed successfully

Done. PASS=5 WARN=0 ERROR=0 SKIP=0 TOTAL=5
```

🕒 Comme pour les modèles, il est possible d'ajouter des arguments à la commande `dbt test` pour éviter de lancer tout les tests à chaque fois.

- Exemple : `dbt test --select order_dates_in_the_past`

⚠️ Si vous obtenez une erreur de parsing json, la raison peut en être une incompatibilité avec protobuf. Pour la résoudre, entrez la commande suivante dans bash :

```
pip install protobuf==4.25.5
```

Introduction

Dans cette quête, nous allons explorer la fonctionnalité **DBT Docs**, qui permet de générer une documentation interactive pour tous les modèles, sources, et tests dans un projet **DBT**. La documentation joue un rôle essentiel dans la compréhension et la maintenance des pipelines de données, facilitant la collaboration entre les équipes et améliorant la transparence des processus.

La commande `dbt docs` vous permet de visualiser tous les modèles, tests, et sources de données d'un projet sous forme d'une interface web. Vous pouvez naviguer à travers cette interface pour comprendre comment les différentes transformations sont reliées entre elles.

Génération de la documentation

Une fois que vous avez configuré vos modèles et sources, vous pouvez générer la documentation de votre projet en exécutant la commande suivante dans le terminal :

```
1 dbt docs generate
```

Cette commande va compiler toute la documentation basée sur les fichiers YAML (.yml) et SQL présents dans votre projet. Elle va également collecter des informations sur les relations entre les modèles et les sources, ainsi que sur les tests intégrés.

Ensuite, pour visualiser cette documentation sous forme de site web interactif, exécutez :

```
1 dbt docs serve
```

Cela va lancer un serveur local qui vous permettra d'explorer la documentation via un navigateur web.

Ajouter des descriptions aux modèles et aux sources

Il est essentiel d'enrichir vos fichiers YAML avec des descriptions détaillées des modèles et des colonnes pour améliorer la lisibilité et la compréhension du projet. Par exemple, dans le fichier `stg_customers.yml`, vous pouvez ajouter des descriptions des colonnes de la manière suivante :

```
1 version: 2
2
3 models:
4   - name: stg_customers
5     description: Customer data with basic cleaning and transformation applied, one row per customer.
6     columns:
7       - name: customer_id
8         description: The unique key for each customer.
9       - name: customer_name
10        description: The full name of the customer.
```

Les descriptions sont ensuite visibles dans l'interface **DBT Docs**, ce qui permet à tout utilisateur du projet de comprendre facilement le rôle de chaque modèle et colonne.

Il est également possible d'ajouter des descriptions pour chaque table dans le fichier `sources.yml`.

```
1 version: 2
2
3 sources:
4   - name: jaffle_shop
5     schema: my_dbt_db
6     description: E-commerce data for the Jaffle Shop
7
8   tables:
9     - name: raw_customers
10       description: One record per person who has purchased one or more items
```

Pour des descriptions plus détaillées, vous pouvez créer un fichier `store_id_description.md` contenant la description, puis y faire référence dans le fichier `.yml` de la table.

`store_id_description.md` :

```
1 {% docs store_id_description %}
2
3 This is the ID for the store.
4
5 It references the ID column from the stores table.
6
7 {% enddocs %}
```


stg_orders.yml:

```
1 version: 2
2
3 models:
4   - name: stg_orders
5     description: Order data with basic cleaning and transformation applied.
6     columns:
7       ...
8
9
10  - name: store_id
11    description: "{{ doc('store_id_description') }}"
12    tests:
13      - not_null
```

La description spécifiée dans le fichier `.md` devrait s'afficher correctement après avoir exécuté `dbt docs generate` et `dbt docs serve`.

Columns

COLUMN	TYPE	DESCRIPTION	CONSTRAINTS	TESTS	MORE?
order_id	text	The unique key for each order.		U N	>
customer_id	text	The key for the customer referencing the customer_id k...		N F	>
order_date	text	The date the order was placed.		N	>
store_id	text				^

Details

< >

Description

This is the ID for the store.

It references the ID column from the stores table.

Generic Tests

Not Null

Introduction

Dans **DBT**, en plus des modèles et des sources de données, il est possible d'importer des données statiques ou des tables de référence directement dans un projet à l'aide des **DBT seeds**. Ces tables sont particulièrement utiles pour intégrer des données constantes ou externes (telles que des listes de codes, des taux de conversion, ou encore des règles métiers) dans votre pipeline de transformation.

Dans cette quête, nous allons explorer le dossier `seeds`, comprendre son rôle et découvrir comment charger des fichiers CSV dans votre data warehouse à l'aide de cette fonctionnalité.

Qu'est-ce qu'un seed ?

Les **seeds** sont des fichiers CSV stockés dans le dossier `seeds` de votre projet **DBT**. Ces fichiers peuvent être chargés directement dans le **data warehouse** et traités comme des tables permanentes. Cela permet d'importer facilement des données statiques ou externes, et d'assurer une gestion centralisée de ces données.

Par exemple, vous pouvez utiliser un seed pour stocker :

- Une liste de codes ISO de pays.
- Des taux de conversion monétaires.
- Des catégories ou segments de clients pour les analyses.

Configuration et utilisation des seeds

Pour configurer un seed dans **DBT**, commencez par ajouter un fichier CSV dans le dossier `seeds` de votre projet. Par exemple, un fichier `customer_status.csv` pourrait contenir une liste des statuts de clients (bronze, silver, gold) avec le nombre de commandes requises pour atteindre chaque statut.

Une fois le fichier CSV ajouté dans le dossier `seeds`, vous pouvez le charger dans votre **data warehouse** avec la commande suivante :

```
1 dbt seed
```

```
Found 2 models, 14 tests, 1 seed, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
Concurrency: 1 threads (target='dev')

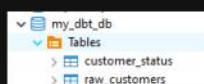
1 of 1 START seed file my_dbt_db.customer_status ..... [RUN]
1 of 1 OK loaded seed file my_dbt_db.customer_status ..... [INSERT 3 in 0.16s]

Finished running 1 seed in 0 hours 0 minutes and 0.41 seconds (0.41s).

Completed successfully

Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1
```

Cette commande importe les données du fichier CSV dans une table du data warehouse. Le nom de la table créée sera le même que celui du fichier CSV (sans l'extension `.csv`). Dans notre exemple, la commande créera dans une table appelée `customer_status`.



Exécution des seeds

Une fois que le seed est chargé, il est possible de l'utiliser dans vos modèles de transformation DBT en tant que source de données. Pour ce faire, vous pouvez vous référer à la table seed à l'aide de la fonction `ref` :

```
1 with status_data as (
2   select *
3   from {{ ref('customer_status') }}
4 )
5 select *
6 from status_data
```

Cela vous permet d'utiliser les données statiques dans vos modèles DBT pour des analyses ou des transformations.